INFINITE-OCR · GENERATIVE AI COURSEWORK · SYSTEM DESIGN

Architecture Report

How a PDF of any length becomes a Markdown document, checked tables and a report: every component, every rule, and the file and function where each one lives.

STATE OF THE CODE

22 October 2026. 59 Python files, about 14,000 lines, 141 unit tests.

WRITTEN FOR

Reading next to the code. Each section names the files and functions it describes, with line numbers taken from the working copy.

ENGINES

Sarvam Vision (₹0.50 per page) and Bodhan indic-ocr (₹0.20 per image). Vision models through OpenRouter for table checking.

UNITS

Rupees (₹) and metric. Line numbers shift when the code changes; function names do not.

Contents

How to use this report 4
- Conventions 4
- Two ways to read it 4

1 The system in one page 5
- 1.1 The two stages 5
- 1.2 Four rules the design follows 5
- 1.3 A run from the command line 6
- 1.4 What each component costs 6

2 Repository map 8
- 2.1 The source modules 8
- 2.2 History that is still in the tree 9
- 2.3 Running things 9

3 State on disk: files, manifest, cache, configuration 10
- 3.1 The folder of one document run 10
- 3.2 The manifest 10
- 3.3 The shared block format 11
- 3.4 The cache 11
- 3.5 Configuration 11

4 Starting a run 12
- 4.1 From the command line to a running job 12
- 4.2 prepare(): decide where, and refuse a mismatch 12
- 4.3 execute(): threads and the main loop 12
- 4.4 Stopping on purpose 13

5 Triage, page classes, rotation and languages 14
- 5.1 What triage records 14
- 5.2 The OCR mode: a switch, not a language 14
- 5.3 Page classes 14
- 5.4 Upright images: rendering and chunk PDFs 15

6 Routing and the task queue 16
- 6.1 The life of a task 16
- 6.2 The Router 16
- 6.3 How a claim works 17

7 Rate limits and retries 19
- 7.1 The limiter 19
- 7.2 Classifying every response 19

8 The Sarvam lane 21
- 8.1 The client 21
- 8.2 The loop 21
- 8.3 Submitting a job 21
- 8.4 Polling, downloading, ingesting, resuming 22

9 The page lane: Bodhan and Qwen 24
- 9.1 Backends 24
- 9.2 One page, start to finish 24
- 9.3 What the Bodhan client handles 24

10 Budget, cache, events and resuming 27
- 10.1 The budget 27
- 10.2 The cache 27
- 10.3 The events log 27
- 10.4 What can be paid twice 27
- 10.5 Resuming and starting over 27

11 Blocks and languages: what Stage 2 starts from 28
- 11.1 The block format 28
- 11.2 Language identification 28

12 Stage 2 in order 30
- 12.1 What assembly loads 30
- 12.2 The order of the work 30
- 12.3 The table record 31

13 Reading tables: the grid and the number checks 32
- 13.1 The table model 32
- 13.2 Parsing 32
- 13.3 Numbers in thirteen digit systems 32
- 13.4 The checks 32
- 13.5 What else is in tables.py 33

14 The free table check 34
- 14.1 The idea in four lines 34
- 14.2 The data structures 34
- 14.3 Step 1: comparison form and phantom columns 34
- 14.4 Step 2: the shape 34
- 14.5 Step 3: aligning the readings 35
- 14.6 Step 4: deciding each cell 35
- 14.7 The text layer must prove itself 35
- 14.8 Step 5: do not trust agreement 36
- 14.9 Questions and status 36

15 Rows and columns from ruling lines 38
- 15.1 Why not PyMuPDF's table finder 38
- 15.2 How the lines are found 38
- 15.3 Three corrections that were measured 38
- 15.4 Measured 39

16 Repairing a scrambled text layer 41
- 16.1 What is wrong with the layer 41
- 16.2 Learning the mapping 41
- 16.3 Result 41

17 Cut-out sheets 42
- 17.1 The cutter 42
- 17.2 The two sheet formats 42

18 The image check: a validator, not a corrector 43
- 18.1 Why a validator 43
- 18.2 The rule 43
- 18.3 Asking: one sheet at a time 44
- 18.4 Audits and the seam check 44
- 18.5 The model client 44
- 18.6 The pilot 45

19 Joining tables across pages 48
- 19.1 The signature 48
- 19.2 The score 48
- 19.3 Looking ahead 48
- 19.4 Merging and checking the seam 49
- 19.5 Measured 49

20 The page-text check 51
- 20.1 The steps 51
- 20.2 What the first measurement showed 51

21 Writing the outputs 54
- 21.1 doc.md 54
- 21.2 Tables, report and JSON 54
- 21.3 The three statuses 54

22 Modes, page classes and the estimate 56
- 22.1 Modes 56
- 22.2 The page classes in the pipeline 56
- 22.3 The estimate 57

23 The legacy path 60
- 23.1 What it does 60
- 23.2 Why it was replaced 60

24 The benchmark and the model pilot 61
24.1 The pieces 61
24.2 What the benchmark found 61
24.3 Reproducing 62

25 Tests 64

26 Results, decisions and corrections 65
26.1 The sample documents in all three modes 65
26.2 What the user decided 65
26.3 What was tried and rejected 66
26.4 Mistakes and corrections 66
26.5 What this phase cost 66

27 Limits and where to go next 67
27.1 Known limits 67
27.2 Left for later, by decision 67
27.3 The RTX 5070 67

Appendix A: Code map 69
src/ : the program 69
bench/ : the benchmark and pilots 72
tests/ : unit tests (no API calls) 73

Appendix B: Configuration 76

Appendix C: Glossary 77

How to use this report

The report is built to be read in a second window next to the code. It starts with the whole picture, then follows the data in the order the program handles it: Stage 1 reads the pages, Stage 2 turns the readings into a document.

Conventions

- **Code references.** A grey tag such as `lanes.py SarvamLane._submit_next L652` means: open that file and search for that name. The number after *L* is the line where the definition starts. The tags were generated from the source by a script, so a name that does not exist would have stopped the build.
- **Measured against estimated.** A number is measured (on the benchmark, the model pilot or the real sample documents) unless the text says "estimate". The place where it was measured is named, usually a file in `bench/`.
- **"Harm".** A correct reading changed to a wrong one. The rules of the table checker are built to keep this at zero. It is the word to remember for the whole of Stage 2.
- **Boxes.** A green box, *Read the code*, lists where to look. An amber box, *Decision*, records a choice and why it was made. A red box, *Limit*, says what the part cannot do.
- **Figures** sit on their own landscape pages and are the same drawings as in the online system map.

Two ways to read it

IF YOU WANT TO	READ
Understand how pages get read, safely and cheaply	Chapters 1 to 10 (Stage 1): the overview, the files, the run, triage, routing, rate limits, the two engine lanes, the budget, the cache and resuming after a crash.
Understand how tables are checked	Chapters 11 to 21 (Stage 2): blocks, the order of assembly, parsing, alignment, the free check, ruling lines, text-layer repair, cut-outs, the image check, joins, the page-text check and the outputs.
Know what was measured and why it was built this way	Chapters 22 to 27: modes and estimate, the legacy path, the benchmark, the tests, the results, the decisions, the mistakes and the limits.
Find a function quickly	Appendix A, the code map: every file with its classes and functions and their line numbers.

1 The system in one page

Input: a PDF of any length (born digital or scanned, Indic scripts and English). **Output:** `doc.md` (headings, paragraphs, figures, HTML tables), one file per table, and `report.md` saying what is certain and what is not. The two OCR engines are paid per page and rate limited, so the job is to read every page once, never lose or repeat a paid call, and spend model money only where it buys a correct answer.

1.1 The two stages

	STAGE 1: ENGINE RUN	STAGE 2: ASSEMBLY
Entry point	`run.py main L385`, class `run.py Run L84`	`assemble.py main L1194`, class `assemble.py Assembler L200` (called by Stage 1 at its end, or alone on a finished run)
Job	Get every page read by Sarvam Vision and, where useful, Bodhan, inside their rate limits; survive crashes; never pay twice	Turn per-page readings into one document; line up the two engines' tables, settle cells, join tables across pages, say how sure we are
Kind of code	Deterministic scheduling, threads, SQLite, HTTP clients	Deterministic checks first; a vision model only for what is left
Costs money	Yes: ₹0.50 per Sarvam page, ₹0.20 per Bodhan image	Only the image check (OpenRouter models), and only in Normal and Think
Writes	manifest.sqlite, raw engine results, per-page blocks, summary.json	doc.md, tables/, figures/, report.md, assembly.json
Repeat cost	₹0 (cache)	₹0 (cache, including model answers)

Figure 1 (after this chapter) draws the whole system. The two stages are separate on purpose. Stage 1 is the expensive part and the part that can break (network, rate limits, crashes), so it leaves everything it learned on disk. Stage 2 can then be run again, in another mode or with a changed rule, without reading a single page again.

1.2 Four rules the design follows

1. **The engines read; models check.** Sarvam and Bodhan are the trained readers. A general vision model was tested on 395 real questions and could not read what both engines missed (about 3 in 50). So a model is only ever asked to choose between readings the engines produced, and its answer counts only if it equals one of them.
2. **Cheapest evidence first.** For every table cell the order is: the readings agree; they differ only in spacing; a printed formula decides; a proven PDF text layer decides; punctuation only; and only then a model. The model sees a small cut-out, not the page.
3. **Every paid call is written down before it is made and kept after.** A job is recorded in the manifest before it is submitted; every result is stored under a key made from its content; every model answer is cached and added to a spend ledger. A crash, a Ctrl-C or a re-run cannot lose a result or pay for it twice (with the one exception in section 10.4).
4. **Say what is not certain.** Every table ends as *verified*, *repaired* or *needs review*, and the report lists the cells nothing could settle. A guess presented as a fact is the failure the whole of Stage 2 is built to avoid.

1.3 A run from the command line

```
python -m src.run samples/union-receipt-budget-2026-27-hindi.pdf --pages 90-101 -l hi-IN

union-receipt-budget-2026-27-hindi.pdf: 12 of 106 pages, mode normal, policy sarvam-primary, language hi-IN,
document type auto
Preflight (12 of 12 pages read, 2.1 s): 12 born-digital/unicode; about 12 pages with tables
  mode        OCR (Sarvam + Bodhan)       table checks            total         time
  fast                        ₹8.40                ₹0            ₹8.40      2–3 min
  normal                      ₹8.40         ₹0.12–3.60           ₹9–12      2–3 min <-
  think                       ₹8.40         ₹0.36–3.60           ₹9–12      2–4 min
  (cache hits cost nothing; no spending cap unless --max-cost-inr is given)
Start? [Y/n]
  ...
Done: 12/12 pages (complete)
Wall 94.3 s, 7.6 pages/min, cost ₹8.4
```

The estimate is the real output of `estimate.py render L193` for these pages. The last two lines are the measured Stage 1 summary of the run on 10 October (the table-check cost came on top, in Stage 2). Chapter 22 explains the estimate.

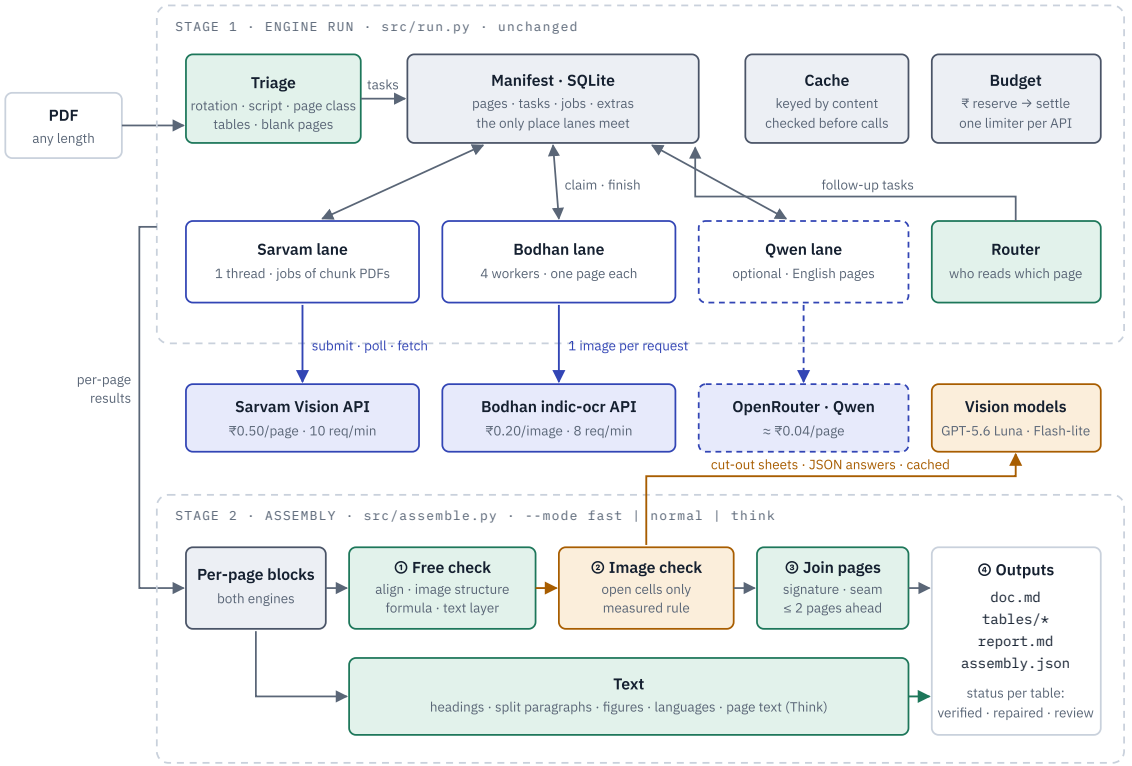
1.4 What each component costs

COMPONENT	KIND	COST	LIMIT
Triage, page classes, estimate, geometry, free table check, joins, page-text check, layer repair	Local code (PyMuPDF, NumPy, Pillow)	Free	CPU: about 0.5 to 1.5 s per table
Sarvam Vision `doc_ai.digitise`	Cloud OCR	₹0.50 per page	10 requests per minute per account; status polls count
Bodhan indic-ocr	Cloud OCR	₹0.20 per image	8 requests per minute (measured); 4 workers
Qwen (optional, English pages)	Cloud model, OpenRouter	about ₹0.04 per page	60 starts per 62 s
Image check models: GPT-5.6 Luna, Gemini 3.1 Flash-lite, Gemini 3.5 Flash-Lite	Cloud models, OpenRouter	about ₹0.015 per question per model	120 starts per 62 s, 8 in flight; US$10 ledger cap

READ THE CODE

- The CLI and the run loop: `run.py main L385`, `run.py Run.prepare L117`, `run.py Run.execute L196`.
- The assembly order: `assemble.py Assembler.run L980`.
- The modes as data: `modes.py MODES L57`. The settings that must match on a resume: `lanes.py Settings.fingerprint L71`.
- The project's own notes: `README.md` (commands and measured numbers) and `AGENT_BRIEF.md` (the brief and its known weaknesses).

FIGURE 1 **The whole system**



Lanes never call each other, and Stage 2 pays only for leftovers. The manifest coordinates the lanes, and the cache makes a re-run cost ₹0. Triage now also records a class for every page (born digital or scan). In Stage 2 every table first goes through the free check; only cells it cannot settle reach a vision model, as small cut-outs rather than whole pages. The mode (`--mode`) decides how much of this runs. `--table-qa legacy` keeps the previous path.

HOW TO READ IT

1. Follow the arrows left to right. A PDF enters **triage** (free, local), which writes one task per (page, engine) into the **manifest**. The manifest is the only place the lanes meet.
2. Each **lane** claims tasks from the manifest and calls its API through a rate limiter. The lanes never call each other. The **Router** adds follow-up tasks: a table Sarvam finds gets a Bodhan task; a page one engine failed goes to the other.
3. Per-page results go down into **Stage 2**: the free check, then the image check (amber: the only step that costs model money), then joins, then the outputs. Text blocks skip the table path and go straight to composition; in Think mode the two engines' page text is compared first.
4. Colours: green is local code (free), blue is a paid OCR engine, amber is a vision model, grey is stored state. Code: `run.py` (Stage 1), `lanes.py` (Router and lanes), `assemble.py` (Stage 2).

2 Repository map

One Python package (`src/`), a benchmark (`bench/`), unit tests (`tests/`), sample documents and two output folders. Python 3.11 in a local `.venv`. The project is not a git repository.

```
infinite_ocr/
├─ AGENT_BRIEF.md          the original brief and its list of known weaknesses
├─ README.md               commands, how each part works, measured numbers
├─ requirements.txt        pymupdf, sarvamai, openai, python-dotenv, requests (+ numpy, pillow, apted for bench/)
├─ .env / .env.example     API keys and tunables (never printed or committed)
├─ src/                    the program: 28 modules
├─ tests/                  141 tests, fake engines and a fake model, no API calls
├─ bench/                  synthetic tables with known truth, simulated and real engine runs, model pilots
├─ samples/                test PDFs: Union budget (Hindi), Marathi fiscal report, NCERT maths, Yojana (Telugu), ...
├─ cache/                  content-keyed results of every paid call (about 1,000 files)
├─ out/
│  ├─ docs/<name>-<sha8>/   one folder per document run (chapter 3)
│  ├─ openrouter_ledger.jsonl   every OpenRouter call: purpose, model, tokens, US$ and ₹
│  └─ estimates.jsonl           estimate against actual, one line per run
└─ docs/                   this report
```

2.1 The source modules

MODULE	LINES	WHAT IT IS	CHAPTER
`src/run.py`	476	The job engine and CLI: settings, triage thread, lanes, progress, summary, then assembly	4
`src/triage.py`	197	Free per-page checks before any API call	5
`src/page_class.py`	83	Scan or born digital, and the kind of text layer, per page	5, 22
`src/pdf_pages.py`	224	Upright rendering, rotation, chunk PDFs for Sarvam, coordinate transforms	5
`src/langid.py`	240	Language identification of OCR output (script, then function words)	5, 11
`src/lanes.py`	756	Settings, Budget, Context, Router, SarvamLane, PageLane, engine backends	6, 8, 9, 10
`src/manifest.py`	438	SQLite run state (pages, tasks, jobs, extras) and the events log	3, 6
`src/rate_limit.py`	414	Limiter, retry loop and response classifiers	7
`src/sarvam_digitise.py`	344	Sarvam Vision client: submit, poll, fetch	8
`src/bodhan_ocr.py`	452	Bodhan client: one image per call, with its quirks handled	9
`src/qwen_ocr.py`	343	Qwen reader of English pages through OpenRouter (Week 1 code)	9
`src/cache.py`	47	Disk cache keyed by sha256, atomic writes	10
`src/config.py`	170	Environment and constants (prices, limits, models)	3, B
`src/blocks.py`	210	One block format for both engines	11
`src/assemble.py`	1224	Stage 2: tables, joins, composition, outputs, report	12, 21, 23
`src/tables.py`	719	Table grid, HTML parsing, printed-formula checks, legacy fusion and joins	13, 23
`src/table_qa.py`	504	The free check: alignment, deciders, risk flags, status, questions	14
`src/table_geometry.py`	250	Ruling lines, rows and columns from the page image	15
`src/layer_repair.py`	146	Learns a scrambled font's glyph mapping	16
`src/table_cutouts.py`	172	Numbered cut-out sheets for the image check	17
`src/table_judge.py`	262	The image-check rule, audits, seam check	18

MODULE	LINES	WHAT IT IS	CHAPTER
`src/agent.py`	330	OpenAI-compatible model client with cache, ledger and JSON schemas	18
`src/table_join.py`	247	Signatures, column mapping, cut rows, seam checks	19
`src/text_qa.py`	234	Page-text check (Think)	20
`src/modes.py`, `src/estimate.py`	73 + 216	Fast, Normal, Think; the preflight and the estimate	22
`src/compare.py`, `src/run_week1.py`	304 + 232	Week 1: engine comparison scripts (shred, run, report). Not used by the pipeline	2.2

2.2 History that is still in the tree

- **Week 1 code** (`src/run_week1.py`, `src/compare.py`, the sequential and parallel loops in `src/bodhan_ocr.py` and `src/qwen_ocr.py`, `pdf_pages.py` **`shred_pdf`** `L164`, the older helpers in `src/rate_limit.py`): the first comparison of Sarvam, Bodhan and Qwen on a few pages. The pipeline reuses the clients and the limiter from it; the rest is left for reference.
- **The legacy table path** (`--table-qa legacy`): the first version of Stage 2, kept so results can be compared. Chapter 23.
- **Placeholders:** the local Bodhan backend (`lanes.py` **`BodhanLocalBackend`** `L279`) raises `NotImplementedError`; it is the hook for the RTX 5070 (chapter 27).

2.3 Running things

TASK	COMMAND
OCR a PDF	`python -m src.run file.pdf [-l hi-IN] [--mode fast|normal|think] [--doc-type auto|digital|scanned] [--pages 1-30] [--max-cost-inr 15] [--resume|--fresh]`
Assemble a finished run again	`python -m src.assemble out/docs/<name>-<sha8> [--mode think] [--no-agent] [--table-qa legacy]`
All tests (no API calls)	`python -m unittest discover -s tests -t .`
Benchmark (free unless stated)	`python -m bench.run_bench --set v2 --checker standard · bench.geometry_eval · bench.calibrate · bench.pilot --dry-run`

3 State on disk: files, manifest, cache, configuration

Everything the program knows about a run is on disk, so any run can be stopped, resumed, assembled again or inspected with a text editor and the `sqlite3` tool. Nothing important lives only in memory.

3.1 The folder of one document run

The folder is `out/docs/<slug>-<sha8>/`: a slug of the file name (at most 40 characters, see `run.py slug L74`) and the first 8 characters of the SHA-256 of the PDF's bytes (`run.py file_sha256 L66`). Two different files never share a folder, and the same file always returns to its own.

PATH	WRITTEN BY	HOLDS
`manifest.sqlite (+ -wal, -shm)`	Stage 1 (`manifest.py Manifest L86`)	Pages, tasks, Sarvam jobs, paid extras and run metadata. The only place the lanes meet.
`events.jsonl`	both stages (`manifest.py EventLog L67`)	One JSON object per line: page done, outcome, job submitted, steal, fallback, lane dead, assembled. Metrics and the report come from it.

`sarvam/pages/page_NNNNN.json`
`bodhan/pages/…`
`qwen/pages/…`
	lanes (`lanes.py write_json L105`)	The raw result of one engine for one page, with the render's size, dpi and the transform from PDF points to pixels.

`sarvam/jobs/<job_id>.zip`	`lanes.py SarvamLane._complete L731`	The ZIP Sarvam returned for a job, kept as downloaded.
`blocks/page_NNNNN.<engine>.json`	`lanes.py Context.save_page_outputs L158`	The page's blocks in the shared format (chapter 11), each with a language tag.
`md/page_NNNNN.<engine>.md`	same	A readable Markdown preview of one engine's page (`blocks.py page_markdown L194`).
`summary.json`	`run.py Run._summary L276`	Pages done, wall time, cost per engine, cache hits, 429s, retries, Sarvam jobs, event counts.
`doc.md · tables/ · figures/ · report.md · assembly.json`	Stage 2 (`assemble.py Assembler.run L980`)	The results (chapter 21).

Two files are shared by all runs: `cache/` (section 3.4) and `out/openrouter_ledger.jsonl` (every OpenRouter call with its purpose, model, tokens, US$ and ₹; `agent.py Agent._record L153`). The ledger is how the US$10 cap is enforced across runs: `agent.py Agent.spent_usd L134` adds it up before every call.

3.2 The manifest

SQLite in WAL mode behind one shared connection and one lock (`manifest.py Manifest L86`). The rules at the top of `src/manifest.py` matter more than the tables: *no network call inside a transaction*, and *result files are written before the row that points to them*. A crash can therefore leave a file without a row (harmless) but never a row without its file.

TABLE	KEY	COLUMNS THAT MATTER
meta	key	`fingerprint` (the settings, section 4.2), `source` (the PDF's path), `sha256`.
pages	page	One row per triaged page: `rotate`, `blank`, `text_chars`, `script`, `script_share`, `legacy_font`, `language` (the code sent to Sarvam), `table_bboxes` (JSON), `ocr_mode` (indic or en) and `mode_source`, `lang_hint`, `detected` (languages found in each engine's output), `mode_retried`, and since October `render`, `text_layer`, `image_cover` (page classes).
tasks	(page, engine)	`state` (pending, inflight, queued, done, failed, skipped, stolen), `priority`, `attempts`, `reason`, `job` (Sarvam job row), `result_path`, `latency_s`, `tokens`, `cost_inr`, `cache_hit`, `degraded` (how a Bodhan request was altered).
jobs	id	A Sarvam job: `job_id` (from the API, empty until submitted), `state` (submitting, submitted, done, failed, abandoned), `language`, `files` (JSON: the page numbers in each chunk PDF), `n_pages`, `status`, `error`, three timestamps.
extras	id	Paid calls that are not page tasks: `kind` (table_image_check, table_audit, table_seam_check, bodhan_table_reread, vlm_tiebreak, sarvam_mode_retry …), `page`, `cost_inr`, `detail` (JSON).

Total spend of a run is `SUM(tasks.cost_inr) + SUM(extras.cost_inr)` (`manifest.py Manifest.spent_inr L375`). A new column is added with an entry in `manifest.py MIGRATIONS L55`, which is applied to older databases when they are opened. That is the rule for any schema change: add a migration, never edit the schema alone.

> **DECISION: ONE SQLITE FILE AS THE MEETING POINT**
>
> The lanes never call each other. Triage writes tasks; each lane claims tasks, writes results and finishes them; the Router adds follow-up tasks. Because every claim is one transaction behind one lock (`manifest.py Manifest._tx L107`), two lanes cannot take the same page, and the state of a run can be read at any moment with a SQL query. A message queue or in-memory shared state would have needed its own crash recovery; SQLite gives that for free.

3.3 The shared block format

Both engines return different JSON. Both are turned into a list of `blocks.py Block L85` objects: `page, engine, order, kind, level, raw_label, bbox, text, conf, lang`. The `bbox` is four numbers between 0 and 1 in the *upright* page frame, so a box from Sarvam and a box from Bodhan can be compared directly (chapter 11).

3.4 The cache

`src/cache.py` is 47 lines. A key is the SHA-256 of a payload plus three strings (`cache.py make_key L13`); the value is a file named by the key in `cache/`, written atomically (`cache.py put L37`: temp file, then rename). Three families of keys exist, and the differences explain what a re-run costs:

WHAT	PAYLOAD BYTES	THE THREE STRINGS	DEFINED IN
Bodhan page	the rendered page image	`"indic-ocr"`, the model name, `"html|4096"` (table format and max tokens)	`lanes.py PageLane._process L437`, `lanes.py BodhanAPIBackend.cache_extra L266`
Sarvam page	`"<file sha256>|p<page>|r<rotation>"`	`"sarvam-vision-page"`, `"digitise"`, `"<language>|md"`	`lanes.py SarvamLane._page_key L617`
Model answer	the full message list as JSON	`"agent"`, the model, `"<purpose>|<base url>|<JSON schema>"`	`agent.py Agent._chat L168`

> **LIMIT: THE SARVAM KEY CONTAINS THE LANGUAGE CODE**
>
> The language is part of the Sarvam page key, so reading the same page with `-l hi-IN` and then with the default `auto` misses the cache and pays ₹0.50 again. This was measured on 22 October: every Indic code gives byte-identical output, so the exact code only causes re-billing. During the Think-mode runs it cost ₹6.50 on one document. Always repeat the original `-l` when re-running, or check the cache first. A fix (normalising Indic codes in the key, with a fallback to the old key) has been proposed but not made.

3.5 Configuration

`src/config.py` loads `.env` and defines every limit, price and model name as a constant that an environment variable can override. Keys are read once and never printed. The full list is in appendix B; the ones that shape behaviour are:

- **Rates and prices:** `config.py SARVAM_RATE_PER_MIN L124` (10), `config.py BODHAN_RATE_PER_MIN L120` (8), `config.py SARVAM_INR_PER_PAGE L147` (0.50), `config.py BODHAN_INR_PER_PAGE L148` (0.20), `config.py INR_PER_USD L149` (85).
- **Rendering:** `config.py RENDER_DPI L128` (200) and `config.py RENDER_MAX_LONG_PX L129` (3000) for the images sent to Bodhan.
- **Models:** `config.py TABLE_JUDGE_SINGLE L142` (GPT-5.6 Luna) and `config.py TABLE_JUDGE_MODELS L143` (Gemini 3.1 Flash-lite, then Luna) for the image check; Think's third judge is in `modes.py THIRD_JUDGE L32`.
- **Model traffic:** `config.py AGENT_MAX_INFLIGHT L138` (8) and `config.py AGENT_STARTS_PER_MIN L139` (120); the OpenRouter budget `config.py OPENROUTER_BUDGET_USD L135` (US$10).

4 Starting a run

`src/run.py` (476 lines) is the conductor. It checks the input, decides where the results go, starts a triage thread and the engine lanes, waits until nothing is left to do, writes a summary and hands over to Stage 2. It contains no OCR logic of its own.

4.1 From the command line to a running job

1. `run.py parse_args L351` reads the options. The ones that change what is read: `--language` (a hint, default `auto`), `--mode`, `--policy` (overrides the mode's routing), `--doc-type`, `--pages 120-135,200-210` (`run.py parse_pages L336` expands it), `--max-pages`, `--max-cost-inr`, `--dpi`, `--table-format`, `--chunk-pages`, `--files-per-job`, `--max-open-jobs`, `--bodhan-workers`, `--no-steal`, `--qwen`. The ones about the run: `--resume`, `--fresh`, `-y`, `--no-assemble`, `--no-agent`, `--no-reocr`, `--table-qa legacy|standard`.
2. `run.py main L385` checks the language (a code Sarvam knows: `langid.py SARVAM_LANGUAGES L28`), builds a `lanes.py Settings L55` and a `run.py Run L84`, and calls `run.py Run.prepare L117`.
3. The estimate is printed (chapter 22) and, on a terminal, `Start? [Y/n]` is asked unless `-y` was given.
4. Two Ctrl-C handlers are installed (section 4.4), `run.py Run.execute L196` runs the job, `run.py print_summary L322` prints the result, and if any page is done and `--no-assemble` was not given, an `assemble.py Assembler L200` is created and run with the same mode.

4.2 prepare(): decide where, and refuse a mismatch

`run.py Run.prepare L117` does the checks that must pass before any page is touched:

- The folder is `out/docs/<slug>-<sha8>`. If a manifest already exists there, `--resume` or `--fresh` is required. `--fresh` deletes the manifest, the events log and the result folders, but **not the cache**, so a fresh run of the same pages costs nothing.
- **The fingerprint.** `lanes.py Settings.fingerprint L71` returns the settings that change outputs: language, policy, dpi, table format, the Bodhan model name, Sarvam's output format and the Qwen flag (and `doc_type`, only when it is not `auto`, so runs made before page classes still resume). It is stored in the manifest's meta table. A resume with different settings stops with the differences listed, because pages read under one setting and pages read under another would not be comparable.
- The page selection is checked against the PDF's page count; an encrypted PDF is refused.
- A `lanes.py Budget L85` is created with the cap from `--max-cost-inr` (none by default) and the amount already spent, a `lanes.py Context L112` and a `lanes.py Router L180`. Missing API keys for the engines the policy needs are reported together.

Policies and what they create

POLICY (`--POLICY`)	ENGINES	TASKS AT TRIAGE	USED BY
sarvam-primary (default)	Sarvam, Bodhan	Sarvam on every non-blank page; Bodhan, priority 1, on every page that triage or Sarvam saw a table on	Fast, Normal
both	Sarvam, Bodhan	Both engines on every non-blank page	Think
sarvam-only	Sarvam	Sarvam on every non-blank page	cheapest OCR
bodhan-only	Bodhan	Bodhan on every non-blank page	a Sarvam outage

The mapping from mode to policy is the `policy` field of `modes.py Mode L47`; the Router's code is `lanes.py Router.initial_tasks L194` (chapter 6).

4.3 execute(): threads and the main loop

`run.py Run.execute L196` is the only place threads are created:

THREAD	COUNT	DOES
main	1	The loop below: waits, prints progress, decides when to stop

THREAD	COUNT	DOES
triage	1	`run.py Run._triage L177`: for every selected page not yet in the manifest, runs `triage.py triage_page L135` and inserts the page and its tasks (`manifest.py Manifest.add_page L136`). Sets `ctx.triage_done` at the end
sarvam	1	`lanes.py SarvamLane.run L569`: every Sarvam call is made here
bodhan	`--bodhan-workers`, default 4	`lanes.py PageLane._worker L409`: one page at a time each
qwen	4, only with `--qwen`	`lanes.py PageLane L365` with the Qwen backend

The triage thread runs *while* the lanes work, so Sarvam can start its first job as soon as 50 pages are ready, without waiting for triage of a 1,000-page book to finish (`lanes.py SarvamLane._submit_next L652` holds a partial job back only while triage is still running). PyMuPDF is not thread-safe, so every thread opens its own copy of the document (`lanes.py Context.doc L140`).

Order of events in `run.py Run.execute L196`:

1. On a resume, tasks that were *inflight* when the last run stopped go back to *pending* (`manifest.py Manifest.requeue_inflight L339`).
2. Lanes are created. Bodhan's lane may **steal** from Sarvam's queue only when the policy is sarvam-primary and stealing is on; the backlog it needs before it steals is `per_job × max_open_jobs` = 50 × 3 = 150 pages.
3. The triage thread and all lanes start.
4. Loop every 0.2 s until one of these ends it: a hard stop (second Ctrl-C); **done**: triage finished, no active tasks, no open Sarvam jobs and no busy lane; a stop or exhausted budget with nothing in flight; or every needed lane dead. A progress line is printed every 10 s (`run.py Run._progress L261`).
5. `ctx.done` is set, lanes are joined (30 s each), `run.py Run._summary L276` writes `summary.json`, and the manifest is closed.

The exit code is 0 for a complete run, 3 when the budget stopped it and 1 otherwise.

4.4 Stopping on purpose

The first Ctrl-C sets `ctx.stop`: no lane takes new work (`lanes.py Context.accepting L151` turns false), requests in flight finish, state is saved. A second Ctrl-C sets `ctx.hard_stop` and leaves at once. Either way `--resume` continues: it re-attaches to Sarvam jobs that were already submitted instead of paying for them again (section 8.4). The same mechanism is what `--max-cost-inr` uses: when the budget is exhausted, `ctx.budget_exhausted` makes `accepting()` false.

READ THE CODE

- Whole life of a run: `run.py Run.prepare L117`, then `run.py Run.execute L196`, then `run.py Run._summary L276`. About 150 lines.
- What the lanes share: `lanes.py Context L112` (stop, hard_stop, done, triage_done, budget_exhausted, alive per engine, the manifest, the events log, the budget).
- What gets written at the end: `run.py Run._summary L276` and `run.py print_summary L322`.

5 Triage, page classes, rotation and languages

Before any money is spent, every page is looked at by free code (PyMuPDF only). The result decides which engine reads the page, in which mode, how the page must be turned to be upright and, later in Stage 2, whether its text layer can be trusted. Triage takes a few milliseconds per page.

5.1 What triage records

`triage.py triage_page L135` returns a `triage.py PageFlags L79` row, which `manifest.py Manifest.add_page L136` stores in the *pages* table together with the page's tasks. Steps, in order:

1. **Text and scripts.** The text layer is read with `page.get_text("text")` and every letter is assigned a script by `triage.py char_script L66` (the Unicode blocks of 12 Indic and related scripts in `triage.py SCRIPT_RANGES L26`, plus Latin and "Latin1" for accented Latin letters). `text_chars` is the number of letters; `script` the dominant one; `script_share` the share of letters in the script expected for the language (not a correctness score).
2. **Legacy fonts.** A page is flagged `legacy_font` if any font name matches `triage.py LEGACY_FONT_RE L55` (DV-TT, Kruti Dev, Shivaji, Walkman, Chanakya, APS-DV, Akruti, Bamini, Vanavil, Nudi, and similar) or, in an Indic document, if more than 20% of the letters are accented Latin (a legacy font maps Indic glyphs onto Latin-1 code points). The text layer of such a page is gibberish and is never used as evidence.
3. **OCR mode** (next section).
4. **Language hint.** If the layer is clean and has at least 30 letters, `langid.py detect L174` gives the shares of languages in it, used later as a prior.
5. **Table hint.** PyMuPDF's `find_tables()` boxes are stored as `table_bboxes`. This is only a hint to schedule Bodhan's second reading on the page. It is unreliable as a reader of table shape (it reports 55 × 23 for a 9-column Union page, because it counts invisible white boxes; section 15.1), and nothing else depends on it.
6. **Blank page.** No letters, no table boxes and, at 18 dpi, fewer than 0.1% dark pixels (`triage.py _is_blank L128`). A blank page gets `skipped` tasks and is never sent.
7. **Rotation.** `pdf_pages.py upright_rotation L99`: the direction in which most characters run decides how many extra quarter turns make the text run left to right. Budget books print wide tables sideways. For a scan with no text layer the answer is 0.
8. **Page class** (section 5.3).

5.2 The OCR mode: a switch, not a language

A measurement made on 10 October changed the design: on a page with English, Tamil, Hindi and Marathi, every Indic language code given to Sarvam, and no code at all, produced byte-identical output. Only `en-IN` changed the result, and it *corrupted Indic digits* (२०२१ became २0२१). So the "language" is really an Indic/English switch, and triage only has to decide between the two, leaning to Indic because reading English in Indic mode does no harm:

CONDITION ON THE TEXT LAYER	MODE	SOURCE
Legacy font, or at least 10 Indic letters	indic	text_layer
At least 50 Latin letters and Indic letters ≤ 1% of them	en	text_layer
No usable text and the user said `en-IN`	en	user (checked after OCR)
Anything else	indic	user, or default

The code sent to Sarvam is `en-IN` in English mode, else the user's code, else none (field omitted). The actual languages are found *after* OCR, per block, from the text (section 11.2). A safety net sits in `lanes.py SarvamLane._ingest L622`: a page read in English mode whose output has at least 3% Indic letters is read once more in Indic mode (the extra is logged as `sarvam_mode_retry`).

5.3 Page classes

`page_class.py classify L67` gives every page two labels, from two quick PyMuPDF facts (the largest placed image's share of the page, `page_class.py image_cover L49`, and the count of visible and invisible characters, `page_class.py text_counts L55`, which reads the render mode of each text span):

LABEL	VALUES	RULE
render	vector · raster	raster if one image covers at least 80% of the page and fewer than 20 characters are visible (`page_class.py RASTER_COVER L29`, `page_class.py MIN_TEXT L30`); otherwise vector
text_layer	unicode · legacy · hidden_ocr · none · ignored	at least 20 visible characters: unicode, or legacy if triage flagged a legacy font; else at least 20 invisible characters (render mode 3 or fully transparent) is hidden_ocr, someone else's OCR over a picture; else none. `ignored` only with `--doc-type scanned`

The class is stored in the manifest (columns `render`, `text_layer`, `image_cover`, via `manifest.py MIGRATIONS L55`). For a run triaged before page classes existed, `assemble.py Assembler._load L237` classifies the page on the spot. The class replaced one test that did both jobs (`text_chars < 20` meant scanned), which took a scan with hidden OCR for born digital. On the sample PDFs the classes come out as shown below; the Union budget is the mixed case that motivated per-page classes.

SAMPLE	PAGES	CLASSES (RENDER / TEXT LAYER)
Union budget 2026-27 (Hindi)	106	vector/unicode 67 · vector/legacy 19 · vector/none 2 · raster/none 18
NCERT Class 12 maths ch. 6	41	vector/legacy 41
Marathi fiscal report	8	vector/legacy 5 · vector/unicode 3
Yojana (Telugu), March 2025	76	vector/unicode 72 · vector/none 3 · raster/none 1
Economic Survey ch. 4 · Patrick Deneen · Ponniyan Selvan	57 · 8 · 8	vector/unicode on every page

5.4 Upright images: rendering and chunk PDFs

Both engines must see the page the way a reader does, and their boxes must land in the same frame so they can be compared. `src/pdf_pages.py` does this:

- `pdf_pages.py render_page L71` renders a 1-based page to PNG at the requested dpi (200 for Bodhan), turned by the page's extra rotation, and returns, together with the image, the `transform` from unrotated PDF points to image pixels. The zoom is capped so the long side is at most 3,000 pixels.
- `pdf_pages.py build_chunk_pdf L131` builds the small PDFs sent to Sarvam (at most 10 pages each): the chosen pages are copied, every page's MediaBox is set equal to its CropBox (`pdf_pages.py pdf_cropbox L125` converts between PDF's bottom-left and PyMuPDF's top-left coordinates) and the extra rotation is applied. Sarvam therefore renders exactly what Bodhan sees, and both engines' bounding boxes can be normalised to 0–1 in one frame (`blocks.py _norm L107`).
- `pdf_pages.py page_transform L59` computes the same transform without rendering. Stage 2 uses it to cut table crops and to read the PDF's own words inside a box (`assemble.py Assembler._layer_text L420`).

READ THE CODE

- All of triage: `triage.py triage_page L135` (about 70 lines) and `triage.py PageFlags L79`.
- Page classes: `src/page_class.py` (83 lines), tested in `tests/test_page_class.py` on small synthetic PDFs (digital, scan, scan with hidden OCR text, legacy font, user override).
- Rendering and rotation: `pdf_pages.py render_page L71`, `pdf_pages.py upright_rotation L99`, `pdf_pages.py build_chunk_pdf L131`, tested in `tests/test_pdf_pages.py`.

6 Routing and the task queue

A **task** is one (page, engine) pair. The Router decides which tasks exist; the manifest hands them out one at a time and keeps their state. There is no other queue.

6.1 The life of a task

Figure 2 (after this chapter) shows every transition.

STATE	MEANING	SET BY
pending	Waiting to be claimed	manifest.py Manifest.add_page L136, manifest.py Manifest.ensure_task L191, manifest.py Manifest.release L305, manifest.py Manifest.retry_or_fail L291
queued	A Sarvam page inside a submitted or submitting job	manifest.py Manifest.claim_sarvam L209
inflight	A Bodhan or Qwen worker is reading it	manifest.py Manifest.claim_page_task L234
stolen	A Sarvam task that Bodhan took over from the tail of Sarvam's queue	manifest.py Manifest.claim_page_task L234
done	Result saved; blocks written	manifest.py Manifest.finish L278
failed	Attempts used up, or the lane is dead	manifest.py Manifest.retry_or_fail L291, manifest.py Manifest.fail_lane L322
skipped	Blank page	lanes.py Router.initial_tasks L194

6.2 The Router

lanes.py Router L180 is 60 lines and holds every routing decision:

WHEN	CONDITION	TASK CREATED	FUNCTION
Triage	Blank page	every engine: skipped	lanes.py Router.initial_tasks L194
Triage	Sarvam alive	sarvam pending, priority 0	same
Triage	Sarvam is dead and policy is not sarvam-only	bodhan pending, priority 2 (a fallback)	same
Triage	Policy both or bodhan-only	bodhan pending, priority 0	same
Triage	Policy sarvam-primary and the page has table boxes	bodhan pending, priority 1 (second reading of the table)	same
Triage	English page and --qwen	qwen pending	same
Sarvam finished a page	It contains a table block (triage missed it)	bodhan pending, priority 1	lanes.py Router.after_sarvam_page L213
A page failed	Sarvam failed and Bodhan is alive	bodhan pending, priority 2	lanes.py Router.on_page_failed L219
A page failed	Bodhan failed a page it had stolen	the Sarvam task reopened	same (via manifest.py Manifest.reopen_stolen L313)

WHEN	CONDITION	TASK CREATED	FUNCTION
A lane stops	Bad key, no credit, crash	its open tasks fail; each page goes to the other engine. For Sarvam without credit (HTTP 402) a warning says that tables will need review and to add credit and run with `--fresh`, because failed pages are not retried by `--resume`	`lanes.py Router.on_lane_dead L226`

Which engines a policy uses at all is in `lanes.py Router.engines L185`. The reason Bodhan reads only table pages in the default policy is cost: text-only pages cost Sarvam's ₹0.50 and nothing more. Measured on the samples, Bodhan read 3 of 8 pages of the Economic Survey window, 1 of 8 of Yojana and 5 of 13 of NCERT.

6.3 How a claim works

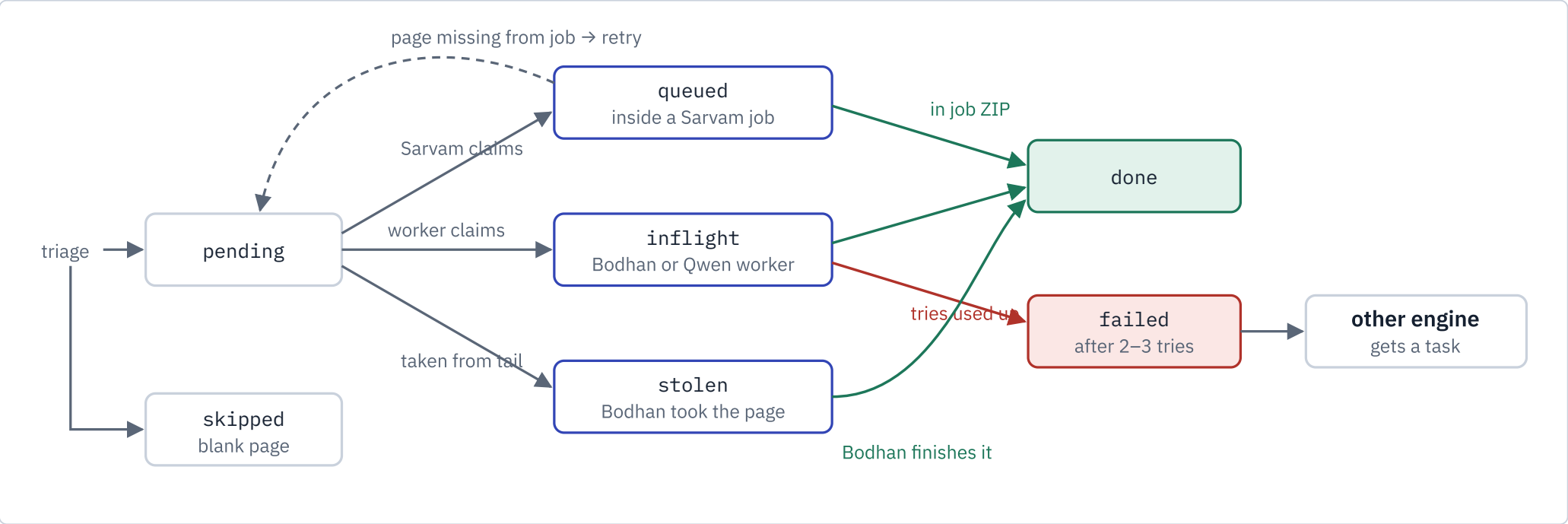
Each claim is a single transaction in `manifest.py Manifest L86`, so it cannot be interleaved with another:

- `manifest.py Manifest.claim_sarvam L209` finds the first pending Sarvam task (highest priority, lowest page), reads that page's language, and takes up to `max_pages` pending pages *of that same language* (a job has one language), marking them *queued*. The cap is `per_job` = `chunk_pages` × `files_per_job` = 10 × 5 = 50.
- `manifest.py Manifest.claim_page_task L234` gives a per-page worker its own pending task first. Only when it has none, and the engine it may steal from has more pending pages than the threshold, does it take that engine's *last* pending page (the one Sarvam would reach last), mark Sarvam's task *stolen* and create or update its own as *inflight*.
- `manifest.py Manifest.retry_or_fail L291` counts an attempt and returns the task to pending until `max_attempts` is reached: 2 for Sarvam (`lanes.py SarvamLane L543`), 3 for the page lane (`lanes.py PageLane L365`).
- `manifest.py Manifest.release L305` returns tasks to pending *without* counting an attempt (a budget stop, a 429 storm, a crash before submission).

READ THE CODE

- All claim and state logic is in `src/manifest.py`, lines 186 to 392 (the SQL is short and worth reading in full).
- Tests: `tests/test_blocks_manifest.py` (claims, steals, retries, resume) and `tests/test_engine.py` (a whole run with fake engines, crash and resume, budget stop, steals).

FIGURE 2 **Life of a (page, engine) task**



Nothing is lost on failure. A Sarvam page missing from its job goes back to pending for a new job; after two misses, or three failed Bodhan attempts, the Router gives the page to the other engine. If Bodhan fails a page it stole, Sarvam's task reopens.

HOW TO READ IT

1. Every page starts *pending* for each engine that will read it; a blank page is *skipped*.
2. Sarvam puts up to 50 pages of one language into a job (*queued*). A Bodhan or Qwen worker marks one page *inflight*. When Sarvam has a backlog of more than 150 pages, an idle Bodhan worker takes a page from the tail of Sarvam's queue (*stolen*).
3. Green arrows end in *done*. The dashed arrow is a page missing from a job's ZIP: it goes back to *pending* for a new job. The red arrow is attempts used up (2 for Sarvam, 3 for the page lane): the task is *failed* and the Router creates a task for the other engine.

7 Rate limits and retries

Every API call, from every thread, goes through a `rate_limit.py Limiter L34` and `rate_limit.py call_with_retry L228`. There is one limiter per API, shared by all threads that use that API. `src/rate_limit.py` is where "never exceed the quota, never hammer after a 429, never retry what cannot succeed" is implemented once.

7.1 The limiter

`rate_limit.py Limiter L34` enforces four things at once:

1. at most `max_starts` request starts in any window of `window_s` seconds (a sliding window of recent start times);
2. a minimum spacing between starts (`min_spacing_s`), so a server with a fixed window never sees a burst;
3. at most `max_inflight` calls at the same time (a semaphore);
4. a lane-wide pause: `rate_limit.py Limiter.pause_for L104` makes every thread that uses the limiter wait. This is how a 429's `Retry-After` is honoured by all workers of a lane, not just the one that was told.

A caller uses `with limiter.slot():` (`rate_limit.py Limiter.slot L92`), which takes the semaphore, waits for permission to start, runs the call and releases. The window is 62 s, not 60, as a margin over the servers' 60-second window.

API	WHERE CREATED	STARTS PER 62 S	SPACING	IN FLIGHT	WHY
Sarvam Vision	`sarvam_digitise.py sarvam_limiter L74`	10	3 s	1	10 requests a minute per account; submits, status polls and download-URL calls all count
Bodhan	`bodhan_ocr.py bodhan_limiter L65`	8	8 s (60 / 8 + 0.5)	4	Measured on 10 October: the 9th request in a minute gets `429, Retry-After: 42`
Qwen (OpenRouter)	`lanes.py QwenBackend L298`	60	none	4	Optional lane
Image-check models	`agent.py Agent L98` (created in `__init__`)	120	none	8	Raised from 30 and 2 after measurement: with 2 in flight a Union-budget assembly took 8 to 9 minutes; now 44 to 57 s

7.2 Classifying every response

An HTTP result or an exception is turned into an `rate_limit.py Outcome L117` with a `rate_limit.py Verdict L108`:

VERDICT	WHEN	WHAT CALL_WITH_RETRY DOES
OK	2xx	returns the result
RATE_LIMITED	429 whose body does not mention money	pauses the whole limiter for `Retry-After` (60 s if absent) plus 0.5 to 2 s of jitter, then retries
RETRY	408, 425, any 5xx, a timeout or connection error	waits $\min(120,\ 2 \cdot 2^{attempt-1})$ seconds, times a random 0.8 to 1.2, then retries
DEGRADE	a client-defined case (Bodhan's null body, token overflow)	raises `rate_limit.py DegradeError L134` so the caller can change the request
FATAL	401, 402, 403, other 4xx, an unknown exception, a 429 that mentions credit, balance, billing, payment, top-up	raises `rate_limit.py FatalError L130` at once. Retrying cannot help, and the lane stops (`credit_exhausted`, `http_401` ...)

The classifiers are `rate_limit.py classify_status L186` and `rate_limit.py classify_http L212`; the engines add their own (`bodhan_ocr.py classify_bodhan L100`, `sarvam_digitise.py classify_submit L106`). A call is tried at most 5 times (`rate_limit.py MAX_RETRIES L28`); after that `rate_limit.py RetriesExhausted L138` is raised. Every non-OK outcome is reported

to the caller's `on_outcome`, which writes an `outcome` line to events.jsonl (the 429 and retry counts in the summary come from these lines).

DECISION: THE CLIENTS SWITCH OFF THEIR OWN RETRIES

Both the Sarvam SDK and the OpenAI-compatible client resend on 429, 408, 409 and 5xx by default (twice). A resent Sarvam `digitise` call can start a *second paid job*, and hidden retries would also bypass the limiter. So all clients are created with `max_retries=0` (`sarvam_digitise.py NO_SDK_RETRY L50`, and in `agent.py Agent.client L125` and `lanes.py QwenBackend L298`) and every retry decision is made in one place.

READ THE CODE

- `rate_limit.py Limiter L34`, `rate_limit.py classify_status L186`, `rate_limit.py call_with_retry L228`: about 200 lines in total.
- Tests: `tests/test_rate_limit.py` uses an injected clock and sleep, so timing is tested without waiting.
- The older `TokenBucket`, `RateLimiter` and `call_with_429_retry` below it are Week 1 code, used only by `src/qwen_ocr.py`.

8 The Sarvam lane

Sarvam Vision is an asynchronous document service: submit a job, poll its status, download a ZIP. The Sarvam lane (`lanes.py SarvamLane L543`) is one thread that owns every call to it. Its design goal is that a crash at any instant can be recovered without losing or silently repeating a paid job.

8.1 The client

`src/sarvam_digitise.py` wraps the `sarvamai` SDK. It never passes a `model` (the server default for `digitise` is Sarvam Vision, and the API rejects "sarvam-vision" as a value), always asks for Markdown output (`sarvam_digitise.py OUTPUT_FORMAT L49`), and checks the language code against `langid.py SARVAM_LANGUAGES L28` before submitting, because the API accepts and bills any string.

- `sarvam_digitise.py submit L117` sends the chunk PDFs (`(name, bytes, "application/pdf")` triples) and returns the job id. It uses `sarvam_digitise.py classify_submit L106`: **only a 429 is retried**, because a 5xx or a timeout may already have created the job. Those become a `submit_uncertain:<reason>` fatal outcome that the lane handles itself (section 8.3).
- `sarvam_digitise.py poll L150` is one status call; `sarvam_digitise.py fetch L187` gets a download URL (a limited call) and then downloads the ZIP from the pre-signed storage URL (not a Sarvam API call, so no limiter), writing atomically.
- Terminal statuses are `sarvam_digitise.py TERMINAL L47` (completed, partially_completed, failed, rejected); only `sarvam_digitise.py OK_STATUS L48` (completed, partially_completed) has results.

8.2 The loop

The whole exchange is drawn in Figure 3 (after this chapter).

`lanes.py SarvamLane.run L569` first calls `lanes.py SarvamLane._restore L596` (section 8.4), then repeats every 0.25 s: poll every open job whose next-poll time has come; and, while the run is accepting work, Sarvam is alive and fewer than `max_open_jobs` (default 3) jobs are open, try to submit another. It returns when no job is open and the run is no longer accepting. An exception in the lane is logged as `lane_crash` and the lane is declared dead, so its pages move to Bodhan instead of the run hanging.

8.3 Submitting a job

`lanes.py SarvamLane._submit_next L652`, step by step:

1. **Claim.** If nothing is pending, return. If triage is still running and fewer than `per_job` (50) pages are pending, wait for the job to fill. Otherwise `manifest.py Manifest.claim_sarvam L209` takes up to 50 pending pages of one language and marks them queued.
2. **Serve the cache.** For each page, look up `lanes.py SarvamLane._page_key L617`. A hit is ingested at once for ₹0 (`cache_hit=1`) and never reaches the job.
3. **Reserve money.** ₹0.50 per remaining page is reserved in the `lanes.py Budget L85`. If it does not fit, the pages are released and the run is marked budget-exhausted.
4. **Chunk and record.** The pages are split into files of at most 10 pages (`chunk_pages`). `manifest.py Manifest.new_job L394` writes the job (state *submitting*, the page numbers of each file, the language) and points the pages' tasks at it, *before* anything is sent. Each chunk PDF is built by `pdf_pages.py build_chunk_pdf L131` and named `lanes.py job_file_name L509` (`j00007f02.pdf`: job 7, file 2).
5. **Submit.** `sarvam_digitise.py submit L117` returns a job id; `manifest.py Manifest.job_submitted L409` stores it and moves the job to *submitted*. The lane keeps an `lanes.py OpenJob L532` with the next poll time: `max(5, pages)` seconds from now (`sarvam_digitise.py first_poll_delay L162`: an 8-page job took about 4 s on the server, so polling earlier wastes the 10-a-minute budget).

If the submit fails: a 429 that survived all retries abandons the job and releases the pages without counting an attempt; `submit_uncertain` abandons the job, logs `possible_duplicate_job` and counts an attempt for each page; a fatal key or credit error releases the pages and kills the lane; any other failure marks the job failed and counts attempts. A page that fails twice becomes *failed* and the Router gives it to Bodhan.

8.4 Polling, downloading, ingesting, resuming

- **Poll** (`lanes.py SarvamLane._poll L712`): the delay between polls starts at 5 s and grows by 1.5× up to 30 s. A call failure retries in 60 s; a fatal one completes the job as failed. After 3,600 s (`lanes.py JOB_TIMEOUT_S L51`) the job is given up as `job_timeout`.
- **Complete** (`lanes.py SarvamLane._complete L731`): on an OK status, download the ZIP to `sarvam/jobs/<job_id>.zip` and read it with `lanes.py map_job_pages L513`, which opens `<file>/metadata/page_NNN.json` for each position in each chunk and maps it back to the original page number. A failed download or a bad ZIP retries in 30 s, up to 5 times. Pages present in the ZIP are ingested; missing ones are counted as a failed attempt (`missing:<status>`). The job is marked done (or failed if it returned nothing) and the reserved money is settled against what was actually returned.
- **Ingest** (`lanes.py SarvamLane._ingest L622`): store the page under its cache key, write `sarvam/pages/page_NNNNN.json`, convert to blocks (`blocks.py from_sarvam L140`), tag languages, apply the English-mode retry rule (section 5.2), let the Router add a Bodhan task if the page holds a table (`lanes.py Router.after_sarvam_page L213`, *before* the task is marked done, so the run cannot end in between), and finish the task with its cost.
- **Resume** (`lanes.py SarvamLane._restore L596`): every job still *submitting* or *submitted* in the manifest is looked at. A job without a job id is *abandoned*: the call may or may not have reached Sarvam, the pages are released and a `job_abandoned … possible_duplicate` event is logged. This is the one place a job can be paid for twice, at most the price of one job. A job with a job id is re-attached: its money is reserved again (forced), and it is polled at once. Queued tasks that belong to no open job are released.

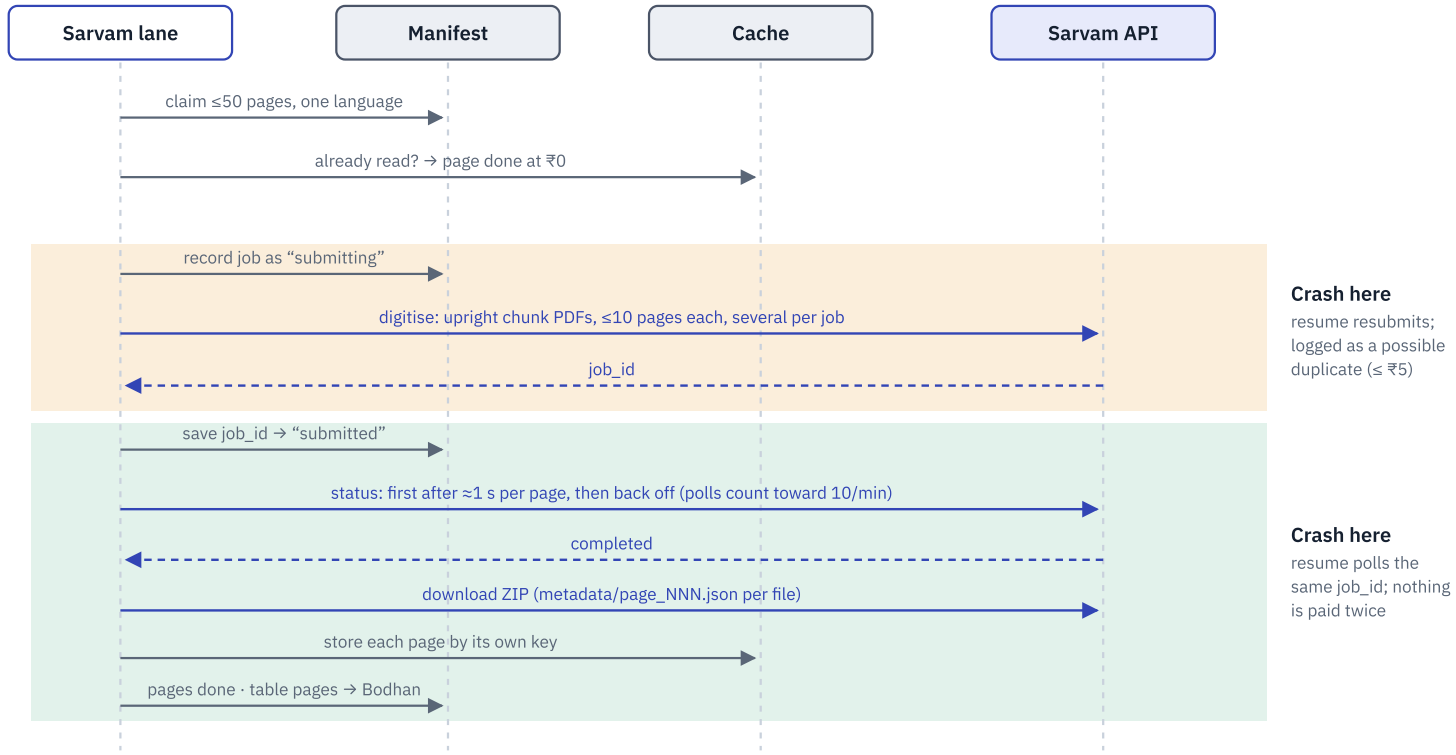
DECISION: WRITE THE JOB DOWN BEFORE SUBMITTING IT

The job row exists with state *submitting* before the API is called, and gets its id only after. A crash *after* the id is saved costs nothing: resume polls the same job. This was tested live: the Tamil sample was killed with `kill -9` right after submission, and `--resume` polled the same job and finished 8 of 8 pages with one submission in total.

READ THE CODE

- All of `lanes.py SarvamLane L543` (lines 540 to 753, about 210 lines) and `lanes.py map_job_pages L513`.
- Tests in `tests/test_engine.py` use a fake Sarvam module that returns ZIPs: submit, resume after a crash between recording and submitting, a missing page, a failed download, the 429 path.

FIGURE 3 **Sarvam lane: one job, crash-safe**



Tested live: the Tamil sample was killed with `kill -9` right after submission; `--resume` polled the same job and finished 8/8 pages with one submission in total. The SDK's own hidden retries are switched off, because a retried `digitise` can start a second paid job.

HOW TO READ IT

1. Each horizontal arrow is one message between the four columns (Sarvam lane, manifest, cache, Sarvam API). Read top to bottom: claim pages, serve what the cache already has, then record the job, submit, poll, download, store, finish.
2. The **amber band** is the dangerous window: the job is recorded as *submitting* but its id is not yet saved. A crash here makes the resume abandon and resubmit the job, and log a possible duplicate (at most one job is paid twice).
3. The **green band** is safe: the id is saved, so a crash only means the resume polls the same job. Pages are stored by their own cache key as soon as they are downloaded.

9 The page lane: Bodhan and Qwen

Bodhan reads one page image per HTTP request. `lanes.py PageLane L365` runs several worker threads for any such engine; what differs between engines is a small backend object.

9.1 Backends

A backend (the `lanes.py PageBackend L243` protocol) says how to call one engine and how to read its answer: `engine`, `cache_ns`, `model`, `price_inr`, `fatal_reasons`, and the methods `cache_extra()`, `ocr(image, mime, on_outcome)`, `to_blocks(...)` and `cost_inr(payload)`.

BACKEND	DOES	COST
`lanes.py BodhanAPIBackend L256`	Calls `bodhan_ocr.py bodhan_request L136` at `https://api.bodhan.ai/v1/chat/completions` with the image as a base64 data URL, `table_format` html (keeps merged cells) and `max_tokens` 4,096 per block. Returns layout blocks with a label, a box in pixels, text (tables as HTML) and a confidence. Converted by `blocks.py from_bodhan L118`	₹0.20 fixed
`lanes.py QwenBackend L298`	Optional. Sends the page to a Qwen model on OpenRouter with a fixed OCR prompt (HTML tables, original script, no commentary), reasoning off, 4,000 tokens. Returns Markdown only, so the page becomes one text block (`blocks.py from_text_page L169`)	from tokens: US$0.15 / 0.47 per million in / out
`lanes.py BodhanLocalBackend L279`	A placeholder for the RTX 5070: raises NotImplementedError	none

9.2 One page, start to finish

Figure 4 (after this chapter) shows the branches.

`lanes.py PageLane._worker L409` loops: if the run is not accepting or its engine is dead, leave; claim a task (own pending first, then a steal; `manifest.py Manifest.claim_page_task L234`); if none, sleep 0.4 s. Any exception is caught per page (the page is retried or failed; one page never kills a lane). For each page `lanes.py PageLane._process L437`:

1. renders the page upright at the run's dpi (`pdf_pages.py render_page L71`) from the thread's own PDF handle;
2. builds the cache key from the image bytes and looks it up. **A hit costs ₹0 and makes no call.** A miss reserves the price in the budget (if it does not fit, the page is released and the run stops accepting work);
3. calls the backend. A `rate_limit.py CallFailed L123` settles the reservation to zero; a reason in `fatal_reasons` (http_401, 402, 403, credit_exhausted) releases the page and kills the lane through `lanes.py Router.on_lane_dead L226`; any other reason counts an attempt (3 allowed) and, when used up, calls `lanes.py Router.on_page_failed L219`;
4. on success stores the payload in the cache, writes `<engine>/pages/page_NNNNN.json` with the render's width, height, dpi, rotation and transform, converts to blocks, tags languages and finishes the task with latency, tokens, cost and `degraded`.

9.3 What the Bodhan client handles

These are quirks found by testing, not documented by the vendor:

RESPONSE	MEANING	HANDLING
HTTP 200 with body `null` or no `choices`	An image the server could not decode (the docs say 502)	`bodhan_ocr.py classify_bodhan L100` returns DEGRADE *undecodable*; the request is sent again as JPEG (`bodhan_ocr.py to_jpeg L126`, quality 90), noted `reencoded_jpeg`
HTTP 502 whose detail mentions token, length or truncation	A block overflowed `max_tokens` (a huge HTML table)	sent again with markdown tables, page marked `markdown_tables` (merged cells are lost on that page)
HTTP 502 with no detail, or `bad_json`	Transient	normal retry with back-off
HTTP 429	The 8-per-minute limit (the response headers wrongly advertise 120)	the limiter pauses for `Retry-After`

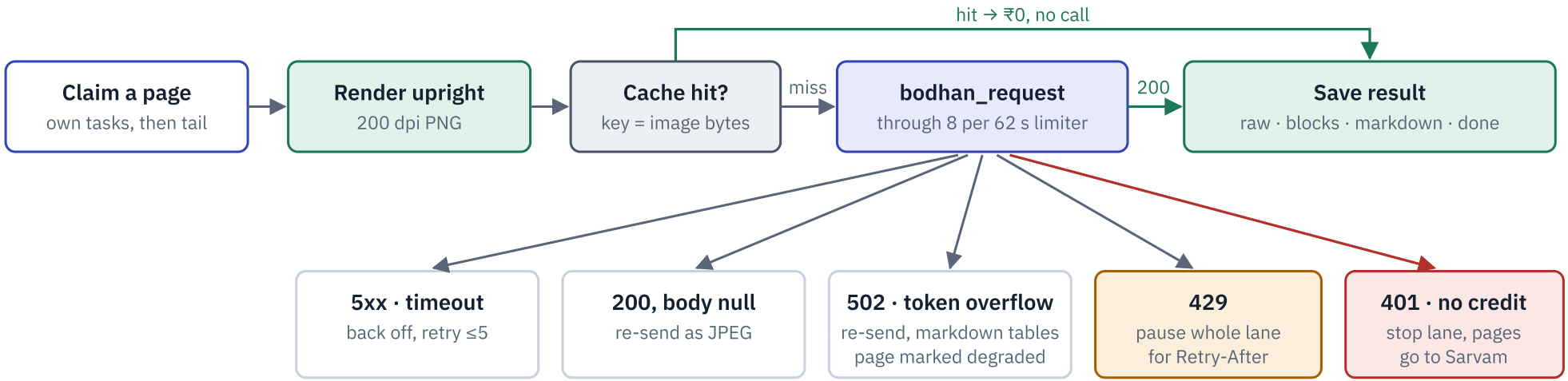
RESPONSE	MEANING	HANDLING
401 or 402	Bad key or no credit	fatal: the lane stops, its pages go to Sarvam

At most three attempts to change the request are made (`for _ in range(3)`), then a fatal `degrade_loop`. The payload gets an `_infinite` section recording the table format used, the MIME type, what was degraded, the latency and the rate-limit headers. Four workers run at once because a dense table page can take up to 44 s.

READ THE CODE

- `lanes.py` **`PageLane._process`** `L437` (about 70 lines): the whole page path including every failure branch.
- `bodhan_ocr.py` **`bodhan_request`** `L136` and `bodhan_ocr.py` **`classify_bodhan`** `L100`; `tests/test_clients.py` checks each response case with fake HTTP.

FIGURE 4 **Bodhan worker: one page, every outcome handled**



Both quirks were found by testing, not from the docs. An undecodable image returns HTTP 200 with a `null` body (the docs say 502), and the 8-per-minute limit is per model even though the headers advertise 120. Four workers run at once because dense table pages take up to 44 s each.

HOW TO READ IT

1. The top row is the normal path of one page: claim, render upright, cache lookup, request through the shared limiter, save. A cache hit (the arrow over the top) skips the request and costs ₹0.
2. The bottom row is what each response can mean and what happens next: a server error or timeout is retried with back-off (up to 5 times); a null body is resent as JPEG; a token overflow is resent with markdown tables; a 429 pauses every worker of the lane for Retry-After; a bad key or no credit stops the lane and hands its pages to Sarvam.

10 Budget, cache, events and resuming

Four small mechanisms together make a run safe to interrupt and cheap to repeat.

10.1 The budget

`lanes.py Budget L85` holds `cap` (None means no cap), `spent` and `reserved`. `lanes.py Budget.reserve L92` refuses if `spent + reserved + amount` would pass the cap; `lanes.py Budget.settle L99` replaces a reservation with the actual cost. A reservation is made before every paid call, so parallel workers cannot together overshoot the cap. It starts from the money the manifest already shows as spent, so a resumed run counts earlier spending. The cap exists only when the user gives `--max-cost-inr`. Stage 2 applies the same cap to the whole document through `assemble.py Assembler._can_spend L230`. OpenRouter spending has its own cap, the US$10 ledger (section 18.5).

10.2 The cache

Described in section 3.4. Three properties matter here: a key is made of the *content* of the request, so the same page asked again is free whatever run asks; a write is atomic; and the result is stored *before* the manifest row says done. A re-run therefore finds and reuses everything that was paid for, including model answers.

10.3 The events log

`manifest.py EventLog L67` appends one JSON line per event, with a timestamp, behind a lock. Kinds that exist: `run_start, run_end, outcome, page_done, page_error, steal, fallback, lane_dead, lane_crash, budget_stop, job_submitted, job_status, job_done, job_resumed, job_abandoned, job_submit_failed, possible_duplicate_job, job_fetch_failed, mode_retry, verify_table, triage_crash, assembled`. `run.py Run._summary L276` counts them and the report prints them; they are also the best place to look when a run behaved oddly.

10.4 What can be paid twice

WHERE THE PROCESS STOPS	WHAT RESUME DOES	PAID AGAIN?
Before a job row is written	Pages are queued with no job; they are released as orphans	No
Row written (*submitting*), id not yet saved	The job is abandoned, its pages are released and read again. Logged as a possible duplicate	**Possibly: at most one job** (₹0.50 × its pages, up to ₹25)
Job id saved, job running or finished	The same job is polled and downloaded	No
Bodhan reply received, cache not yet written	The page is read again	Yes, ₹0.20 (a window of microseconds)
Bodhan reply cached, manifest row not yet finished	The page is found in the cache	No
Run repeated with another `-l` code	Sarvam's cache key differs	**Yes**, the whole Sarvam cost (section 3.4)

10.5 Resuming and starting over

- `--resume`: same folder, same fingerprint (section 4.2). In-flight tasks go back to pending; open Sarvam jobs are re-attached. Adding pages to an earlier selection works the same way.
- `--fresh`: deletes the manifest, events and result folders, keeps the cache. The pages read before cost nothing the second time, so this is the way to change settings without paying again (with the same language code).
- **Assembling again** (`python -m src.assemble`) needs neither: it reads the finished manifest and blocks, and its model answers come from the cache.

11 Blocks and languages: what Stage 2 starts from

Stage 1 ends with, for every page and every engine that read it, a list of blocks in one format and a language tag on each. Everything in Stage 2 is written against this format, never against Sarvam's or Bodhan's raw JSON.

11.1 The block format

`blocks.py Block L85` has eleven fields: `page, engine, order, kind, level, raw_label, bbox, text, conf, lang`. The `kind` is one of nine (`blocks.py KINDS L14`):

KIND	EXAMPLES OF THE ENGINES' LABELS	USED FOR
`heading`	Title, Chapter-title, Section-title … (Bodhan, 5 levels); headline, title, section-title, sub-headline (Sarvam)	# levels in doc.md
`text`	Paragraph, List, Infobox, Question, Answer, Solved-example, Code, Reference … / paragraph, list	Prose
`contents`	Table-of-contents, Index	Prose
`table`	Table / table	Stage 2's table path
`caption`	Table-caption, Image-caption	Context for joins; italic line in doc.md
`figure`	Image, Chart, Diagram, Flag / image, photograph, chart, diagram, figure	Cropped to a PNG
`math`	Equation, Expression / equation, formula	Kept as LaTeX
`footnote`	Footnote	Blockquote at the end of the page
`furniture`	Header, Footer, Page-number, Folio, Advertisement	Dropped; page numbers are read for a consistency check

The two label tables are `blocks.py BODHAN_KIND L17` (37 labels) and `blocks.py SARVAM_KIND L57`; an unknown label becomes `text` (`blocks.py kind_of L112`). The conversions are `blocks.py from_bodhan L118` (sorted by the engine's `order`; the box in pixels is divided by the uploaded image's size) and `blocks.py from_sarvam L140` (sorted by `reading_order`; uses `bbox_norm` when present, else `coordinates` divided by the page's image size). Both call `blocks.py merge_furniture L182`, which drops an empty furniture block that overlaps (IoU above 0.5) one with text, a Header and a Folio on top of each other.

The box frame is the point of the whole of chapter 5.4: Bodhan sees our upright render and Sarvam sees our upright chunk PDF with MediaBox equal to CropBox, so both boxes are in the same 0 to 1 frame. `blocks.py iou L174` can therefore match "the table" of one engine to "the table" of the other, which is how Stage 2 finds the second reading of every table.

11.2 Language identification

The language is not known before OCR, and the engines do not report it. `src/langid.py` (free, offline) finds it from the text, per block, in these steps:

1. **Script** from Unicode ranges (`langid.py SCRIPT_RANGES L34`). Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada, Malayalam, Ol Chiki and Meitei Mayek each belong to one language (`langid.py SINGLE L49`), so script decides.
2. **Within a shared script**:
   - *Devanagari* (Hindi, Marathi, Nepali, Sanskrit, Konkani, Maithili, Dogri, Bodo): whole-word function words (Hindi है, और, में; Marathi आहे, आणि, च्या; Nepali छ, पनि, हरू …) and suffixes, plus the letter ळ as a Marathi sign (`langid.py devanagari_scores L107`, `langid.py DEVANAGARI_MARKERS L63`). Words common to two languages were left out on purpose.
   - *Bengali script*: Assamese if it has ৰ or ৱ.
   - *Arabic script*: Sindhi or Kashmiri if it has their extra letters, else Urdu.
3. **Short blocks** (a table cell, a heading) carry too few words. `langid.py page_languages L197` lets a block with at least 2 marker hits and a clear winner (confidence ≥ 0.6) decide for itself; every other block takes the language of the nearest decided block in reading order (preferring the one before). A Marathi table under a Marathi caption stays Marathi even when a Hindi paragraph sits higher on the page. With no decided block, the page's pooled markers decide, then the document prior.

4. **The document prior.** `assemble.py Assembler._tag_languages L264` runs the pass twice: first with the user's language as a prior; if there was none, it counts the pages whose Devanagari confidence is at least 0.5, takes the most common language and runs again with that as the prior, so pages with too few words fall back to the document's language.

Markup is removed before counting (`langid.py strip_markup L93`: HTML tags, LaTeX commands and `$math$` are not language). The result is a share per language for each block and page, e.g. `{hi-IN: 0.78, en-IN: 0.22}` (`langid.py TextLanguages L154`), saved in the block files and printed in doc.md as `<!-- lang: … -->` comments where a block differs from its page.

> **READ THE CODE**
>
> - `src/blocks.py` (210 lines) and `src/langid.py` (240 lines) are self-contained and short.
> - Tests: `tests/test_languages.py` (Marathi against Hindi, mixed pages, the mode retry) and `tests/test_blocks_manifest.py`.

12 Stage 2 in order

`src/assemble.py` is 1,200 lines and does everything after the engines: it is the second biggest file in the project. This chapter is the table of contents of that file. The following chapters take its parts one by one.

12.1 What assembly loads

`assemble.py Assembler._load L237` opens the manifest and, for every triaged page, looks at which engines have a *done* task and a blocks file. It keeps:

- `self.pages`: pages with at least one reading;
- `self.blocks[page][engine]`: the blocks of each engine;
- `self.primary[page]`: the first of Sarvam, Bodhan, Qwen (`assemble.py ENGINE_ORDER L66`) that read the page. The *primary* engine's text and layout are what doc.md shows;
- `self.info[page]`: the page's manifest row, including the page classes (computed on the spot for runs triaged before they existed);
- language tags for every block (section 11.2).

12.2 The order of the work

`assemble.py Assembler.run L980` is the sequence below (the standard path; the legacy path in chapter 23 replaces steps 4 to 7):

#	STEP	FUNCTION	COST	CHAPTER
1	Load pages, blocks, page classes, languages	`assemble.py Assembler._load L237`	free	12
2	For every table block of the primary engine, build a checked table: find the other engine's table, align, decide cells by free evidence, read the page image's structure, list what is left	`assemble.py Assembler._build_table_qa L430`	free	13 to 15
3	Think only: compare the two engines' prose	`assemble.py Assembler._check_text L476`	free	20
4	Learn each scrambled font's glyph mapping from agreed cells and re-check the tables whose layer changed	`assemble.py Assembler._repair_layers L494`	free	16
5	Image check of the open cells, then blind audits of the riskiest agreed cells	`assemble.py Assembler._image_check L516`	OpenRouter (Normal, Think)	17, 18
6	Join tables that continue on a later page	`assemble.py Assembler._join_tables_standard L750`	free; a rare seam picture	19
7	Compose doc.md (headings, paragraphs joined across pages, tables, figures, footnotes)	`assemble.py Assembler._compose L863`	free	21
8	Write one HTML, CSV and JSON file per table	`assemble.py Assembler._write_tables L952`	free	21
9	Write assembly.json and report.md	`assemble.py Assembler._result L1047`, `assemble.py render_report L1113`	free	21

Everything that can be decided without the model is decided *before* step 5, so the image check sees only what is left, and the free part can be re-run and tested without any API.

12.3 The table record

`assemble.py TableRec L152` is what the assembler keeps for every table segment (one per table block per page): `id` (`t<page:05d>_<k>`), page, order and box; the checked `table`; the engine used as the reference; the `qt` (the `table_qa.py QTable L99` behind it); the `geometry` from the page image; the `layer_raw` (the PDF's own text inside the box); the `readings` (each engine's table before the check, so a re-check can start again from them); the `caption` above the table; `scanned`; and the results: `qa`, `qa_cells`, `questions`, `image_check`, `flags`, `pages` and `joined_into` once a later segment has been absorbed.

`assemble.py Assembler._build_table_qa L430` does, for one table block:

1. `assemble.py Assembler._versions L295` finds the same table in every engine that read the page: the engine's table block with the highest box overlap (IoU at least 0.3) with this block, parsed by `tables.py parse_tables L282` (the largest table in the block if there are several). If an engine has no table block there, `assemble.py table_in_text L126` looks for a table it wrote *inside a text block*: Bodhan sometimes files a whole page body as one `List` block with the tables as HTML inside (measured on page 317 of the Hindi annual report: three tables in one block). The block must cover 80% of the table, the parsed table holding most of the reference's cells is taken, and the table keeps the primary's box so the crop does not stretch to the page;
2. the box is the union of the engines' boxes; the page image of that box is analysed once at 200 dpi (`table_geometry.py table_geometry L245`); a failure here is logged and the table goes on without geometry;
3. `assemble.py Assembler._layer_text L420` reads the PDF's own text in the box, only if the page has at least 50 letters and its text-layer class is *unicode*;
4. `table_qa.py check L276` does the alignment and decides cells (chapter 14);
5. `assemble.py Assembler._sync L461` copies the result onto the record: the table, the summary, the cells that were not plain agreement or carry a risk, the question queue, the count of printed-formula checks and the status as a single flag.

READ THE CODE

- Start at `assemble.py Assembler.run L980` (about 35 lines), then `assemble.py Assembler._build_table_qa L430`, then `assemble.py Assembler._image_check L516`.
- `assemble.py Assembler._result L1047` lists everything that ends up in assembly.json (chapter 21).
- The first 120 lines of `src/assemble.py` are small helpers: `assemble.py crop_png L80`, `assemble.py layer_numbers L95`, `assemble.py cleanup_text L117`.

13 Reading tables: the grid and the number checks

`src/tables.py` (719 lines) turns an engine's table (HTML or Markdown) into a grid of cells, understands numbers written in any Indian script, and checks the arithmetic printed in the table itself. It is the oldest part of Stage 2 and still supplies the table model that everything else uses. Its role changed in October: arithmetic used to *decide* a table's quality; now only a *printed* formula may decide a cell, and everything else it finds is a hint.

13.1 The table model

A `tables.py Table L106` is a list of `tables.py Cell L96` objects (`r, c, rowspan, colspan, text, header`), the table's size and the number of header rows. A merged cell is one Cell that covers several grid positions. Three views are used:

- `tables.py Table.grid L113`: a grid of Cell references (a merged cell appears in every position it covers);
- `tables.py Table.rows_text L121`: a grid of strings. With `expand=True` a merged cell's text repeats in every position (for labels and CSV); with `expand=False` it appears only at its top-left position (for sums, so a merged value is counted once);
- `tables.py Table.to_html L134`: the HTML written to doc.md, with `rowspan` and `colspan`, header rows in `<thead>` and line breaks as `<br/>`.

13.2 Parsing

`tables.py parse_tables L282` takes a block's text. If it contains `<table` it is parsed as HTML by `tables.py _HTMLTables L159` (a small `html.parser` subclass: only the outermost table counts, `<br>` becomes a newline, `th` or `thead` marks a header cell, bad span values count as 1); otherwise as a Markdown pipe table (`tables.py _parse_markdown L259`). `tables.py _layout L219` then places the cells: it walks the rows, skips positions already covered by an earlier rowspan or colspan, and clips spans that run past the table. `tables.py _header_rows L244` decides how many leading rows are header: rows made only of header cells, the printed column-numbering row, and rows covered by a multi-row header cell.

The two engines write the same table differently, which is why alignment (chapter 14) exists. Sarvam tends to write a merged header as empty `<th>` cells; Bodhan uses `rowspan`. The benchmark reproduces both dialects (`engines.py to_dialect L246`).

13.3 Numbers in thirteen digit systems

- `tables.py DIGIT_MAP L32` maps the digits of 13 scripts to 0–9: Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, Malayalam, Arabic-Indic, Extended Arabic-Indic, Ol Chiki and Meetei Mayek, plus the Unicode minus and en dash. `tables.py normalize_digits L57` applies it.
- `tables.py parse_number L61` accepts an optional sign, thousands commas, a decimal part, a percent sign, and brackets for a negative number: `(1,296.57)` is −1296.57. Anything else is not a number (None).
- `tables.py number_key L74` gives a canonical string (`'१,२९६.५७'` and `'1296.57'` both give `1296.57`), used to find a number in the PDF text layer. `tables.py decimals L81` counts decimal places (the tolerance of a sum depends on them).

13.4 The checks

`tables.py check_table L378` returns a list of `tables.py Check L303` objects (`kind, row, col, expected, found, ok, detail`). It first finds the printed column-numbering row (`tables.py _is_numbering_row L316`: at least three cells, each starting with a number, increasing, starting at 0, 1 or 2) and the numeric columns (at least a third of body cells are numbers, not mostly budget codes such as ००४९, not a 1, 2, 3 … serial column). Then:

KIND	WHAT IT CHECKS	EXAMPLE	ROLE NOW
`row_formula`	A header says how its column is made: एकूण (४ ते ७) or `(4+5+6)` means this column equals the sum of the columns with those printed numbers (found through the numbering row)	Marathi report: column 8 = columns 4 to 7	**Decider** (a printed formula)
`col_formula`	A total row says which rows it adds: एकूण (क+ख+ग+घ) equals the rows whose labels start with (क), (ख) … (a full bracket is preferred over a loose match)	Marathi report: grand total row	**Decider**
`row_consensus`	A column headed "total" in any of ten languages equals the sum of the run of numeric columns on its left, *only if* at least 60% of rows obey	an English "Total" column	Hint (risk flag `total_hint_fails`)

KIND	WHAT IT CHECKS	EXAMPLE	ROLE NOW
`col_consensus`	A total row equals the sum of the body rows since the previous total row, at least 60% of columns obeying	"Total" at the bottom	Hint
`row_product`	A column is the product of two others (annual amount × years = total): at least 5 rows and 80% agreeing; a column that mostly copies another is skipped	Union budget: cost × years	Hint

The tolerance of a sum is half a unit of the last printed decimal place, per term (`tables.py _tolerance L355`), so `12.50 + 7.25 = 19.75` is exact but a lost decimal point is not. Total words are matched as *whole words* (`tables.py TOTAL_TOKENS L38`: एकूण, कुल, योग, ಮಾಡಿ, மொத்தம், మొత్తం, ಒಟ್ಟು, ആകെ ...), because substring matching finds योग inside उपयोग and मोट inside मोटर.

DECISION: ARITHMETIC IS A HINT, ONLY A PRINTED FORMULA DECIDES

The first Stage 2 chose between two engines' tables by how many sums added up, and swapped numbers until they did. Two things were wrong with that. Most real tables have no totals, so most tables got no check at all; and two engines that agree can both be wrong, which a sum cannot reveal. On the real engine readings the old Stage 2 accepted 10 wrong tables without a flag. A sum that is *not printed in the table* is a coincidence-prone guess (a column can happen to be the sum of its left neighbours), so it now only raises a risk flag. A formula the table itself prints is different evidence, and it may choose between two readings of a cell, but only when the winner makes *every* formula in its row and column hold (chapter 14).

13.5 What else is in tables.py

The *legacy* machinery is still in the file for `--table-qa legacy` (chapter 23): `tables.py fuse L576` (repair one engine's table cell by cell from the other's where that makes more checks pass), `tables.py continuation L639` and `tables.py join L685` (the old page-pair join score), and the text-layer number match `tables.py text_layer_match L560`.

READ THE CODE

- Model and parsing: lines 96 to 300 of `src/tables.py`. Checks: `tables.py check_table L378` (about 160 lines, five numbered blocks).
- Tests: `tests/test_tables.py` (parsing merged cells, digits, every kind of check, the Marathi header example, fusion and joins).

14 The free table check

`src/table_qa.py` (504 lines) is the centre of Stage 2. It takes every engine's reading of one table and produces one grid in which every cell keeps every reading, decides what free evidence can decide, flags what looks risky even where the engines agree, and says what is left. It never calls an API and never writes a value that no engine produced.

14.1 The idea in four lines

1. **Line the readings up** into one grid (rows by similarity, columns by content).
2. **Check the shape** against the page image: the ruling lines say how many rows and columns the table has.
3. **For every cell, cheapest evidence first:** agree, cosmetic, formula, text layer, minor. What is left is *open*.
4. **Do not trust agreement.** A cell both engines read the same can still be wrong (both engines read Tamil numerals as letters, identically); risky ones become audit questions.

Figure 5 (after this chapter) is the flow as a picture.

14.2 The data structures

CLASS	HOLDS
`table_qa.py` **`Evidence`** `L174`	Free evidence about one table, all optional: the `geometry` from the page image, the `layer_text` (the PDF's text inside the box) and `scanned`.
`table_qa.py` **`QCell`** `L79`	One cell of the aligned grid: `r, c, rowspan, colspan, header`, `reads` (engine → its raw text, or None if that engine has no such cell), `value` (the chosen text), `decided_by` and a list of `risk` flags. Its property `open` is true when nothing decided it.
`table_qa.py` **`QTable`** `L99`	The result: the `reference` engine, the final `table`, the `cells`, the `structure` decision, the `layer` proof, a `status` and the `questions`.
`table_qa.py` **`Layer`** `L191`	The PDF's own text in the table box, as one string with no spaces (`blob`), its set of words (`tokens`) and a `clean` flag (no unexplained glyphs).

14.3 Step 1: comparison form and phantom columns

`table_qa.py` **`canon`** `L52` is the comparison form of a string: Unicode NFC, zero-width marks removed, one kind of dash and quote, a $…$ LaTeX wrapper removed, a doubled combining mark undoubled (Chrome's text layer repeats matras), whitespace collapsed. Two readings that are equal after `canon` are the same reading apart from spacing and style.

`table_qa.py` **`squeeze`** `L216` drops *phantom columns* from each reading: a column that is empty in every body row and has no header cell of its own (a reader that misplaces a colspan invents them; measured once on a Bengali table where Bodhan invented two). Only empty columns go, so no text can be lost.

14.4 Step 2: the shape

`table_qa.py` **`choose_structure`** `L247` picks the *reference* reading (the one whose rows and columns the grid follows) and records how it was decided:

DECISION	WHEN	EFFECT
`confirmed`	The ruling lines give a full grid and *every* engine's shape equals it	Reference: the first engine (Sarvam, then Bodhan)
`decided_by_image`	A full grid, and only some engines have its shape	Reference: the first engine with that shape. Status: *repaired*
`conflict`	A full grid, and no engine has its shape	Reference: the first engine; the table **needs review** and a structure question is queued
`agreed`	No usable grid, all engines have one shape	First engine

DECISION	WHEN	EFFECT
decided_by_numbering	Shapes differ; the table prints a column-numbering row 1…n and exactly one shape (among engines it confirms) has n columns	That engine. *repaired*
decided_by_gaps	Shapes differ; the white gaps between columns in the image give a count and exactly one engine has it	That engine. *repaired*
readers_differ	None of the above	First engine; needs review

The image **chooses between the engines' shapes; it never overrides both**. Where it does not match either, the conflict is shown to a person. How the lines are read is chapter 15.

14.5 Step 3: aligning the readings

With the reference chosen, each other reading is lined up to it:

- `table_qa.py align_rows L124` turns every row into one string (cells joined by `|`) and finds, by dynamic programming, the in-order matching of reference rows to the other reading's rows that maximises the total similarity (`table_qa.py similarity L67`, difflib's ratio), allowing a dropped or extra row as a gap and refusing a pair below 0.5.
- `table_qa.py align_cols L145`: if both have the same number of columns it is the identity; otherwise each reference column is compared with each column of the other over up to 40 matched rows, and columns are paired greedily by average similarity (at least 0.5), so a column one engine split in two does not shift everything after it.
- For every reference cell, `reads` collects each engine's raw text through the two maps; where an engine has no matching row or column, the entry is None (the engine "has no such cell").

14.6 Step 4: deciding each cell

For every cell, in this order, the first rule that fires wins (`table_qa.py check L276`, lines 321 to 368):

#	DECIDED_BY	RULE	MEASURED
1	single	Only one engine has this cell (the other missed a row or column). Accepted; flagged `single_reader` for audit	-
2	agree	All readings are identical	-
2	cosmetic	Readings are equal after `canon` (spacing, dashes, quotes, `$..$`); the plainest form (no LaTeX) is kept	27 of 27 right
3	formula	The cell is numeric and the table's own printed formulas hold for exactly one candidate. The winner must make *every* formula in its row and column hold, not merely fewer fail	6 of 7, then tightened to this rule
4	text_layer	A *proven* PDF text layer contains exactly one candidate (all candidates testable)	4 of 4
5	minor	The candidates differ only in punctuation or spacing and none is a number (`table_qa.py minor_difference L380`). The one the proven layer contains, else the reference	14% of Union disputes; the comma-against-full-stop weak spot of the models
6	open	Nothing free decided it: a question for the image check	-
(7)	image: <models>	Set later by the image check (chapter 18)	-

Two guards live in this loop. A number is never a "minor" difference: `12.50` against `1250` is a lost decimal point, a real error. And a proven layer that contains *none* of the candidates raises the risk flag `layer_has_neither` (the printed value is probably a third one), but only if the layer is *clean*: on a partly repaired layer the word may simply still be scrambled (measured: 56 Union cells were wrongly held back before this condition).

14.7 The text layer must prove itself

A born-digital PDF carries its own text, which is excellent evidence when it is accurate and useless when the font scrambles it. `table_qa.py _layer_proof L406` decides per table, using the cells both engines already agree on as an answer key: it counts how many agreed cells it can test and how many of them it finds. The layer is **proven** only if at least 5 cells were testable and at least 90% were found. `table_qa.py Layer.contains L204` is deliberately strict: a number must match a *whole word* (`1,234` is not inside

`11,234.50`), text must be at least 4 characters and match as a substring of the layer with spaces removed, and anything shorter returns None ("too short to say"), because short strings occur by chance everywhere. A layer in a legacy font fails the proof by itself.

14.8 Step 5: do not trust agreement

`table_qa.py _risks L415` adds flags to cells, including cells every engine agrees on. Each flag has a weight (`table_qa.py RISK_WEIGHT L465`) that ranks the audit questions; the first five are *high risk* (`table_qa.py HIGH_RISK L461`) and keep a table in review. The "precision" is the share of flagged cells that were really wrong, measured on real-engine runs and the simulation on 21 October:

FLAG	MEANING	WEIGHT	HIGH RISK	MEASURED PRECISION
`layer_disagrees`	A proven layer does not contain an agreed cell's value	5	yes	3 / 13
`layer_has_neither`	A clean proven layer contains none of the candidates	5	no	-
`ink_but_empty`	The cell is empty in both readings but the image shows ink in its box (ruled table, same shape)	4	yes	4 / 4
`lookalike_numeral`	The cell is made only of letters that Tamil numerals are misread as (கஉஙசருசுஎஅகூ)	4	yes	27 / 30
`formula_fails`	A printed formula fails at this cell	3	yes	6 / 19
`audit_disagrees`	A blind model read it differently (set by an audit)	-	yes	-
`rare_numerals`	Tamil or Telugu digits (rare in print, often misread)	2	no	34 / 237
`not_a_number_in_number_column`	A non-number in a column that is at least 70% numbers (5+ cells)	1	no	5 / 37
`total_hint_fails`	A consensus total or product rule fails	1	no	-
`single_reader`	Only one engine had the cell	1	no	-
`text_but_blank`	Text where the image is blank	0.5	no	0 / 15

14.9 Questions and status

`table_qa.py _questions L477` builds the list the image check works from, most valuable first: a **structure** question (priority 10) when the shape is in conflict or the readers differ, or (priority 9, reason `single_reading`) when only one engine read the table; a **cell** question (priority 6 + risk weight) for every open cell, with its candidate readings; an **audit** question (priority = risk weight) for every non-open cell that carries a risk and has not been confirmed. `table_qa.py _status L495` then gives the table's status:

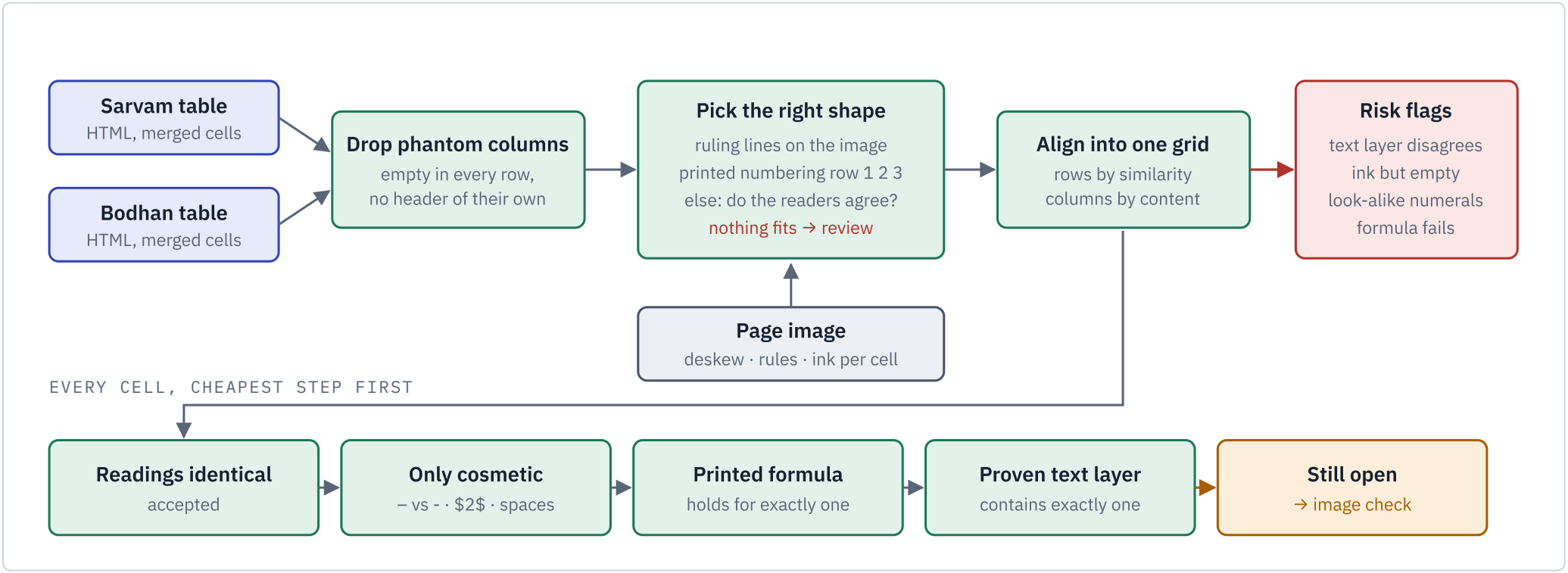
- **needs_review**: the structure is in conflict or the readers differ, or any cell is open, or any cell carries a high-risk flag, or at least half of the cells have *one reader only* (`table_qa.py single_reading L470`: the other engine never read this table, so nothing was compared; added 22 October after a table with a single reading came out "verified");
- **repaired**: nothing is open, but the shape was decided by the image, the gaps or the numbering row, or some cell was settled by formula, text layer, minor rule or the image check;
- **verified**: none of the above.

An audit can only *add* doubt (`audit_disagrees` is high risk); a blind "same" never cancels a high-risk flag. This is measured: GPT-5.6 Luna read the Telugu cell `E-33` as `E-33` and "confirmed" the engines' shared error. After the image check, `table_qa.py apply L388` writes a model's choice into a cell and `table_qa.py refresh L394` recomputes questions and status.

READ THE CODE

- `table_qa.py check L276` is 100 lines and reads top to bottom as the steps above. Everything else in the file is a helper it calls.
- Tests: `tests/test_table_qa.py` (26 tests: alignment, phantom columns, each decider, the proof, risks, status, the judge rule with a fake model).
- How the numbers above were produced: `bench/calibrate.py` and chapter 24.

FIGURE 5 **Free table check: one grid, cheapest evidence first**



Measured on real engine output: cosmetic decisions were right 27 of 27 times and text-layer decisions 4 of 4. The printed-formula rule was right 6 of 7 times, then tightened so the winner must make every formula in its row and column hold. A text layer counts only after it contains at least 90% of the cells both engines agree on, which legacy-font layers fail on their own. The image structure caught a real Bodhan error: two invented, empty columns in a Bengali table.

HOW TO READ IT

1. Top row: the two engines' tables lose phantom columns, the right shape is chosen (ruling lines on the page image, else the printed column-numbering row, else agreement of the readers; nothing fits means review), the readings are lined up into one grid, and risky cells are flagged even where the engines agree.
2. Bottom row: every cell goes through five steps from left to right and stops at the first that decides: identical readings, only cosmetic differences, a printed formula that holds for exactly one reading, a text layer that has proved itself, and (not drawn) differences in punctuation only. What is still open goes to the image check.
3. No step writes a value that no engine produced. Code: `table_qa.py check`, with the shape chosen by `table_qa.py choose_structure`.

15 Rows and columns from ruling lines

`src/table_geometry.py` (250 lines) reads a table's structure from the page *image*, independently of both OCR engines. That independence is the point: it can catch an error both engines share (a row both dropped, a cell both left empty). It uses NumPy and Pillow only, and costs a fraction of a second per table.

15.1 Why not PyMuPDF's table finder

PyMuPDF has `page.find_tables()`, and triage uses it as a hint. It is not used to read shape because, on the real documents, it is wrong too often (measured on the 28 real tables of the sample set): it gave exactly the right rows and columns for 8 of them.

- On the Union budget it reports 55 rows by 23 columns for a table of 17 rows and 9 columns. The page draws 423 invisible white rectangles and thin black rules; the finder counts the invisible cell boxes.
- On sideways pages (the Marathi report) it returns the two numbers swapped or unrelated, because it works on the page as stored, not as read.
- It cannot work at all on a scan, where there are no drawn lines.

The image-based reader sees the same visible lines a person sees. Where the PDF draws its lines as rules (the Marathi and Economic Survey tables) the image counts equal the drawn ones, checked against `page.get_drawings()`: Marathi page 1 has 21 row lines and 14 column lines in both. It looks only inside the table's box (taken from the engines' boxes), so it cannot be confused by lines elsewhere on the page.

15.2 How the lines are found

`table_geometry.py table_geometry L245` renders the table's box at 200 dpi with a margin of 1.2% of the page (`table_geometry.py crop_gray L66`), then `table_geometry.py analyse L213` does:

1. **Straighten** (`table_geometry.py deskew L93`). Scans are often a degree or two off. The dark mask is rotated to each angle from −2.5° to +2.5° in 0.1° steps and the angle that makes the row profile sharpest (largest variance) is kept. The mask is rotated with *nearest-neighbour* sampling: rotating grey pixels blurs them and biases the search toward 0°, which was a real bug (the first version was biased toward 0°). The crop is then rotated by that angle.
2. **Ink**: a pixel is dark if it is at most `min(Otsu threshold, 170)` (`table_geometry.py otsu L78`; dark is *less than or equal to* the threshold, another off-by-one that was found and fixed).
3. **Column lines** (`table_geometry.py vertical_rules L141`): a pixel column that has a dark vertical run of at least 30% of the table's height, after closing gaps of up to 2 pixels (noise and JPEG break thin lines); neighbouring columns are merged into one line.
4. **Row lines** (`table_geometry.py horizontal_rules L148`): a pixel row with a dark run of at least 5% of the width that *crosses at least two column lines*, if the table has at least 3 column lines; or a run of at least 60% of the width if it does not. The crossing rule matters for Devanagari: the headline stroke over every word looks like a line but never crosses a cell border.
5. **Decision**: with at least 3 column lines and 3 row lines it is a **ruled** table (source `rules`, rows = row lines − 1, columns = column lines − 1). With only 2 or more row lines (top, header, bottom rules: the Union budget) the columns are counted from the white gaps between text columns in the body (`table_geometry.py column_gaps L197`; gaps narrower than 6 px are spaces) and the source is `gaps`: a hint, never a decision on its own. Otherwise `none`.

The result is a `table_geometry.py Geometry L36`: `ruled, rows, cols, source, skew`, the pixel positions of the lines in the straightened crop, the ink mask and the straightened grey image (kept so cut-outs can reuse it). `confident` means source `rules`. `table_geometry.py Geometry.cell_ink L52` gives the share of dark pixels inside a cell, borders excluded: the basis of the `ink_but_empty` flag.

15.3 Three corrections that were measured

The first version read 309 of 322 ruled benchmark tables exactly. The 13 misses were all scans with one of three defects, and each was fixed and measured (`bench/geometry_eval.py`; the figure is after this chapter):

DEFECT	FIX	FUNCTION
A column line broken by merged cells: it is too short for the 30% rule	Count a thin dark line with clean paper either side that runs from one row line to the next in at least two of the bands (and at least a quarter of them)	`table_geometry.py partial_rules L164`
A note row at the bottom of the frame (टीप: ...) that no inner column line crosses was counted as a row	If the bottom band is crossed by none of the inner column lines (coverage under 90%), it is not a table row	`table_geometry.py note_band L186`
The box cut the table: a row line runs into the crop's edge beyond the outermost column line	Detect it and read the box again once, widened to 4.5% of the page on each side	`table_geometry.py clipped L232`

A fourth idea, relaxing the 30% rule globally, was not used: it admitted text columns as lines. The tuning of the second correction needed one more step (the first threshold, 0.5, missed the Marathi note row because text sits over the line positions; 0.9 works).

15.4 Measured

WHERE	RESULT
Benchmark, ruled tables (v1 and v2, born digital and scanned copies)	**322 of 322** read exactly (rows and columns). Before the corrections: 309
Benchmark, unruled tables	**0 false grids** in 172: the reader never claimed a grid that was not there
Benchmark, column gaps on unruled tables	Right only 40–70% of the time on synthetic pages, so they are a hint only
Real documents, 16 tables with full grids	14 confirmed by both engines; Marathi page 1 decided between the engines (Sarvam had dropped the एकूण total row, Bodhan had it: 19 rows); Marathi page 8 a conflict (lines 28 × 8, engines 23 × 8 and 28 × 9): review
Union budget, 12 tables	No grid: only three horizontal rules per page. Columns by gaps: 9 on 11 of 12 pages (8 on one), matching both engines on 11

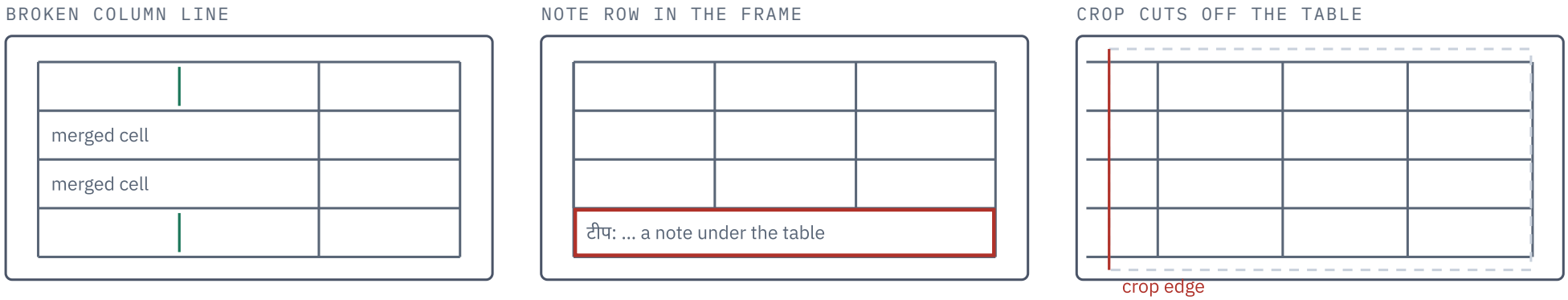
DECISION: THE LINES CHOOSE BETWEEN THE ENGINES' SHAPES, AND NOTHING MORE

When the engines disagree on the row count, trusting the lines to decide is safe only if the table has a full grid *and* the grid equals one engine's shape. That is what `table_qa.py choose_structure L247` does. Overriding both engines with the lines was never turned on: Marathi page 8 shows a table where the lines (28 × 8) match neither reading, and the right answer is a person looking at it. The same rule cannot help the Union budget at all, because it has no row lines; there the engines decide and cut-outs use row strips.

LIMITS

- Skew beyond 2.5°, sideways scans, dashed or double lines and thick shaded header bands are untested on real pages (the benchmark scans are skewed by up to 2°).
- A table with lines in one direction only gets no row count from the image.
- The crop box comes from the engines' boxes; if both are badly wrong the crop is too.

FIGURE 6 **Rows and columns from the ruling lines**



Merged cells break a column line.
Counted if it runs rule to rule in at least 2 rows (and a quarter of them), with clean paper either side.

The last row is one wide note.
No inner column line crosses it, so it is not a table row: the row count stops above it.

A ruling line runs into the crop edge,
beyond the outermost column line: the box cut the table. It is widened once (1.2% → 4.5% each side) and read again.

Benchmark: 322 of 322 ruled tables read exactly (born digital and scanned); no grid claimed on any of 172 unruled tables.

Three real defects, three fixes, each measured. Before them the benchmark read 309 of 322 ruled tables exactly; now all 322. On real documents the fixes took Marathi page 1 from "conflict" to a decision (Sarvam had dropped the total row and Bodhan had it) and confirmed page 2. The lines only choose between the engines' shapes: if no engine has the shape the lines show, the table goes to review and nothing is overridden.

HOW TO READ IT

1. Each panel is a schematic of one real defect of the line reader and the fix that was measured. Red marks what the old reader got wrong; green marks the line that is now counted.
2. Left: merged cells interrupt a column line, so it no longer runs a third of the table's height; it is still counted when it runs rule to rule in at least two rows. Middle: the last row is a note inside the frame; no inner column line crosses it, so it is not counted as a row. Right: a row line runs into the crop's edge, so the box cut the table; the crop is widened once.
3. The measured result under the panels is the check that the fixes did not cause false grids: 172 unruled benchmark tables, none read as ruled. Code: `table_geometry.py analyse`.

16 Repairing a scrambled text layer

`src/layer_repair.py` (146 lines, free) exists because of one observation: the Union budget is born digital, but its text layer contains *neither* engine's reading of most text cells, so a layer that should be the best evidence is useless. The cause is the font.

16.1 What is wrong with the layer

Some Indian government PDFs use fonts that store conjuncts and vowel signs under unrelated code points, the same way every time. Measured on the Union budget:

KIND	GLYPH IN THE LAYER	STANDS FOR	SEEN AS
sub: a fixed string	ơ	क्त	अơ॒ = अक्तू.
prebase: a vowel sign stored *before* its consonant cluster	ŏ	ि	ŏकमी = किमी
postbase: a sign stored after the cluster it belongs before (the reph)	[	र्	वाŏष[की = वार्षिकी

16.2 Learning the mapping

`layer_repair.py learn L83` takes, for every table, the layer's text and the list of values both engines agree on, and learns per document:

1. For each agreed value (4+ characters) it finds the best matching stretch of the layer (`layer_repair.py _best_alignment L63` tries starts a few characters back, because one glyph can stand for a longer string, and keeps the one with fewest differences) and lines the two up character by character with difflib.
2. Every *odd* glyph in the layer (outside ASCII and the Indic blocks, or one of `[ ] { } | ~ ^ `` ; `layer_repair.py candidate L38`) records what it replaced, and in which way: a replacement (sub), a deletion followed by an insertion after the next stretch (prebase), or a deletion preceded by an insertion (postbase). A literal use (a real bracket) is recorded as "other" and counts against any rule.
3. A glyph becomes a **rule** only if it was seen at least 3 times and meant the same thing in at least 90% of them (`layer_repair.py _learn_round L94`). A glyph that means two things depending on context, such as ŏ, is refused.
4. Learning runs in up to 4 rounds: each round applies what is known first, so a word with two odd glyphs (एऍसमैेसवे = एक्सप्रेसवे) yields the second once the first is understood.

`layer_repair.py apply L135` rewrites a layer with the rules: fixed strings first, then pre-base signs moved after their consonant cluster (`layer_repair.py CLUSTER L58` is a regular expression for a consonant cluster in any Indic script), then post-base signs moved before it.

The assembler runs this after all tables are built (`assemble.py Assembler._repair_layers L494`): it learns from *all* tables of the document together, re-runs `table_qa.py check L276` with the repaired layer for every table whose layer changed, and never trades a proven layer for an unproven one. A repaired layer is evidence only after it passes the same proof as any layer (section 14.7).

16.3 Result

On the Union budget: 13 rules learned; tables whose layer proves itself went from 1 to 4 of 12; about 4% fewer questions reach a model. Modest, and free. It also shows the limit: some glyphs are context-dependent and stay unrepaired. On the Marathi report (legacy fonts, a different mapping) the learner found few rules and the layer stays unusable.

READ THE CODE

- `layer_repair.py learn L83`, `layer_repair.py _learn_round L94` and `layer_repair.py apply L135`, 146 lines in all. `tests/test_layer_repair.py` builds a scrambled layer from known words and checks that all three rule kinds are recovered, that a glyph used two ways is refused, and that three examples are the minimum.

17 Cut-out sheets

When the free check leaves cells open, a model is shown *small numbered pictures of the table*, not the page. `src/table_cutouts.py` (172 lines) makes the pictures. A sheet costs far fewer tokens than a whole table, and the model does not have to find "row 15" by counting.

17.1 The cutter

`table_cutouts.py Cutter L49` is built for one table. It reuses the geometry the free check already computed (a second render and analysis is skipped; this alone saved about 1 to 2 s per table) and records:

- `exact_rows`: the ruling lines gave exactly as many rows as the table has, so each row's band is known to the pixel; `exact_cols`: columns match too, so each cell's box is known;
- row `bands`: from the row lines if exact; else from the text lines found in the image (`table_cutouts.py _text_lines L43`, rows with at least 1% ink in a run of 4+ pixels) if their number equals the row count; else proportionally between the first and last rule. These are guesses, which is why scanned unruled tables get no model decisions (chapter 18).

17.2 The two sheet formats

	FORMAT A: ROW STRIPS (THINK)	FORMAT C: CELL SNIPPETS (NORMAL)
What is shown	The whole row as a strip, with the asked cell inside a red box when the lines give the cell; the header strip (HEAD) once on top	Only the cell, under the header cells of its column, red box around it
Questions per sheet	12 (`table_cutouts.py PER_SHEET L24`)	24
Needs exact cell boxes	No (without boxes the question names the column and the other cells of the row in words)	Yes. Without boxes it falls back to row strips
Cost per question	about ₹0.015 per model	about ₹0.005 per model where boxes exist (`table_judge.py INR_PER_QUESTION L42`)
Pilot (391 real questions)	More correct choices by about 3 in 90 decided; harm 0	Harm 0; about 3 more wrong-but-harmless misses in 90
Built by	`table_cutouts.py Cutter.strip L75`, `table_cutouts.py Cutter.header L116`, `table_cutouts.py sheets L147`	`table_cutouts.py Cutter.cell_snippet L96`, `table_cutouts.py _grid_sheet L121`

Details: strips are cut at 200 dpi from the straightened image, padded by one row of context when the rows are not exact, drawn with a 3-pixel red rectangle (220, 0, 0) around the asked column when cell boxes exist, scaled to at most 1,200 pixels wide, and given a 70-pixel left margin that carries the question id (Q1, Q2 ..., in blue). Snippets are laid out left to right and wrap at 1,200 pixels.

DECISION: FORMAT C FOR NORMAL, FORMAT A FOR THINK; LETTER-ONLY ANSWERS REJECTED

Format C is about half the price where cell boxes exist and gave harm 0, with slightly more harmless misses, so Normal uses it. Think keeps the slightly more accurate format A. A third variant was tried and rejected: asking the model only to *choose a letter* (A or B) without transcribing the cell. Anchoring on the candidates raised the number of correct cells changed to wrong (Flash-lite: 66 against 36 with transcription; Luna: 24 against 20), and Flash-lite answered only 101 of 268 questions in the required format. The model must first read the cell, then say which candidate it matches.

LIMIT: FORMAT C DOES NOT HELP WHERE THERE IS NO GRID

The Union budget has no vertical lines, so the cell-only saving cannot apply: its cost fell only from about ₹0.30 to ₹0.26 per table page. A way to crop cells without a grid (using the PDF's own word positions) was considered and set aside as too fragile for the variety of layouts.

18 The image check: a validator, not a corrector

The cells that free evidence could not settle go to a vision model, as cut-out sheets. The model **chooses between the engines' readings; it never overrules both**. This chapter describes the rule (`src/table_judge.py`, 262 lines), the model client (`src/agent.py`, 330 lines) and why the rule is shaped as it is. Figure 7 (after this chapter) shows it.

18.1 Why a validator

The first idea was a corrector: show the model the crop and let it write what it sees. A pilot of 395 real questions with known answers (`bench/pilot.py`, 21 October) showed that fails. Of 229 disputes (Tamil numerals left out), the right value was one of the two engines' readings in 179 and *neither* in 50. The best model read an engine's reading right in most of the 179 but the missed value in only 8 of 50 (Gemini 3.8 Flash: 4 of 14 on scanned ruled, 3 of 35 on scanned unruled). And when two model families agreed on a value neither engine produced, that value was the truth 0 times of 25 (Flash-lite with Luna) and 3 of 24 (Gemini 3.8 with Sonnet). A model that overwrites freely changed 20 to 38 correct cells to wrong ones in the pilot, Claude Sonnet 5.5 included. The trained OCR engines are the readers; the model only chooses between them when it can see the answer clearly.

PAGE TYPE	TRUTH IS ONE ENGINE'S READING	BEST MODEL READS IT	TRUTH IS IN NEITHER READING	BEST MODEL READS IT
Born digital	25	24	1	1
Scanned, ruled	55	52	14	4
Scanned, unruled	99	54	35	3

18.2 The rule

`table_judge.py settle L150` asks the models about the open cells of one table and applies this rule, which was chosen on the pilot:

ROUTE (`TABLE_JUDGE.PY ROUTE L85`)	MODELS	ACCEPT A CELL WHEN	PILOT
Born digital	GPT-5.6 Luna	its reading equals one engine's reading	24 of 26 right, harm 0
Scanned, ruled (cell boxes exist)	Normal: Gemini 3.1 Flash-lite, then Luna. Think: those two plus Gemini 3.5 Flash-Lite	Normal: both read the same value and it equals an engine's. Think: any 2 of 3 read the same value that equals an engine's	Pair: 45 of 51 right, harm 0 (38 of 44 on the 63 disputes of the Think comparison). 2 of 3: 46 of 56, harm 0
Scanned, unruled	none	never: the row strips are guesses	about 40% right with harm 7 to 17: no model is used

In code, `need = min(profile.vote or len(models), len(models))`. The models are asked **cheapest first**, and a later model sees only the cells it can still help decide (a cell not yet settled, with enough models left to reach `need`), so the decisions are the same as asking everyone, at lower cost. The helper `table_judge.py _agreeing L141` returns the engine reading most models gave and who gave it; `table_judge.py _matches_engine L104` is the "equals an engine's reading" test (equal after `canon`).

What a model may never decide

GUARD	WHY	WHERE
A value no engine produced	Accepting only engine readings is the whole rule	`table_judge.py settle L150`
An empty cell over printed text	An empty reading is also what a missed red box looks like. A Marathi cell's text was "deleted" this way in the pilot	`table_judge.py settle L150`
Rare numerals and look-alikes (Tamil, Telugu digits)	Every model tried read Tamil numerals as letters: at most 3 of 45 right. They need a rule or a person, not a model	`table_judge.py NOT_FOR_MODELS L54`, `table_judge.py for_models L57`

GUARD	WHY	WHERE
A cell where a clean proven layer contains neither reading	The value is probably a third one; choosing between two wrong readings was the main born-digital error	same set
"Confidence"	It said "high" on blurred digits it got wrong, so it is ignored	`agent.py JUDGE_SCHEMA L269` asks for it, nothing uses it
A scanned table without ruling lines	No reader was reliable there	`table_judge.py route L85`

18.3 Asking: one sheet at a time

`table_judge.py _ask L117` builds the sheets (chapter 17), then for each sheet a list of questions: `id` (Q1, Q2 ...), `where` (*inside the red box*, or, without a box, *column headed '...', in the row that also contains: ...* with up to three other cells of the row) and, for a dispute, `candidates` {A: reading of engine 1, B: reading of engine 2}. `agent.py Agent.judge_cells L247` sends the sheet and the questions and expects JSON. The prompt (`agent.py JUDGE_INTRO L291`, `agent.py JUDGE_STYLE L300`) says: the image is a sheet of strips from one table; the top strip is the header; every other strip carries a question id; read the cell exactly as printed, keeping the script of every letter and digit (do not turn Indian digits into 0–9), keep punctuation; an empty cell is an empty string; if you cannot read it, say low confidence; then set `match` to the letter of the candidate that is exactly what is printed, or `none`. The model must *transcribe first*, then match: the reading is what gets compared.

Answers have a strict schema (`agent.py JUDGE_SCHEMA L269`: `answers[{id, reading, match, confidence}]`); some open models skip the wrapper or rename ids (`Q1_red_box`), which `agent.py _parse_json L310` and the id clean-up in `agent.py Agent.judge_cells L247` repair. The token budget is 200 + 90 per question, plus 3,000 more when the model reasons (reasoning tokens count against `max_tokens`; a truncated answer otherwise).

18.4 Audits and the seam check

- **Audits** (`table_judge.py audit L193`). Agreed cells that look risky (chapter 14.8) are read *blind* (no candidates) by one model, GPT-5.6 Luna. A different reading adds `audit_disagrees` (high risk); the same adds `audit_confirmed`. An audit never changes a value and never cancels a high-risk flag. `assemble.py Assembler._image_check L516` takes the riskiest cells of the whole document, not already headed for review, up to the mode's number (0, 5, 15), and groups them by table. Measured: audits flagged only 9 of 35 wrong agreed cells and also flagged 33 right cells, so they are a flag, not a judge.
- **The seam check** (`table_judge.py seam L228`). When the join score is 2 or 3 (chapter 19), one picture is made: the header, the last three rows of the table above, a red line for the page break and the first four rows of the next (HEAD, END, NEXT). Luna answers (`table_judge.py SEAM_SCHEMA L215`): does it continue, how many leading rows repeat the header, is the first row the rest of the last row. Without a model the join is queued for review.

18.5 The model client

`agent.py Agent L98` talks to any OpenAI-compatible server: OpenRouter now, a `llama-server` on the RTX 5070 later (set `AGENT_BASE_URL`). Every call goes through `agent.py Agent._chat L168`:

1. **Cache first.** The key is the message list as JSON, the model and the purpose, base URL and schema (section 3.4). A hit returns at once with cost 0.
2. **Budget.** If the ledger says US$10 (`config.py OPENROUTER_BUDGET_USD L135`) is spent, the call is not made and the caller falls back to its rule (cells stay open). The ledger (`out/openrouter_ledger.jsonl`) gets one line per call; `agent.py Agent._cost_usd L145` prices it from OpenRouter's model list (cached for a day; an unknown model is assumed dear).
3. **Request.** Temperature 0, strict JSON schema, retries through `rate_limit.py call_with_retry L228` (3 tries) behind the model limiter (120 starts per 62 s, 8 in flight; `config.py AGENT_MAX_INFLIGHT L138`).
4. **Reasoning.** By default reasoning is switched off on OpenRouter. Some models refuse that ("Reasoning is mandatory", HTTP 400): Gemini 3.8 Flash, Claude Sonnet 5.5 and Gemini 3.5 Flash-Lite. For those the call is made with the provider's default or `effort: low`; OpenAI models take `effort: minimal` (`table_judge.py EXTRA L39`).

MODEL (OPENROUTER ID)	US$ PER MILLION TOKENS IN / OUT	₹ PER SHEET; WALL TIME OF THE 395-QUESTION PILOT (4 WORKERS)	ROLE
openai/gpt-5.6-luna	0.20 / 1.20	about 0.11; pilot 371 s	Born digital; second model on scans; audits; seam checks
google/gemini-3.1-flash-lite	0.25 / 1.50	about 0.08; pilot 104 s	First model on scans
google/gemini-3.5-flash-lite	0.30 / 2.50	about 0.12; pilot 43 s	Think's third model on scans
google/gemini-3.8-flash	0.825 / 4.125	about 0.24	Tried; harm 13 alone on scanned ruled; not used
anthropic/claude-haiku-5.5	0.10 / 0.50	about 0.03	Tried; harm 13 on scans
anthropic/claude-sonnet-5.5	2 / 10	about 0.69	Tried; harm 7 on scanned ruled

The legacy path uses the other two names in `src/config.py`: `AGENT_MODEL` (Qwen 3.5-9B, to judge whether tables join) and `VERIFIER_VLM_MODEL` (Flash-lite, the old cell tie-breaker); see chapter 23.

18.6 The pilot

395 real questions from the real engine readings of the benchmark (274 disputes and 121 audits), each model run on all of them, accepted only under the rule above. "Right/decided" is the number of cells the rule changed that became right out of those it decided; harm is correct readings changed to wrong ones.

MODEL OR RULE	BORN DIGITAL: RIGHT/DECIDED, HARM	SCANNED RULED	SCANNED UNRULED	PER SHEET
No model (default reading)	15/26	43/69	28/134	₹0
Gemini 3.1 Flash-lite	16/20 · 0	51/65 · 2	43/115 · 15	₹0.08
Claude Haiku 5.5	15/21 · 0	38/66 · 13	47/130 · 11	₹0.03
GPT-5.6 Luna (chosen, born digital)	**24/26 · 0**	49/69 · 2	43/134 · 10	₹0.11
Gemini 3.8 Flash	24/25 · 0	52/64 · 1	53/116 · 17	₹0.28
Claude Sonnet 5.5	24/24 · 0	45/64 · 7	50/115 · 11	₹0.69
Flash-lite + Luna must agree (chosen, scans)	16/18 · 0	**45/51 · 0**	26/76 · 7	₹0.19
Gemini 3.5 Flash-Lite (22 October, same questions)	19/22 · 0	42/50 · 0	34/108 · 17	₹0.12

On the 63 scanned ruled disputes that decide Think's third model:

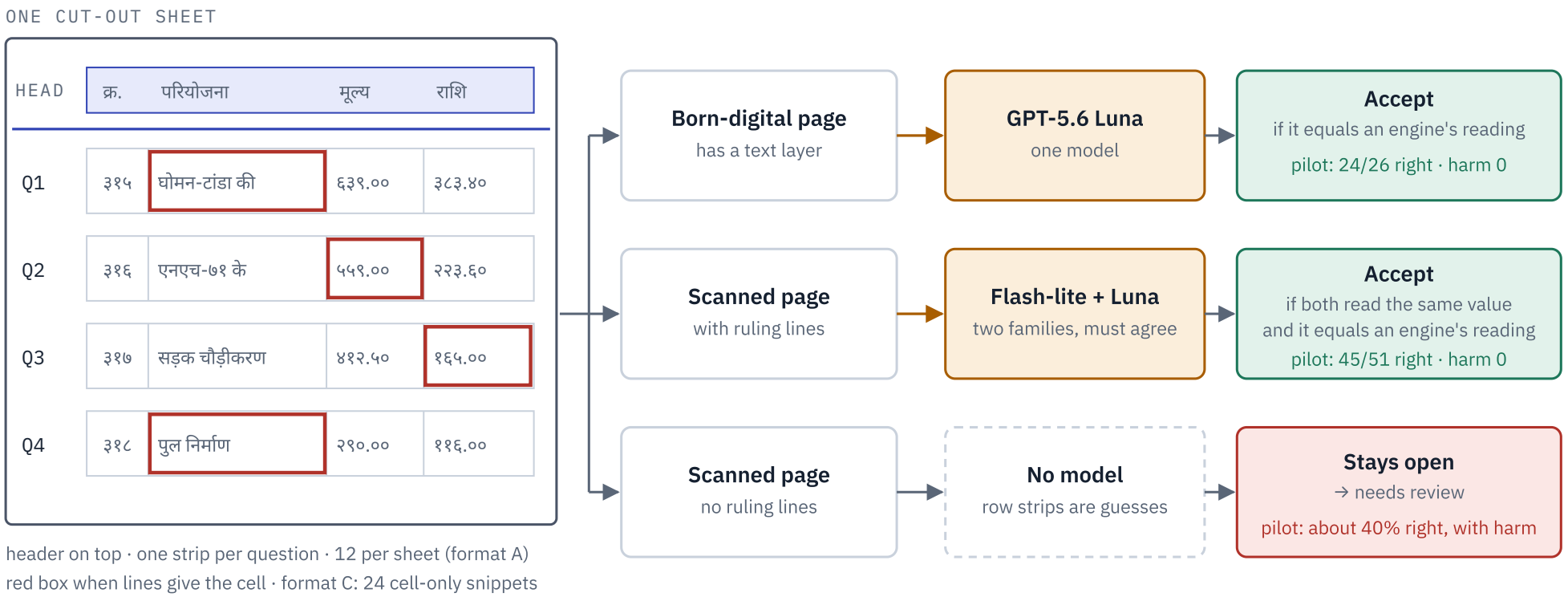
SCANNED, RULED	DECIDED	RIGHT	WRONG, HARMLESS	HARM
Flash-lite + Luna must agree	44	38	6	0
3.5 alone	50	42	8	0
3.8 Flash alone	58	27	31	13
Flash-lite, Luna, 3.8 Flash: 2 of 3	52	41	11	0
Flash-lite, Luna, 3.5: 2 of 3 (Think)	56	46	10	0

Gemini 3.5 as a replacement for Luna on born-digital tables was rejected: it settles fewer cells (22 against 26 of 26 disputes) and gets about 5 fewer right, for 12% more cost per question; its advantage is speed (the pilot took 43 s against Luna's 371 s with four workers). On scanned ruled tables a single 3.5 looks good (42 of 50, harm 0) but the sample is only 63 disputes, the same model with another reasoning setting gave one harm, and every other single model caused 2 to 3, so the pair rule stays in Normal. A small open-weight pilot was stopped: Qwen 3.8-27B finished with 100 of 274 right and 27 correct cells made wrong, no better than the closed models, and Gemma 4 mostly timed out.

READ THE CODE

- `table_judge.py` **`settle`** `L150` (about 45 lines) is the rule. `assemble.py` **`Assembler._image_check`** `L516` is the scheduling around it: a thread pool sized to `AGENT_MAX_INFLIGHT`, one PyMuPDF document per thread, each table's cost recorded in the manifest as a `table_image_check` extra, then the audits. With `--max-cost-inr` it reserves each table's worst case and retries tables turned away once others have settled at their (often much lower) actual cost; without it nothing is ever turned away.
- `agent.py` **`Agent._chat`** `L168` for the call; `bench/pilot.py` and `bench/pilot_report.py` for how the table above was made.
- Tests: `tests/test_table_qa.py` has the rule with a fake model: one model accepted only if it matches an engine; a value no engine produced rejected; two models must agree on scans; printed text never deleted; the cascade skips the second model when the first cannot decide; Think's 2 of 3.

FIGURE 7 **Image check: a validator, not a corrector**



"Harm" is a correct reading changed to a wrong one. Letting any single model overwrite cells freely made 20 to 38 correct cells wrong in the pilot, Claude Sonnet 5.5 included. Requiring the model to agree with one engine, and two model families to agree on scans, brought harm to zero where the image is clear. Normal uses format C (24 questions per sheet; the cell and its column header only, where ruling lines give exact boxes), which costs about half as much and gave about 3 fewer right answers in 90. Think uses format A (12 whole row strips per sheet).

HOW TO READ IT

1. Left: one cut-out sheet in format A. The header strip is on top; each numbered strip is one question, with a red box on the asked cell when the ruling lines give it. Format C shows only the cell under its column header.
2. Right: three routes by page type. Born-digital pages go to one model (Luna). Scanned pages with ruling lines go to two models of different families that must read the same value (Think: any 2 of 3). Scanned pages without ruling lines get no model.
3. In every route the accepted value must equal one of the engines' readings (green boxes). Amber boxes are model calls; the red box is the case left for a person. Code: `table_judge.py settle`.

19 Joining tables across pages

A long table is printed over many pages and each page is read separately, so the engines return many table fragments. `src/table_join.py` (247 lines) decides whether a fragment continues the table above it and merges the two. A long table keeps one identity: once two fragments are joined, every later page is compared with the table's *first* fragment, so a 20-page table is not 19 separate guesses. Figure 8 (after this chapter) shows the idea.

19.1 The signature

`table_join.py signature L76` describes a fragment with a `table_join.py Signature L46`:

- `n_cols` and the `header` text of each column (all header rows joined);
- `kinds`: the dominant kind of content in each body column (`table_join.py kind_of L34`: number, date, text, empty; *mixed* if no kind has 60%);
- `columns`: the inner column boundaries as fractions 0–1 across the table, taken from the ruling lines (only for a ruled table whose line count matches the column count);
- `serial_col`, `first_serial`, `last_serial`: a serial-number column among the two leftmost (numbers going 1, 2, 3 …);
- `title`: the table number in the caption above it (`Table 3`, सारणी 3, அட்டவணை 3, పట్టిక, সারণি …).

19.2 The score

`table_join.py decide L129` compares the head fragment with the next one and adds evidence:

EVIDENCE	POINTS
Same column count	+1
Same column positions on the page (ruled tables, boundaries within 0.03)	+2
The next fragment repeats the head's header rows (similarity ≥ 0.75 per row)	+2
The next fragment has no header of its own (the typical continuation)	+2
Same kind of content in at least 80% of comparable columns	+1
Serial numbers continue (first = last + 1)	+2
A "continued" marker between the two (`contd.`, `continued`, क्रमशः, पुढे, தொடர்ச்சி …; `tables.py CONTINUED_RE L613`)	+2
A different header (similarity under 0.5)	−1
Different content kinds in under 50% of columns	−2
Serial numbers restart at 1	−1
A different table title (`Table 3` against `Table 4`)	separate at once
Column counts that cannot be mapped (`table_join.py map_columns L115`: identity if equal; by largest overlap of the ruled boundaries if both are ruled; else none)	separate at once (−5)

A score of **4 or more joins** (`table_join.py JOIN_AT L31`), **1 or less keeps them separate** (`table_join.py SEPARATE_AT L31`), anything between is **uncertain**: one picture of the seam goes to Luna (section 18.4), or, without a model, the join is queued for review and both fragments are marked *needs review*. A position mismatch alone is not held against a join, because a continuation without its header is often laid out narrower.

19.3 Looking ahead

`assemble.py Assembler._join_tables_standard L750` runs over the pages in order. For a page whose last body block is a table it takes the table's first fragment as the head and looks at the next **two** pages (the old joiner looked at one, and never skipped a page):

- `assemble.py Assembler._first_table L735` returns the first table that starts the next page's body, with up to two captions or headings before it (`between`, used for the "continued" marker and the title);

- a page with no table that holds only a figure or a caption (no table and under 200 characters of text) is *skipped*; any other page ends the search. This is what lets a table continue across a figure page, which the old joiner could never do (0 of 16 on the benchmark);
- if the columns cannot be mapped, the *other engine's* reading of that page is tried (an engine may have split or dropped a column) and kept if it scores higher (`version_switched_for_join`).

19.4 Merging and checking the seam

`table_join.py merge L193` appends the next fragment, with its columns mapped, without its repeated header, and mends what a page break does to a row:

- **A row cut by the page break**: the first incoming row continues the previous row when most of the *key columns* (`table_join.py key_columns L185`: filled in at least 90% of body rows) are empty in it, and what it does have is text (numbers are never cut). Its text is appended to the matching cells of the head's last row and the row is dropped. This works whatever the table holds, unlike the old rule (an empty first cell and no numbers).
- **A row printed on both pages**: if the head's last row and the incoming first row have identical text, one is dropped.

`table_join.py seam_checks L229` then looks at the seam only (not the whole column, which was an early mistake): the kind of content in each column must not change across it, and the serial numbers must continue from the last one above to the first one below. A problem marks the table *needs review* with the flag `seam_check_failed`. A joined table is only as good as its worst fragment: the status, the questions and the risky cells of both fragments are merged by `assemble.py Assembler._absorb L821`.

19.5 Measured

On the benchmark (v1 and v2, joined segments out of the cases of each style):

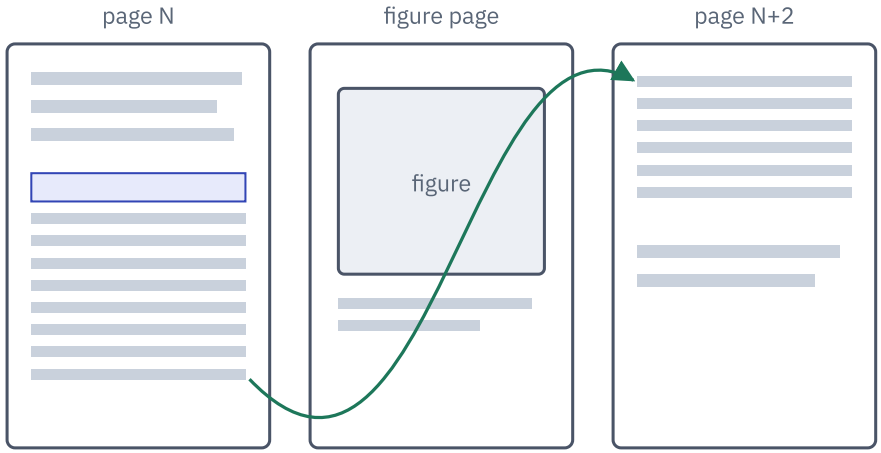
CASE	OLD JOINER (V1 · V2)	NEW JOINER (V1 · V2)
Split, no repeated header	17/18 · 13/14	17/18 · 14/14
Header repeated	16/16 · 13/14	16/16 · 14/14
Row cut by the page break	10/14 · 19/20	12/14 · 19/20
Across a figure page	0/16 · 0/16	16/16 · 16/16
Across a blank page	0/18 · 0/16	17/18 · 15/16
Two different tables wrongly joined	4/12 · 2/12	**0/12 · 0/12**

On the real Union budget, the 12 table pages become one table of 210 rows; the repeated two-level header is dropped at every join.

READ THE CODE

- `table_join.py decide L129` (one screen) and `table_join.py merge L193`; the loop in `assemble.py Assembler._join_tables_standard L750`. Tests: `tests/test_table_join.py` (8 tests: continuation without a header, a repeated header, different titles, unmappable columns, a cut row, a duplicated seam row, a seam kind change).
- The old page-pair rule is `tables.py continuation L639` with `assemble.py Assembler._join_tables L663` (chapter 23).

FIGURE 8 **Joining a table that continues on a later page**



joined across the figure page

compared with the table's first segment

columns mapped · header dropped · cut row mended

EVIDENCE FOR

+2 same column positions (ruling lines)

+2 repeats the header rows

+2 no header of its own

+2 serial numbers continue

+2 “contd.” / क्रमशः marker

+1 same column count

+1 same kind of content per column

AGAINST

a different table title → separate

columns cannot be mapped → use the other engine's reading of that page

−1 a different header

−2 different content kinds

−1 serial numbers restart at 1

≤ 1 separate	2–3 seam check	≥ 4 join

seam check: one picture of the end of the table above and the start of the next, to Luna; without a model the join goes to review

after joining: column kinds stay the same · serials continue at the seam · a row printed twice is dropped

A long table keeps one identity. Once two segments are joined, every later page is compared with the table's first segment, so a 20-page table is not 19 separate guesses. A row cut by the page break is recognised whatever the table holds: most of the columns filled in every other row are empty in it, and what it does have is text.

HOW TO READ IT

1. Left: a table ends page N, a figure page follows, and the same table continues on page N+2 without a header. The arrow is the join across the figure page; the segment is compared with the table's *first* segment.
2. Right: the evidence is added up. Points for a join (same positions, repeated or missing header, serial numbers continue, a continued marker, same kinds of content), points against (a different header, different kinds, serials restart; a different table title ends it at once). The coloured bar gives the decision: 1 or less separate, 2 to 3 ask for a seam picture (or review), 4 or more join.

20 The page-text check

Tables get cell-by-cell checking, but the paragraphs around them were read by one engine. In Think mode Bodhan reads every page, so the two readings of each page's prose can be compared word by word. `src/text_qa.py` (234 lines, free) does that, and settles a difference only when free evidence can pick between the two readings. Figure 9 (after this chapter) shows the flow.

20.1 The steps

1. **Words.** `text_qa.py words_of L117` lists the words of every prose block (kinds text, heading, caption, footnote, contents; `text_qa.py KINDS L45`) in reading order, with each word's block and character offsets. Its comparison form is `text_qa.py key_of L96`, which strips markup first: ^{22}, ^2 and ²² are the same footnote mark; \textit{x}, *x* and x the same word; ”, with or without its ampersand, a quote; then `table_qa.py canon L52`.
2. **Compare.** `text_qa.py compare L149` runs difflib on the two key lists. If under 60% of the words are common (`text_qa.py MIN_PAGE_OVERLAP L46`) the page is not compared: the engines read it in a different order. Each stretch that differs is a *dispute* with four words of context on each side.
3. **Decide**, in this order, each rule only choosing between the two readings (a dispute longer than 25 words, `text_qa.py MAX_SPAN L47`, is a layout difference and is not decided):
   - **elsewhere**: the words one engine has as prose, the other engine has in a block it does not count as prose (a running header, a figure's text, a table; `text_qa.py non_prose_of L129`). Keep the primary's.
   - **format**: one side is still LaTeX after normalising ($\frac{ds}{dt}$): a way of writing, not a misread. Keep the primary's.
   - **spacing**: equal after removing spaces. **minor**: equal after removing punctuation, symbols and marks of format, and no digits. Keep the primary's.
   - **text_layer**: the page's own text, proven on the words both engines agree on (at least 20 testable words, at least 90% found; `text_qa.py _layer_proof L142`), contains exactly one of the readings, or contains words that one engine has and the other lacks. The layer never *deletes* printed words: a word missing from the layer is not proof that it is not printed.
   - **open**: listed in the report.
4. **Apply.** `text_qa.py apply L204` puts the decided readings into the primary engine's block text, *in memory only*: the saved block files are never changed. `assemble.py Assembler._check_text L476` calls this for every page both engines read, before the document is composed.

20.2 What the first measurement showed

The check was built after measuring, on the seven sample documents with both engines on every page (11,669 prose words). The first version found 351 open differences; almost all were not misreads. Two normalisations removed the noise: markup (Bodhan writes footnote marks and inline maths as LaTeX, 198 false differences) and words one engine files as a header and the other as prose.

DIFFERENCE	COUNT	EXAMPLE
format	188	`ds/dt` against $\frac{ds}{dt}$ (NCERT maths)
minor	59	Bodhan's line-break hyphens: `liberal- ism` against `liberalism` (53 in the English book); Tamil பள்ளியரே! against பள்ளியரே
elsewhere	24	"External Sector", a running header, prose in one engine
spacing	17	
text_layer	2	A source line under a chart that Sarvam had dropped; the proven layer contains it
layout	1	
open	**57**	0.5% of the words; mostly Telugu and legacy-font Hindi spellings, e.g. బడ్జెటింగ్ against బడ్జెట్టింగ్, తెలరియుం against తెలయుం

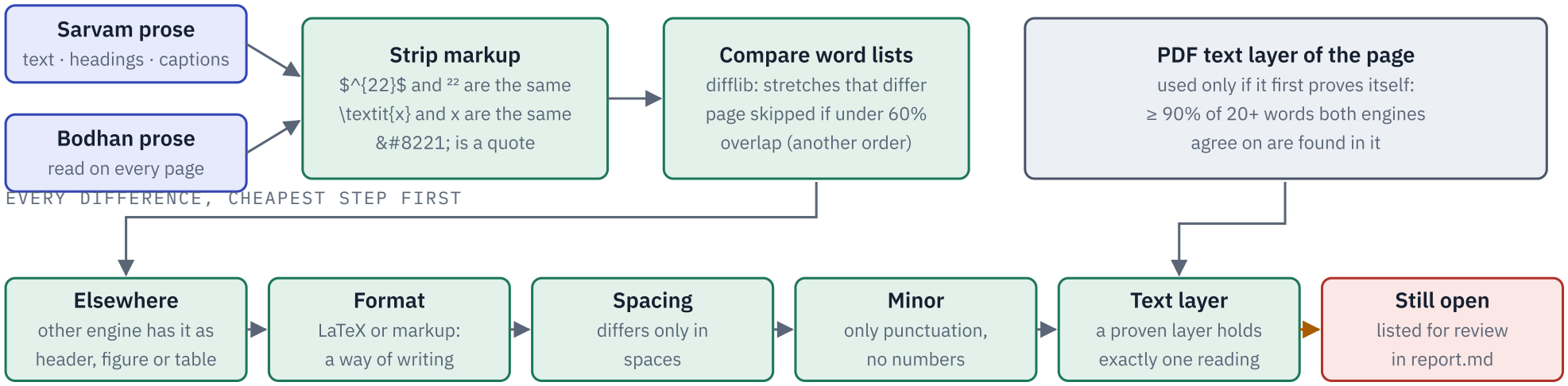
No model reads text yet. There is no text with known answers to measure one against, so the rule "a model must not do harm" could not be checked. The Mozhi word-crop sets (Assamese, Gujarati, Tamil, Kannada, with known words) are the intended measure when this is built. The disputes that stay open are listed in the report (section "Page text") with their context and are

summarised by `bench/text_dispute_report.py`.

READ THE CODE

- `text_qa.py compare L149` (55 lines) is the whole algorithm; `text_qa.py apply L204` and `text_qa.py summarise L223` finish it. Tests: `tests/test_text_qa.py` (10 tests: identical text, spacing, punctuation, an open misread, a proven layer choosing and being applied, an unproven layer deciding nothing, the layer never deleting words, a different reading order, markup normalisation, headers and figure text).

FIGURE 9 **The page-text check**



A decider only picks between the two readings. It never writes a word, and it never deletes printed words.

Most differences are not misreads. Bodhan writes footnote marks and inline maths as LaTeX (^{22} where Sarvam has ²²), which made 198 false differences before markup was stripped. One engine often files a running header as prose and the other as furniture. Bodhan keeps line-break hyphens (`liberal- ism`); the free rules prefer Sarvam's joined word, which is right.

HOW TO READ IT

1. Top row: both engines' prose becomes word lists with markup stripped, the lists are compared, and the PDF's own text is available as evidence only if it has proved itself on the words both engines agree on.
2. Bottom row: every difference passes the free deciders in order and stops at the first that applies. Only the last box is a real misread that nothing free could settle. Code: `text_qa.py compare`.

21 Writing the outputs

After the tables are checked and joined, the assembler writes the document and everything that explains it. Every file is plain text, JSON or CSV.

21.1 doc.md

`assemble.py Assembler._compose L863` walks the pages in order and writes the *primary* engine's blocks:

- A comment per page: `<!-- page 92 (sarvam) · languages: hi-IN 98% -->`, and `<!-- lang: en-IN -->` before a block whose language differs from the page's.
- **Headings** as # marks (level + 1, at most 6); **captions** in italics; **math** as LaTeX ($$ blocks); **footnotes** as a blockquote at the end of the page; **furniture** dropped, except that a printed page number is remembered.
- **Prose** goes through `assemble.py cleanup_text L117` (Bodhan's LaTeX italics and bold become Markdown, superscripts become <sup>). A paragraph split by a page break, a column break or a drop cap is joined: `assemble.py Assembler._continues L937` is true if the previous text ends with a hyphen and the next starts with a letter, or if the previous does not end with a sentence-ending mark and the next starts lower case; `assemble.py Assembler._join_prose L945` removes the hyphen. The counts are in the report.
- **Tables**: a comment `<!-- table t00090_0: pages 90–101, sarvam, checks 198/199 passed -->` and the table's HTML (`tables.py Table.to_html L134`), at the first fragment only: joined fragments are not written again.
- **Figures**: the block's box is cropped from a 150 dpi render with a 0.5% margin into `figures/pNNNNN_k.png` and linked as an image with its text as alt (`assemble.py Assembler._figure L854`).

21.2 Tables, report and JSON

FILE	HOLDS	WRITTEN BY
`tables/<id>.html · .csv`	Every joined table, once, with merged cells in the HTML and expanded text in the CSV	`assemble.py Assembler._write_tables L952`
`tables/<id>.json`	The table's metadata (`assemble.py TableRec.meta L183`: pages, rows, columns, status, structure decision, `qa_cells` with every reading and how each cell was settled, questions, image-check report, join decisions) plus `rows_text`	same
`assembly.json`	The machine-readable result: `document, pages, primary_engine, run, mode, triage` (including page classes), `layout` (which labels each engine used), `tables` (found, after joining, status counts, how cells were settled, questions left, learned layer rules, the list), `joins`, `text_check`, languages, cost (pages, assembly, total, extras by kind), assembly wall time	`assemble.py Assembler._result L1047`
`report.md`	The same in words: pages, run, cost, triage, the status of every table and how each cell was settled, what is left for review, every join with its score and reasons, language mix, the page-text check, the layout labels seen, printed page-number gaps	`assemble.py render_report L1113`

The **cost** block separates three numbers: the cost of the pages (Stage 1), of this assembly (what *this* run of Stage 2 spent, zero when everything came from the cache) and the total, with the model calls broken down by kind.

21.3 The three statuses

STATUS	MEANING
verified	Nothing open, no high-risk flag, the structure agreed or was confirmed by the image
repaired	Something was settled by free evidence or the image check, and nothing is left open. Cells settled by a model are labelled `image:<model>`
needs review	Open cells, a high-risk flag, a structure conflict, an uncertain join or a failed seam check. The cells and reasons are listed

A table status is one flag. On real documents most tables end *needs review* because of a few audit flags or open cells; the report says exactly which, so a person reads those cells and not the whole table.

A BUG FOUND WHILE WRITING THIS REPORT

`assemble.py` **`render_report`** `L1113` read `agent['confidence']` for every join the model had judged. A seam check (standard mode) records no confidence, so the report would have crashed on the first real seam check; none had happened yet because uncertain joins are rare. It now prints `seam check: <reason>` for those.

22 Modes, page classes and the estimate

Three small modules turn the pipeline into something a user can steer: `src/modes.py` (73 lines) says how much to read and check, `src/page_class.py` says what kind of page each page is, and `src/estimate.py` (216 lines) says what a run will cost before it starts. Figure 10 and Figure 11 (after this chapter) are the pictures.

22.1 Modes

A mode is data, not code. `modes.py Mode L47` has a name, the Stage 1 `policy`, whether the image check runs (`image_check`), a `modes.py JudgeProfile L36`, the number of `audits`, and whether the page-text check runs (`text_check`). The profile holds the models for born-digital tables, the models for scanned ruled tables, the `vote` (how many must agree; None means all), the sheet size (`per_sheet`) and `cell_only`. The three modes live in `modes.py MODES L57`:

	FAST	NORMAL (DEFAULT)	THINK
Policy	sarvam-primary	sarvam-primary	both (Bodhan reads every page)
Image check	none (no model calls)	yes	yes
Born-digital models	-	Luna	Luna
Scanned ruled models	-	Flash-lite, Luna: both must agree	Flash-lite, Luna, Gemini 3.5 Flash-Lite: 2 of 3
Sheets	-	24 per sheet, cell snippets (format C)	12 per sheet, row strips (format A)
Audits per document	0	5	15
Page-text check	no	no	yes
Image-check cost, dense Hindi table page	₹0	about ₹0.26	higher

How a mode takes effect: `run.py main L385` sets the policy from the mode (unless `--policy` overrides it) and passes the mode's name to the assembler; `assemble.py Assembler L200` drops its agent when `image_check` is false, hands the profile to `table_judge.py settle L150` and `table_judge.py audit L193`, uses `audits` to cut the audit list and runs `assemble.py Assembler._check_text L476` when `text_check` is true. `python -m src.assemble <dir> --mode think` assembles a finished run again in another mode; only questions not already in the cache cost money, so a user can start with Fast and upgrade later. Every mode keeps the validator rule of chapter 18.

> **DECISION: NO SPENDING CAP INSIDE THE MODES**
>
> A first design gave Normal and Think a ceiling per table page (₹0.15 and ₹0.40) and checked tables with fewer open cells first. It was removed on the user's instruction: a cap only leaves tables unchecked, which defeats the mode. A run can still be limited with `--max-cost-inr`, an opt-in that makes the assembler reserve each table's worst case and retry tables that were turned away once others have finished at their actual cost.

> **LIMIT: THINK IS NOT ALWAYS BETTER THAN NORMAL**
>
> On the Union budget Think left 10 open cells against Normal's 3; on the Marathi report 16 against 21. Model answers vary a little between runs, and the pilot's gap between the two sheet formats is about 3 cells in 90. Think's real gains are the third judge on scanned ruled tables (46 right of 56 decided against 38 of 44, harm 0 in both) and the second reading of ordinary text.

22.2 The page classes in the pipeline

Chapter 5.3 defines the classes. Where they are used: `assemble.py Assembler._scanned L417` is simply `render == "raster"` and decides the image-check route; `assemble.py Assembler._layer_text L420` returns the PDF's text only if the page's text-layer class is *unicode* and it has at least 50 letters; `estimate.py preflight L87` classes a sample of pages; `--doc-type digital|scanned` (stored in the settings fingerprint only when it is not `auto`) forces the class of every page in triage.

22.3 The estimate

`estimate.py preflight L87` takes the PDF and the page selection and reads at most 40 pages, one at a random place in each stretch of the selection (a fixed seed; every page of a short document). Even spacing would alias with a regular layout, such as a table on every third page. For each sampled page it:

1. classes the page (`page_class.py classify L67`, with a light legacy-font test, `estimate.py _legacy L58`);
2. decides whether it has a table: for a vector page, PyMuPDF's table finder or at least 60 number-like words; for a scan, a ruled grid in a 60 dpi render (`estimate.py _scan_has_grid L67`). A scan whose table has no lines cannot be seen this way and only widens the upper end;
3. scales the share found to the whole selection with an interval (`estimate.py _interval L75`: exact when every page was read, otherwise an 80% Wilson interval).

`estimate.py mode_estimate L128` then computes, for each mode, a range for:

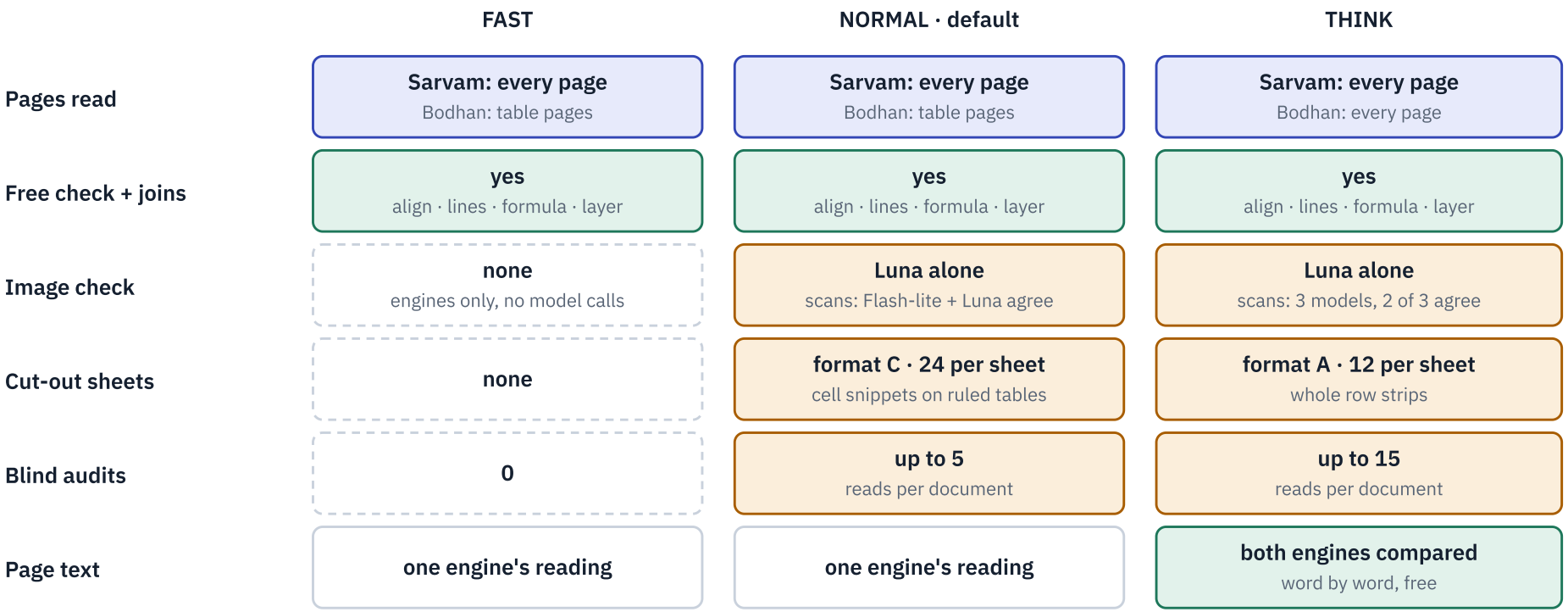
PART	HOW
OCR cost	Sarvam: pages × ₹0.50. Bodhan: table pages × ₹0.20 (every page in Think)
Table-check cost	Per page class: table pages × open questions per table page [low, high] × the price per question of each model of the profile (boxed price for the low end, row price for the high end). Questions per table page: vector/unicode 2–20, vector/legacy 2–10, scanned 0–32 (measured). A scanned table's second model sees about 60% of the cells, a third model about 25%
Time	Stage 1: 0.3–1.0 min of overhead plus the slower of Sarvam (15–30 pages a minute) and Bodhan (8 a minute). Stage 2: table pages × 0.5–1.5 s of free work, plus sheets × 10–30 s ÷ 8 calls in flight

The coefficients are in `src/estimate_coefficients.json`. `estimate.py render L193` prints the table shown in chapter 1 with an arrow at the chosen mode; after the run `run.py main L385` prints the estimate and the actual cost and time together and `estimate.py record L210` appends both to `out/estimates.jsonl` so the coefficients can be refitted.

CHECK AGAINST MEASURED RUNS	ESTIMATED	ACTUAL
Union budget pp. 90–101, OCR	₹8.40; 2–3 min	₹8.40; 94 s Stage 1 plus 44–57 s Stage 2
Economic Survey window, Think (Bodhan on 5 new pages)	₹5.69–6.50 total	₹1.0 (Sarvam and 3 Bodhan pages were cached)
NCERT, Think	₹9.25–9.85 total	₹8.14 including the ₹6.50 mistake of section 3.4

> **LIMIT**
> The estimate does not subtract pages already in the cache, so a repeat run costs less than it says (the table says so under it). The table-check range is wide on purpose: the number of open cells per table page varies from 2 to 20 between documents.

FIGURE 10 **Fast, Normal, Think**

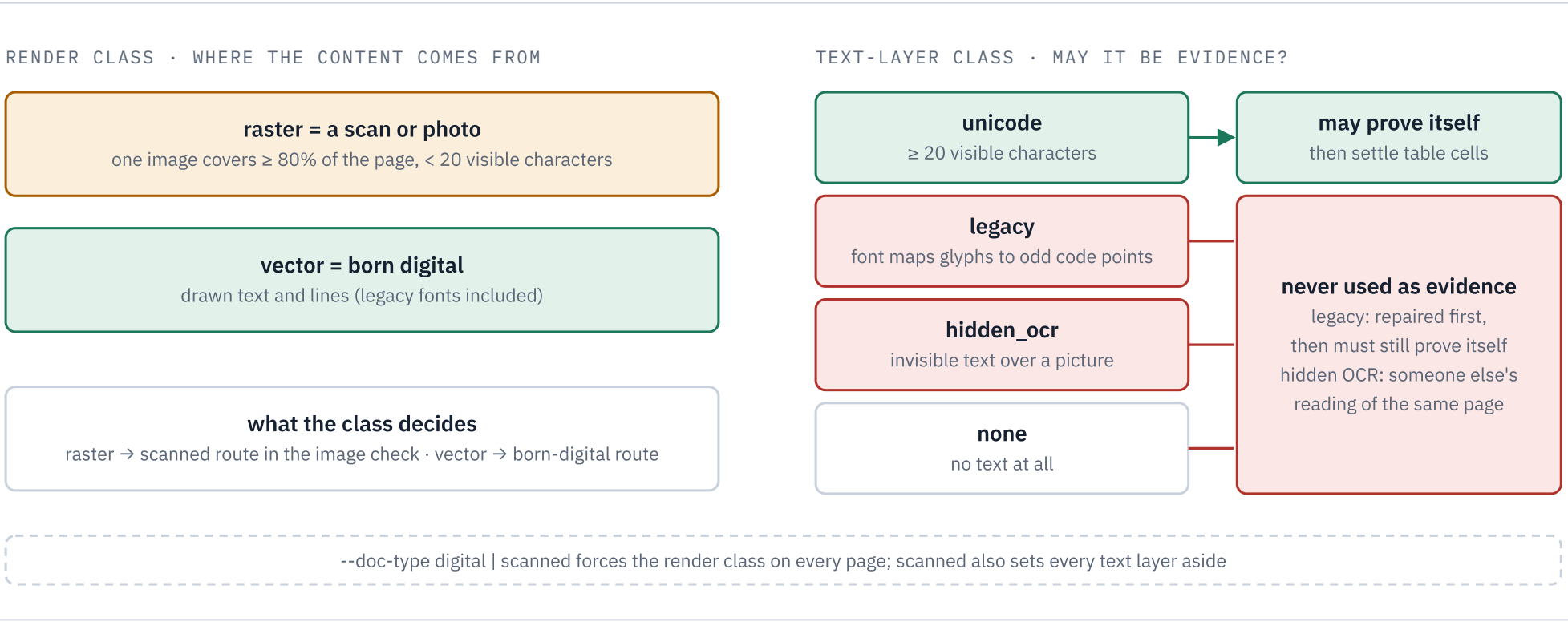


Each mode adds a layer. Fast stops after the free checks and costs nothing beyond OCR. Normal adds the image check on the cells the free checks could not settle. Think reads every page with both engines, asks three models on scanned ruled tables and takes the answer 2 of 3 agree on, and compares the two engines' page text. A finished run can be assembled again in another mode; answers already paid for come from the cache.

HOW TO READ IT

1. Read down a column to see one mode's whole behaviour; read across a row to see what each mode adds. Green is free, amber is model money, dashed is not done.
2. Fast stops after the free checks. Normal adds the image check with the cheap sheet format. Think reads every page twice, uses three models on scanned ruled tables, uses the more accurate sheet format, allows more audits and compares the two engines' text. Code: `modes.py MODES`.

FIGURE 11 **Page classes and the estimate**



The class chooses the route. Scans go to the scanned route in the image check (two models if the table has ruling lines, none if it has not). Only a visible Unicode text layer may be used as evidence, and only after it has proved itself on the cells both engines agree on. On the sample PDFs the Union budget comes out as 67 born-digital Unicode pages, 19 legacy-font pages, 2 without text and 18 scans.

HOW TO READ IT

1. Left: the render class decides the route in the image check (scan or born digital). Right: the text-layer class decides whether the PDF's own text may be used as evidence. A scan's invisible OCR text is not evidence, and a legacy font's text never is.
2. The dashed bar is the user's override: `--doc-type digital` or `scanned` forces the render class of every page. Code: `page_class.py classify`.

23 The legacy path

`--table-qa legacy` keeps the first version of Stage 2, mainly so the two can be compared on the same data. It is useful to read because it shows what the new design replaced.

23.1 What it does

1. `assemble.py Assembler._build_table L590` (legacy branch): score each engine's table by the share of its numbers found in the PDF text layer and by how few printed or consensus checks fail (`assemble.py Assembler._score L316`); take the best.
2. `tables.py fuse L576`: repair cells of that table from the other engine's where that makes more checks pass; cells where the engines differ and checks cannot decide are *unresolved*.
3. `assemble.py Assembler._resolve_with_layer L321`: for each unresolved number, the engine whose number appears in the PDF text layer wins.
4. If checks still fail: `assemble.py Assembler._reocr L337` re-reads the table as a 300 dpi crop with Bodhan (at most 10 per document, ₹0.20 each) and takes the majority of three readings, then fuses again.
5. `assemble.py Assembler._tiebreak L362`: a vision model (Flash-lite) reads up to 20 disputed cells from a crop of the whole table.
6. `assemble.py Assembler._join_tables L663`: page N against page N+1 with `tables.py continuation L639` (score 5 or more joins, 1 or less separates); an ambiguous pair is judged by a small text model (Qwen 3.5-9B reading the two tables' evidence as JSON; `agent.py Agent.decide_continuation L216`, accepted at confidence 0.6).

23.2 Why it was replaced

- **It trusted agreement and arithmetic.** Two engines that agree can be wrong the same way, and most tables have no totals to check. On the real engine readings it accepted 10 wrong tables as fine; the new Stage 2 accepts none.
- **The re-read was not independent** (weakness #6 in the brief): the same engine reading a crop of the same pixels tends to repeat its errors, and "majority of three" counted one reader twice.
- **Models were underused and weak** (weakness #5): a small text model judged joins without seeing the page, and a vision model saw whole-table crops for a few flagged numbers.
- **The structure was never checked.** A row both engines dropped went unnoticed.
- **Joins looked one page ahead and could not skip a figure page**; with equal column counts it joined unrelated tables (4 of 12 pairs on the benchmark).
- **The report said "checks passed"**, which only meant sums agreed.

Measured on the simulated benchmark with the legacy checker and no model (shared-error share 0.15): only 24% of born-digital and 17% of scanned tables came out exactly right; of the tables accepted without a flag, about 70% were wrong; when two readers disagreed it picked the right value 45 to 64% of the time (about a coin flip); shared errors were fixed 0% of the time; joins across a figure or blank page worked 0% of the time.

24 The benchmark and the model pilot

No hand-checked Indic tables were available, and hand-checking real tables was ruled out, so the benchmark makes its own: tables whose content is known because the generator wrote it, printed to PDF, read by simulated and by real engines, and scored. Nothing in Stage 2 was trusted before it was measured here. Figure 12 (after this chapter) shows the flow. The code is in `bench/` and needs numpy, pillow and apted (`bench/metrics.py` uses tree edit distance).

24.1 The pieces

MODULE	DOES
`bench/vocab.py`	Word banks per language: curated place, department and item names plus words harvested from cached OCR output (so words have real shapes). The vocabulary of each benchmark version is **frozen** in `bench/vocab_snapshot.json`: harvesting from a growing `out/docs` once made a case differ from the one that had been printed
`bench/gen.py`	Cases: six languages (Hindi, Marathi, Tamil, Telugu, Bengali, English) × kinds (text, numeric, mixed, dates, merged, sparse, and for v2 wide budget-style tables of 10 to 13 columns) × styles (single page, split, split with repeated header, split with a row cut by the break, split by a figure page, split by a blank page, two different tables). Profiles: **clean** (v1: 10 pt, roomy cells, mild scans) and **tough** (v2: 7.5 pt, tight cells, harsh 150 dpi scans, Latin digits for Tamil and Telugu unless native numerals are asked for). v1 has 72 cases, v2 82
`bench/render.py`	Headless Chrome prints each case (it shapes Indic scripts correctly, keeps merged cells and has a realistic text layer with doubled matras); a scan-like copy adds skew, blur, noise and JPEG. The table's box is read back from the drawn rules. A case fails if a table overflows the page (an earlier `overflow:hidden` silently clipped one)
`bench/engines.py`	Simulated engines: two readers each get an independent copy of the truth with errors planted; a share (`shared_frac`) is planted identically in both, which agreement cannot reveal; character confusions come from real Sarvam-against-Bodhan differences; the two readers write merged headers in the engines' two dialects
`bench/harness.py`	Runs one case through the *real* Stage 1 and Stage 2 with the simulated engines substituted (`SimSarvam` reads the manifest's own job plan, `SimBodhan` matches images by hash); nothing calls an API. The checker is chosen: `baseline` (legacy, rules only) or `standard`
`bench/metrics.py`	Cell accuracy (rows matched by similarity-weighted alignment, so one wrong digit does not break the match), TEDS for small tables, join decisions against the page structure, and **accepted-but-wrong**: tables the system called fine that were not
`bench/run_bench.py`, `bench/report.py`, `bench/compare.py`	Build the set, run a checker, save JSON, summarise, compare two checkers table by table
`bench/real.py`, `bench/calibrate.py`	**Paid.** Run the real Sarvam and Bodhan on benchmark pages (shows the cost first), then compare real readings with the truth and with the simulation's assumptions
`bench/pilot.py`, `bench/pilot_report.py`	The model pilot: the standard checker on the real readings produces the real questions; each model answers them; the report applies the accept rule and counts right, wrong and harm
`bench/geometry_eval.py`, `bench/text_dispute_report.py`	Free measurements of the ruling-line reader (322 of 322) and of the page-text check

24.2 What the benchmark found

1. **The simulation's baseline** (legacy checker): see section 23.2.
2. **Real engines on clean synthetic pages (v1; 24 pages, ₹16.80)**: both engines are near-perfect for Hindi, Marathi, Bengali and English (about 0 errors in 1,100 cells; no missing rows, no column errors). *All* remaining errors were systematic and shared: Tamil numerals read as Tamil letters (௨ → ஒ) and Telugu numerals read as Latin digits, identically by both engines. The old Stage 2 accepted those tables. Modern documents rarely use these numerals, so Latin digits became the default for Tamil and Telugu in v2 and "rare numerals" a separate test class.
3. **The synthetic pages are much easier than real documents.** On the real documents the two engines disagree on about 13 to 16% of Marathi cells and about 29% of the Union budget's Hindi text cells (numbers: 1.1%), and 6 of 28 real tables differ in

shape, against about 0% and 0 of 24 on the synthetic pages. That is why v2 exists.

4. **Real engines on harder pages (v2; 22 pages, ₹15.40)**: cell error 2.3% on born-digital pages against 26.7% on scans (150 dpi, 7.5 pt Bengali digits are unreadable to every reader); 9% of errors are shared and 22% are both-wrong-differently.

Standard against legacy, on the real engine readings (known truth)

	LEGACY	STANDARD, FREE CHECK	STANDARD, WITH IMAGE CHECK
Wrong tables accepted as fine (v1 + v2)	10	0	**0**
v2 born-digital cells right, of 1,457	1,440	1,446	**1,455**
v2 scanned tables sent to review (really wrong)	0 of 9	9 (8)	9 (8)
Wrong joins, benchmark	4/12 · 2/12		**0/12 · 0/12**

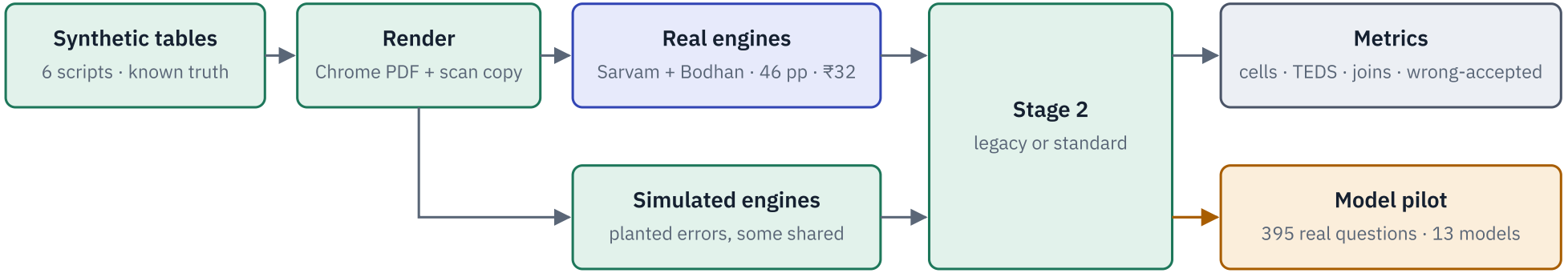
Deciders measured on these readings: cosmetic 27 of 27 right, text layer 4 of 4, printed formula 6 of 7 (then tightened). One change was tried and reverted: relaxing how a proven layer's disagreement is flagged (it let wrong tables through). A model on scans without ruling lines was measured and rejected (harm 7 to 17).

24.3 Reproducing

```
python -m bench.run_bench --set v2 --checker standard --cell-error 0.03 0.12 0.25 --out
bench/results/v2_standard.json
python -m bench.report  bench/results/v2_standard.json            # accuracy, statuses, joins
python -m bench.compare bench/results/v2_baseline.json bench/results/v2_standard.json
python -m bench.real --set v2 --dry-run                           # PAID run: shows plan and cost first
python -m bench.calibrate --set v2 [--image-check]                # real readings against truth (image check is
paid, small)
python -m bench.pilot --dry-run                                   # builds sheets, counts questions, estimates the
cost
python -m bench.geometry_eval                                     # free, about 4 minutes
python -m bench.text_dispute_report out/docs/<run> ...          # after assembling with --mode think
```

Results are kept in `bench/results/` (JSON, one file per run; the pilot's per-model answers are in `bench/results/pilot/`). Real engine outputs of the benchmark pages are in `bench/real/` and `out/docs/`.

FIGURE 12 **How Stage 2 was measured before it was trusted**



Two benchmark sets. v1 uses clean pages. v2 uses dense 7.5 pt pages, budget-style tables of 10–13 columns and harsh 150-dpi scans; the real engines found v2 realistically hard (cell errors 2.3% born-digital, 26.7% scanned). The vocabulary each set was rendered with is frozen, so a case regenerates exactly as it was printed.

HOW TO READ IT

1. Left to right: synthetic tables with known truth are printed to PDF and scan-like copies, then read either by the real engines (paid, 46 pages, about ₹32) or by simulated engines with planted errors (free). Either set of readings goes through Stage 2 (old or new) and is scored.
2. The questions Stage 2 queues on the real readings are the 395 questions of the model pilot (amber). Code: the `bench/` package.

25 Tests

141 unit tests, all without API calls, run in about 17 seconds: `python -m unittest discover -s tests -t .`. The fakes in `tests/fakes.py` (a fake Sarvam module that returns ZIPs, a fake Bodhan backend, a fake model) and the helpers in `tests/helpers.py` (synthetic PDFs, a temporary cache) are what make that possible.

FILE	TESTS	COVERS
`tests/test_rate_limit.py`	10	The Bodhan limiter never exceeds 8 starts in 62 s; a window without spacing allows a burst and then waits; a pause delays the next start and blocks every thread; the in-flight cap with threads; the verdict for each status code; a 429 pauses the lane for Retry-After; 5xx retries then succeeds; fatal stops at once; retries exhausted
`tests/test_clients.py`	9	Bodhan: a null body is resent as JPEG, a token overflow falls back to markdown tables, the default table format is html, an auth error is fatal. Sarvam: submit never retries an uncertain error, retries a 429 with Retry-After, poll retries a 5xx, the first poll delay
`tests/test_blocks_manifest.py`	9	Every documented Bodhan label and every seen Sarvam tag is mapped; box normalisation and furniture merging; Sarvam's `bbox_norm` and reading order; threads never claim the same page; priority, then stealing from the tail; Sarvam claims one language in page order; retry, fail and requeue; the job lifecycle
`tests/test_engine.py`	8	A whole run with fake engines and a free replay from the cache; a missing page retried in a new job; a page missing twice falls back to Bodhan; a hard stop then resume polls the same job; a crash while submitting resubmits and logs it; the budget cap stops cleanly; Bodhan steals from Sarvam's backlog without double work; a fatal Bodhan error stops only that lane
`tests/test_languages.py`	9	Same-script languages (Hindi against Marathi and others), single-language scripts and letter cues, markup is not English, a mixed page block by block, the OCR modes, invalid codes never reach the API, `None` omits the field, an old run database gets the new columns (migration), an English-mode page that comes back Hindi is re-read once
`tests/test_pdf_pages.py`	6	A word box lands on ink for every rotation; upright rotation makes text run left to right; Marathi sideways pages are detected; the dpi and size cap; chunk page order and boxes; rotations make chunk pages upright
`tests/test_page_class.py`	6	Vector, scan, scan with hidden OCR text, text over a picture, legacy font, the user override
`tests/test_tables.py`	15	Digits and numbers; rowspan expansion; Markdown tables; formulas pass on a correct table and a misread total is flagged at the right cell; consensus totals; the product rule skips serial and copied columns; merged cells count once and total words are whole words; fusion; number formatting; the legacy join (decision, footnotes under a table, one column of difference, different tables) and a split table joined in doc.md
`tests/test_table_qa.py`	26	Alignment, phantom columns, each decider, the layer proof, risks, status, ruling-line geometry (grid, skew, three corrections), the judge rule with a fake model, a standard assembly
`tests/test_table_join.py`	8	Continuation without a header joins; a repeated header is found and dropped; a new table with its own title stays separate; unmappable column counts stay separate; columns map by position when ruled; a text row cut by the page break is merged; a seam row printed twice is dropped; the seam check flags a column that changes kind
`tests/test_layer_repair.py`	4	The three rule kinds are learned and applied; a glyph used two ways is not learned; too few examples give no rule; a reph stored after its cluster
`tests/test_modes.py`	6	Mode defaults, Think's format and vote, Fast makes no model calls, the fingerprint, the estimate
`tests/test_text_qa.py`	10	The page-text check (section 20.1)
`tests/test_bench.py`	15	The generator (every style, kind and language builds a consistent case; the same seed gives the same case; split parts add up to the truth; a cut row rejoins; two tables stay different), error planting (shared errors identical in both readers; independent ones not; corruptions always differ; the flat dialect keeps the merged-header grid) and the metrics (a perfect table scores 1; one wrong digit costs one cell; dropped and extra rows; merged columns; Unicode form is not an error; join scoring)

What the tests cannot cover is what only the real services do: the actual rate limits, the model answers, the Chrome renders. Those were covered by the paid benchmark runs and live tests (for example the `kill -9` resume test).

26 Results, decisions and corrections

This chapter collects what was measured on the real sample documents, the decisions the user made and why, what was rejected, and the mistakes that were made and caught, so the design can be judged on its history and not only on its final state.

26.1 The sample documents in all three modes

The five sample documents were assembled again in each mode from the same OCR output (22 October). "Open" means cells that nothing could settle; they stay *needs review* with the primary engine's reading. Audit flags are risky agreed cells that were read blind.

DOCUMENT WINDOW	PAGES	TABLES	FAST: OPEN	NORMAL: OPEN	THINK: OPEN	STATUS IN NORMAL
Union Receipt Budget 2026-27 (Hindi), pp. 90–101	12	12 → 1 joined	193	3	10	needs review (45 audit flags)
Marathi fiscal report	8	8	185	21	16	2 verified, 2 repaired, 4 review
Economic Survey 2025-26 ch. 4, pp. 11–18	8	3 → 1 joined	4	0	0	needs review (2 audit flags)
NCERT Class 12 maths ch. 6	13	4	5	0	0	1 verified, 3 repaired
Yojana (Telugu), March 2025, pp. 50–57	8	1	0	0	0	needs review (7 audit flags)

- **Union budget, page 92.** The project name is now Bodhan's correct घोमन-टांडा की; the old Stage 2 kept Sarvam's wrong घोमने-टांडी को although Bodhan's matches the clean Unicode text layer. All image-settled number decisions agree with the PDF's clean text layer (0 of 23 contradict it).
- **Time.** Union assembly takes 44 to 57 s (it took 8 to 9 minutes while the model client allowed two calls in flight).
- **Cost.** Normal's image check cost about ₹0.26 per dense table page on the Union budget (₹3.55 for 12 pages in format A, about ₹2.9 in format C, uncapped), ₹0.94 on the Marathi report, ₹0.09 on NCERT and ₹0.05 on the Economic Survey. The target of ₹0.10 per table page set at the start was not met on dense Hindi pages.
- **Page text (Think)**: 57 differences left open in 11,669 words (chapter 20).

26.2 What the user decided

DATE	DECISION	CONSEQUENCE
21 Oct	Up to US$10 of OpenRouter credit; the default checking level is *standard* only; "cannot afford to judge every table"; benchmark first	Models only on true disputes, risky tables and a small audit sample
21 Oct	A printed formula and a *proven* text layer may choose between two engine readings; they never overrule both. Hand-checking real Indic tables is not possible	Gold data is synthetic tables plus English born-digital tables; real Indic results are reported as agreement rates, not accuracy
21 Oct	The remaining Sarvam and Bodhan money may be used for real-engine runs on synthetic pages (₹16.80), and another ₹16 for the tough set (₹15.40)	Benchmark v1 and v2 real readings (46 pages, ₹32.20 in all)
21 Oct	Heavy open models rejected as too slow and costly for 100 to 300 pages; the pilot of small open models was cancelled	Closed models through OpenRouter for now; the open-model path is for the RTX 5070
21 Oct	The image-check rule approved: Luna on born-digital; Flash-lite and Luna must agree on scanned ruled; no model on scanned unruled	`src/table_judge.py`
21 Oct	The Tamil numeral rule was written, then removed: "let me judge it for later"	Only the look-alike risk flag stays; Tamil and Telugu numerals are never given to models

DATE	DECISION	CONSEQUENCE
22 Oct	Format C for Normal, format A for Think; a cheaper born-digital model and a 300-question test: later; word-position cropping: rejected	`modes.py MODES L57`
22 Oct	No spending caps inside the modes ("counter-productive")	Only `--max-cost-inr` remains
22 Oct	Think's third judge: Gemini 3.5 Flash-Lite instead of Gemini 3.8 Flash	`modes.py THIRD_JUDGE L32`

26.3 What was tried and rejected

- **A model that corrects the engines.** It changed more correct cells than it fixed (section 18.1).
- **Letter-only answers** (choose A or B without reading the cell): anchoring raised harm (section 17.2).
- **Heavy open-weight models** (Qwen 3.8-27B and larger, DeepSeek): too slow and costly; the 27B finished with 27 correct cells made wrong.
- **A second reading at 300 dpi** by Bodhan as a third reading of open scanned cells: not run, because the benchmark scans are 150 dpi (a higher render adds nothing) and there are no real scanned tables with known truth to measure it on. It would have cost about ₹5.
- **Cropping single cells without a grid** from the PDF's word positions: too fragile across layouts.
- **Arithmetic as the quality test** (the legacy design): chapter 13.
- **A looser proof for the text layer**: let wrong tables through (chapter 14.7).

26.4 Mistakes and corrections

WHAT HAPPENED	EFFECT	WHAT WAS DONE
Think-mode runs without the original -1 code (22 Oct)	Sarvam's cache key contains the language code, so NCERT's 13 pages were paid again: **₹6.50**	Stopped before it repeated on the other documents; a pre-run cache check now precedes paid re-runs. A cache-key fix is proposed (section 3.4)
"93 of 94 ruled tables exact" was quoted for the geometry	It was a subset; on the full benchmark the figure was 309 of 322	Three defects found and fixed: now 322 of 322 (chapter 15)
"14 of 17 real tables match both engines"	There are 16 tables with grids, not 17: 14 confirmed, 1 decided by the image, 1 conflict	Corrected in chapter 15.4
The report writer read a field that seam checks do not have	It would have crashed on the first real seam check	Fixed (section 21.3)
Benchmark: vocabulary drift, a clipped table, a hash that was not reproducible, blank pages shifting Sarvam chunk positions	Cases that did not regenerate; wrong chunk mapping in the simulator	Frozen vocabulary snapshots; render fails on overflow; a stable hash (crc32); the simulator reads the manifest's job plan
Model guards found by measurement	Tamil numerals turned into letters by every model; a model "deleted" a Marathi cell's text; an audit "confirmed" a shared error	The guards of section 18.2
A partly repaired text layer held back 56 Union cells	Cells wrongly marked `layer_has_neither`	Only a fully clean layer may raise it

26.5 What this phase cost

ITEM	SPEND
Real Sarvam and Bodhan runs of the benchmark (v1 ₹16.80, v2 ₹15.40)	₹32.20
Think-mode runs on the samples: Bodhan on 36 new pages ₹7.20, plus the Sarvam mistake ₹6.50	₹13.70
OpenRouter (all pilots, image checks and the Think runs), from the ledger	US$1.54 of US$10

27 Limits and where to go next

What the system cannot do today, what was deliberately left for later, and what the move to the RTX 5070 changes.

27.1 Known limits

- **Scanned tables without ruling lines.** No reader was reliable (about 40% right with harm), so no model decides there. The free evidence does what it can; the rest is listed for review.
- **Tamil and Telugu numerals.** Both engines and every model read them as letters. They are flagged and never given to a model. A mapping rule was written and then withheld at the user's request; it is the open question.
- **Legacy-font PDFs** (Marathi report, NCERT): the text layer is unusable, so the free evidence is only agreement, formulas and the image.
- **Page text.** 0.5% of words stay open in Think and no model reads text; in Normal and Fast prose is one engine's reading.
- **Geometry** is untested beyond 2.5° skew, on sideways scans and on dashed or double lines.
- **The estimate** ignores the cache and its table-check range is wide.
- **The Sarvam cache key** depends on the language code (section 3.4).
- **One duplicate paid job** is possible after a crash during submission (section 10.4).
- **Model answers vary between runs** by a few cells; the differences between modes on a single document are of that size.

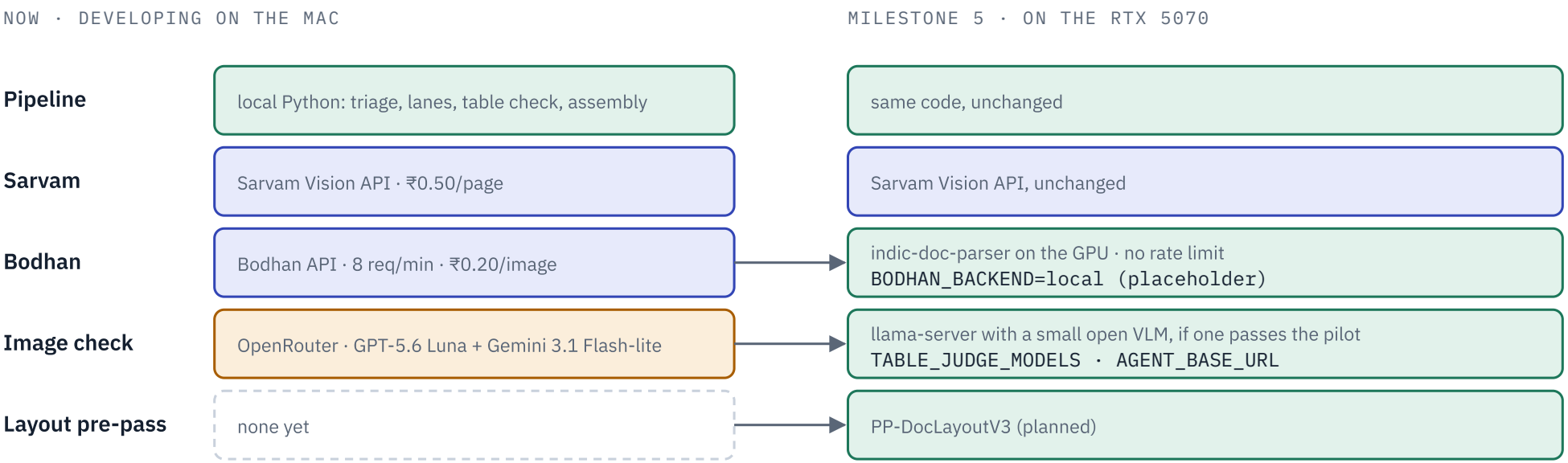
27.2 Left for later, by decision

- A cheaper born-digital judge (Claude Haiku 5.5, Qwen 8B-class vision models), tested on a larger set of about 300 born-digital questions with planted disputes and known truth. The rule for adopting one: harm stays 0, correct answers within about 2% of Luna's, cheaper or faster.
- The Tamil numeral rule.
- An image check for page text, once there is a text set with known answers (the Mozhi word crops).
- Updating the Sarvam cache key and the estimate to account for cached pages.

27.3 The RTX 5070

- **Bodhan locally** (`BODHAN_BACKEND=local`, `lanes.py BodhanLocalBackend L279`, still a placeholder) removes the 8-a-minute limit and the per-page price; Think's extra readings become nearly free.
- **The image-check models** are read from `.env` (`TABLE_JUDGE_SINGLE`, `TABLE_JUDGE_MODELS`, `TABLE_JUDGE_THIRD`) and the server from `AGENT_BASE_URL`, so a small open vision model on a local `llama-server` can replace them *if it passes the same pilot with no harm*.
- **A layout pre-pass** with a local layout model is planned.
- **A local backend** (FastAPI) serving a static page is planned after the move. The same static page can be published on github.io as the project page, but github.io cannot run Python or hold API keys, so it can show results and a demo, not run the pipeline.

FIGURE 13 **Now on the Mac, later on the RTX 5070**



Each swap is a setting, not a rewrite. The models for the image check are read from `.env` (`TABLE_JUDGE_SINGLE`, `TABLE_JUDGE_MODELS`), so a small open model can replace them if it passes the same pilot with no harm. The local Bodhan backend is still a placeholder.

HOW TO READ IT

1. Each row is one component and the one setting that swaps it: Bodhan from the API to a local model, the image-check models from OpenRouter to a local server, an optional layout pre-pass. Sarvam stays in the cloud.
2. None of the swaps changes the pipeline's code; they change `.env`. A local model replaces a cloud judge only if it passes the same pilot with no harm.

Appendix A: Code map

Every module, with the first lines of its own documentation and its classes (blue), functions and constants (amber) with the line where each starts. Methods are written Class.method; private ones (leading underscore) are included because the chapters refer to them. Generated from the source on the day of this report.

src/ : the program

src/agent.py 330 lines

The agent: an LLM consulted only at decision points, with typed JSON answers.

L44 LEDGER L45 PRICES_CACHE L47 CONTINUATION_SCHEMA L59 CELLS_SCHEMA L81 _openrouter_prices L98 Agent
L99 Agent.__init__ L122 Agent.available L125 Agent.client L134 Agent.spent_usd L145 Agent._cost_usd L153 Agent._record
L168 Agent._chat L216 Agent.decide_continuation L231 Agent.read_cells L247 Agent.judge_cells L269 JUDGE_SCHEMA
L291 JUDGE_INTRO L300 JUDGE_STYLE L310 _parse_json L328 same_number

src/assemble.py 1224 lines

Assemble one document from the per-page engine outputs of a run.

L66 ENGINE_ORDER L67 TERMINAL_PUNCT L75 load_blocks L80 crop_png L95 layer_numbers L104 format_like L117 cleanup_text
L126 table_in_text L152 TableRec L183 TableRec.meta L200 Assembler L201 Assembler.__init__ L226 Assembler.say
L230 Assembler._can_spend L237 Assembler._load L264 Assembler._tag_languages L295 Assembler._versions
L316 Assembler._score L321 Assembler._resolve_with_layer L337 Assembler._reocr L362 Assembler._tiebreak
L417 Assembler._scanned L420 Assembler._layer_text L430 Assembler._build_table_qa L461 Assembler._sync
L476 Assembler._check_text L494 Assembler._repair_layers L516 Assembler._image_check L590 Assembler._build_table
L660 Assembler._body L663 Assembler._join_tables L735 Assembler._first_table L750 Assembler._join_tables_standard
L821 Assembler._absorb L838 Assembler._evidence L854 Assembler._figure L863 Assembler._compose L937 Assembler._continues
L945 Assembler._join_prose L952 Assembler._write_tables L969 Assembler._page_number_gaps L980 Assembler.run
L1015 Assembler._language_summary L1047 Assembler._result L1113 render_report L1194 main

src/blocks.py 210 lines

One block format for every engine.

L14 KINDS L17 BODHAN_KIND L57 SARVAM_KIND L85 Block L97 Block.to_dict L103 _clip L107 _norm L112 kind_of
L118 from_bodhan L140 from_sarvam L169 from_text_page L174 iou L182 merge_furniture L194 page_markdown

src/bodhan_ocr.py 452 lines

Bodhan IndicOCR: one page image per HTTP call.

L50 BODHAN_URL L51 MAX_TOKENS L52 WINDOW_S L53 TIMEOUT L56 BodhanAuthError L60 _LIMITER L61 _LIMITER_LOCK L62 _TLS
L65 bodhan_limiter L79 _session L87 _error_detail L100 classify_bodhan L126 to_jpeg L136 bodhan_request
L217 PageOcrResult L231 BodhanRun L240 _cache_key L245 _write_page L258 _result_from_payload L278 ocr_one_page
L319 ocr_pages_sequential L344 ocr_pages_parallel L391 _parse_args L423 main

src/cache.py 47 lines

Disk cache. Key is sha256(bytes + backend + model + extra).

L13 make_key L25 cache_path L30 get L37 put L46 hit

src/compare.py 304 lines

Build out/report.md from Week 1 backend results. No CER. No plots.

L17 DEVANAGARI L18 TAMIL L19 LATIN L22 first_n_lines L27 _read_text L33 speedup L39 token_sum L46 script_note
L59 has_table_markup L63 inspect_bodhan_blocks L87 sarvam_page_text L110 write_report

src/config.py 170 lines

Load environment for Week 1. Do not print secret values.

L11 ROOT L14 SAMPLES_DIR L15 CACHE_DIR L16 OUT_DIR L17 PAGES_DIR L18 PAGES_QWEN_DIR L19 SARVAM_OUT_DIR
L20 BODHAN_SEQ_DIR L21 BODHAN_PAR_DIR L22 QWEN_SEQ_DIR L23 QWEN_PAR_DIR L26 LANGUAGE_ALIASES L63 LANGUAGE_MENU
L77 resolve_ocr_language L88 choose_ocr_language L110 OCR_LANGUAGE L111 QWEN_MODEL L112 BODHAN_MODEL
L114 BODHAN_TABLE_FORMAT L115 QWEN_WORKERS L116 BODHAN_WORKERS L117 PDF_LONG_EDGE L120 BODHAN_RATE_PER_MIN
L121 BODHAN_MAX_INFLIGHT L122 BODHAN_BACKEND L124 SARVAM_RATE_PER_MIN L125 SARVAM_MAX_PAGES_PER_PDF L128 RENDER_DPI
L129 RENDER_MAX_LONG_PX L132 AGENT_BASE_URL L133 AGENT_MODEL L134 VERIFIER_VLM_MODEL L135 OPENROUTER_BUDGET_USD
L138 AGENT_MAX_INFLIGHT L139 AGENT_STARTS_PER_MIN L142 TABLE_JUDGE_SINGLE L143 TABLE_JUDGE_MODELS L147 SARVAM_INR_PER_PAGE
L148 BODHAN_INR_PER_PAGE L149 INR_PER_USD L151 QWEN_USD_PER_M_IN L152 QWEN_USD_PER_M_OUT L153 OPENROUTER_BASE_URL
L154 DOCS_OUT_DIR L156 SARVAM_API_KEY L157 BODHAN_API_KEY L158 OPENROUTER_API_KEY L161 has_sarvam_key
L165 has_bodhan_key L169 has_openrouter_key

src/estimate.py 216 lines

Cost and time of a run, for all three modes, before anything is sent. No API calls.

L39 COEF L40 SAMPLE L41 NUMBER_RE L45 Preflight L54 Preflight.table_range L58 _legacy L67 _scan_has_grid
L75 _interval L87 preflight L128 mode_estimate L179 estimate_all L183 _money L189 _minutes L193 render L210 record

src/lanes.py 756 lines

Engine lanes for one document run.

L49 FATAL_LANE_REASONS L50 POLICIES L51 JOB_TIMEOUT_S L55 Settings L71 Settings.fingerprint L85 Budget
L86 Budget.__init__ L92 Budget.reserve L99 Budget.settle L105 write_json L112 Context L115 Context.__init__
L140 Context.doc L148 Context.path L151 Context.accepting L154 Context.language_prior L158 Context.save_page_outputs
L180 Router L181 Router.__init__ L185 Router.engines L194 Router.initial_tasks L213 Router.after_sarvam_page
L219 Router.on_page_failed L226 Router.on_lane_dead L243 PageBackend L250 PageBackend.cache_extra L251 PageBackend.ocr
L252 PageBackend.to_blocks L253 PageBackend.cost_inr L256 BodhanAPIBackend L263 BodhanAPIBackend.__init__
L266 BodhanAPIBackend.cache_extra L269 BodhanAPIBackend.ocr L272 BodhanAPIBackend.to_blocks L275 BodhanAPIBackend.cost_inr
L279 BodhanLocalBackend L284 BodhanLocalBackend.__init__ L288 QWEN_PROMPT L298 QwenBackend L307 QwenBackend.__init__
L313 QwenBackend.cache_extra L316 QwenBackend.ocr L351 QwenBackend.to_blocks L354 QwenBackend.cost_inr L365 PageLane
L366 PageLane.__init__ L390 PageLane.busy L394 PageLane.start L400 PageLane.join L404 PageLane._on_outcome
L409 PageLane._worker L437 PageLane._process L509 job_file_name L513 map_job_pages L532 OpenJob L543 SarvamLane
L544 SarvamLane.__init__ L553 SarvamLane.start L556 SarvamLane.join L560 SarvamLane.per_job L564 SarvamLane._on_outcome
L569 SarvamLane.run L596 SarvamLane._restore L617 SarvamLane._page_key L622 SarvamLane._ingest
L652 SarvamLane._submit_next L712 SarvamLane._poll L731 SarvamLane._complete

src/langid.py 240 lines

Language identification for OCR output, per block and per page. Free, offline.

L28 SARVAM_LANGUAGES L34 SCRIPT_RANGES L49 SINGLE L54 SCRIPT_OF L60 SCRIPT_DEFAULT L63 DEVANAGARI_MARKERS
L75 DEVANAGARI_SUFFIXES L76 ASSAMESE_LETTERS L77 SINDHI_LETTERS L78 KASHMIRI_LETTERS L79 _MARKUP L80 _WORD
L83 script_of_char L93 strip_markup L98 _letters L107 devanagari_scores L121 decide_devanagari L134 language_of_script
L154 TextLanguages L162 TextLanguages.dominant L165 TextLanguages.present L169 TextLanguages.to_dict L174 detect
L188 indic_share L194 SKIP_KINDS L197 page_languages

src/layer_repair.py 146 lines

Repairing a PDF text layer whose font maps some glyphs to odd code points. No API calls.

L35 SUSPECT_ASCII L38 candidate L42 _cluster_pattern L58 CLUSTER L59 SIGNS L63 _best_alignment L83 learn
L94 _learn_round L135 apply

src/manifest.py 438 lines

Run state for one document: SQLite (WAL) plus an append-only events log.

L27 SCHEMA L51 ACTIVE L55 MIGRATIONS L67 EventLog L70 EventLog.__init__ L75 EventLog.log L80 EventLog.read
L86 Manifest L87 Manifest.__init__ L102 Manifest.close L107 Manifest._tx L117 Manifest._rows L123 Manifest.get_meta
L127 Manifest.set_meta L133 Manifest.triaged_pages L136 Manifest.add_page L151 Manifest.page_info
L161 Manifest.update_detected L169 Manifest.set_page_mode L176 Manifest.all_pages L187 Manifest.task
L191 Manifest.ensure_task L200 Manifest.count L209 Manifest.claim_sarvam L234 Manifest.claim_page_task
L278 Manifest.finish L291 Manifest.retry_or_fail L305 Manifest.release L313 Manifest.reopen_stolen L322 Manifest.fail_lane
L339 Manifest.requeue_inflight L347 Manifest.queued L352 Manifest.pages_done_by_any L357 Manifest.counts
L363 Manifest.add_extra L371 Manifest.extras_by_kind L375 Manifest.spent_inr L384 Manifest.cache_hits
L389 Manifest.tasks_for_page L394 Manifest.new_job L409 Manifest.job_submitted L416 Manifest.job_finished
L423 Manifest.job L428 Manifest.open_jobs L437 Manifest.jobs

src/modes.py 73 lines

Fast / Normal / Think: one switch for how much each document run reads and checks.

L32 THIRD_JUDGE L36 JudgeProfile L47 Mode L57 MODES L66 DEFAULT_MODE L69 get

src/page_class.py 83 lines

What kind of page this is: born digital or a scan, and whether its text layer can be evidence. No API calls.

L28 DOC_TYPES L29 RASTER_COVER L30 MIN_TEXT L34 PageClass L41 PageClass.scanned L45 PageClass.layer_usable
L49 image_cover L55 text_counts L67 classify

src/pdf_pages.py 224 lines

PDF pages: render images, find upright rotation, build Sarvam chunk PDFs.

L20 ROTATIONS L24 PagePng L33 RenderSpec L42 Rendered L52 Rendered.page_to_px L55 Rendered.px_to_page
L59 page_transform L71 render_page L99 upright_rotation L125 pdf_cropbox L131 build_chunk_pdf L158 _sha256_file
L164 shred_pdf L214 main

src/qwen_ocr.py 343 lines

OpenRouter Qwen OCR: one PNG per HTTP call. English pages only.

L27 OPENROUTER_BASE L28 MAX_TOKENS L29 TEMPERATURE L30 QWEN_RATE_PER_MIN L31 SLOW_SECONDS L32 QWEN_LANGUAGE L34 PROMPT
L43 THINK_MARKERS L46 QwenModelMissing L51 PageOcrResult L64 QwenRun L74 _client L78 _has_think_block L83 _cache_key
L88 _write_page L95 _result_from_payload L107 _usage_dict L118 _message_text L126 _print_404 L134 _create
L178 ocr_one_page L222 ocr_pages_sequential L247 ocr_pages_parallel L294 _parse_args L315 main

src/rate_limit.py 414 lines

Rate limiting and retries.

L26 T L28 MAX_RETRIES L31 CREDIT_RE L34 Limiter L43 Limiter.__init__ L68 Limiter._wait_needed
L78 Limiter.acquire_start L92 Limiter.slot L100 Limiter.pause_until L104 Limiter.pause_for L108 Verdict L117 Outcome
L123 CallFailed L124 CallFailed.__init__ L130 FatalError L134 DegradeError L138 RetriesExhausted L142 _headers_of
L150 _status_code L162 _retry_after_seconds L175 _body_text L186 classify_status L200 _is_transport_error
L212 classify_http L228 call_with_retry L292 TokenBucket L293 TokenBucket.__init__ L302 TokenBucket.acquire
L320 RateLimiter L321 RateLimiter.__init__ L334 RateLimiter.slot L343 is_rate_limited L347 call_with_429_retry
L373 _LIMITERS L374 _LIMITERS_LOCK L377 get_limiter L409 reset_limiter

src/run.py 476 lines

Infinite-OCR job engine: OCR a PDF of any length, resumably.

L62 RunError L66 file_sha256 L74 slug L79 _fmt_states L84 Run L85 Run.__init__ L111 Run.say L117 Run.prepare
L177 Run._triage L196 Run.execute L261 Run._progress L276 Run._summary L322 print_summary L336 parse_pages
L351 parse_args L385 main

src/run_week1.py 232 lines

Week 1 run: shred, Sarvam, Bodhan seq/par, Qwen seq/par, then out/report.md.

L29 _parse_args L86 _sarvam_from_disk L118 main

src/sarvam_digitise.py 344 lines

Sarvam Vision through the Document AI API (`doc_ai.digitise`).

L47 TERMINAL L48 OK_STATUS L49 OUTPUT_FORMAT L50 NO_SDK_RETRY L51 WINDOW_S L52 MAX_WAIT_S L54 _LIMITER
L55 _LIMITER_LOCK L56 _CLIENT L60 DigitiseResult L74 sarvam_limiter L88 _client L95 _as_dict L106 classify_submit
L117 submit L150 poll L162 first_poll_delay L167 wait_for L187 fetch L207 unzip L217 _print_tree L225 _metadata_keys
L254 _page_count L259 digitise_pdf L315 main

src/table_cutouts.py 172 lines

Cut-out sheets for the image check: small numbered strips of a table, one per question.

L22 DPI L23 MAX_WIDTH L24 PER_SHEET L25 LABEL_FONT L29 Strip L36 Sheet L43 _text_lines L49 Cutter
L52 Cutter.__init__ L75 Cutter.strip L96 Cutter.cell_snippet L116 Cutter.header L121 _grid_sheet L147 sheets

src/table_geometry.py 250 lines

Table structure read from the page image: ruling lines, column gaps, ink per cell. No API calls.

L32 DPI L36 Geometry L49 Geometry.confident L52 Geometry.cell_ink L62 Geometry.to_dict L66 crop_gray L78 otsu
L89 _rotate L93 deskew L110 _runs L117 _group L128 _bridge L141 vertical_rules L148 horizontal_rules
L164 partial_rules L186 note_band L197 column_gaps L213 analyse L232 clipped L245 table_geometry

src/table_join.py 247 lines

Joining table segments across page breaks (`standard` mode). No API calls except the optional seam check.

L29 DATE_RE L30 TITLE_RE L31 JOIN_AT L31 SEPARATE_AT L34 kind_of L46 Signature L57 _dominant L66 _serial
L76 signature L91 JoinDecision L99 _sim L103 repeated_header_rows L115 map_columns L129 decide L185 key_columns
L193 merge L229 seam_checks

src/table_judge.py 262 lines

The image check for `standard` table checking: settles open cells from cut-outs of the page.

L39 EXTRA L42 INR_PER_QUESTION L43 MODEL_PRICE L44 DEFAULT_PROFILE L47 question_cost L54 NOT_FOR_MODELS L57 for_models
L63 _extra L67 _short L72 JudgeReport L81 JudgeReport.to_dict L85 route L93 boxed L98 estimate_inr
L104 _matches_engine L108 _where L117 _ask L141 _agreeing L150 settle L193 audit L215 SEAM_SCHEMA L228 seam

src/table_qa.py 504 lines

Table checking, `standard` mode: the free part (no API calls).

L37 ENGINE_ORDER L38 FORMULA_KINDS L39 HINT_KINDS L41 _DASHES L42 _ZERO_WIDTH L43 _LATEX L48 LOOKALIKE_OUTPUTS
L49 NUMERAL_SCRIPTS L52 canon L67 similarity L79 QCell L91 QCell.open L94 QCell.to_dict L99 QTable
L108 QTable.counts L113 QTable.summary L124 align_rows L145 align_cols L164 _grid L174 Evidence L181 _LAYER_COMMON
L184 odd_glyph L191 Layer L199 Layer.of L204 Layer.contains L216 squeeze L233 numbering_confirms L247 choose_structure
L276 check L377 _PUNCT_SPACE L380 minor_difference L388 apply L394 refresh L400 _set L406 _layer_proof L415 _risks
L461 HIGH_RISK L465 RISK_WEIGHT L470 single_reading L477 _questions L495 _status

src/tables.py 719 lines

Tables: parse engine HTML/markdown into grids, check the numbers, join pages.

L30 _ZERO_POINTS L32 DIGIT_MAP L35 NUMBER_RE L38 TOTAL_TOKENS L42 _TOKEN_SPLIT L45 _TotalWords L53 TOTAL_WORDS L54 RANGE_WORDS L57 normalize_digits L61 parse_number L74 number_key L81 decimals L86 clean L96 Cell L106 Table L113 Table.grid L121 Table.rows_text L130 Table.origin L134 Table.to_html L159 _HTMLTables L219 _layout L244 _header_rows L259 _parse_markdown L282 parse_tables L303 Check L312 Check.to_dict L316 _is_numbering_row L325 column_numbers L339 _formula_terms L355 _tolerance L360 _row_label L365 _is_serial L372 _is_code L378 check_table L536 failed L545 numbers_in L550 compare_numbers L560 text_layer_match L570 Fusion L576 fuse L613 CONTINUED_RE L616 _header_text L621 repeated_header_rows L632 Continuation L639 continuation L685 join

src/text_qa.py 234 lines

Page-text check (Think mode, free part): compare the two engines' prose word by word. No API calls.

L45 KINDS L46 MIN_PAGE_OVERLAP L47 MAX_SPAN L48 MIN_TESTED L49 PROOF_SHARE L53 Word L62 Dispute L73 Dispute.to_dict L79 PageText L88 PageText.counts L92 _SUPERSCRIPT L93 _MARKUP_EDGE L96 key_of L106 is_markup L111 _letters_only L117 words_of L129 non_prose_of L134 _is_number L138 _flat L142 _layer_proof L149 compare L204 apply L223 summarise

src/triage.py 197 lines

Free per-page checks before any API call (PyMuPDF only).

L26 SCRIPT_RANGES L40 LANGUAGE_SCRIPT L48 SCRIPT_DEFAULT_LANGUAGE L55 LEGACY_FONT_RE L62 language_script L66 char_script L79 PageFlags L99 PageFlags.has_table L102 PageFlags.to_row L109 body_tables L128 _is_blank L135 triage_page

bench/ : the benchmark and pilots

bench/calibrate.py 219 lines

Measure the real engines on the benchmark pages and compare with what the simulation assumed.

L22 ROOT L23 RESULTS L24 ENGINES L27 engine_table L50 char_pairs L59 page_truth L63 main L142 pct L146 summarise L194 render

bench/compare.py 54 lines

Compare two checkers on the same benchmark results, table by table.

L14 pct L18 headline L36 main

bench/engines.py 370 lines

Simulated engine readings of a table, with every planted error recorded.

L27 FALLBACK_CONFUSIONS L34 SYSTEMATIC_SEEDS L36 LOOKALIKE L40 Profile L54 Profile.label L59 Planted L67 Planted.to_dict L77 harvest_confusions L110 systematic_pool L124 _digit_value L129 _restore_script L136 corrupt_number L159 corrupt_text L188 clone L192 _plain_rows L198 _plain_cols L203 drop_row L213 dup_row L221 merge_cols L233 split_col L246 to_dialect L264 Reading L270 simulate L308 _systematic_errors L322 _structure_errors

bench/gen.py 347 lines

Synthetic benchmark tables with known truth.

L35 KINDS L36 KINDS_V2 L37 STYLES L41 PageSpec L52 Case L64 Case.meta L75 group_indian L87 fmt_number L97 Builder L100 Builder.__init__ L103 Builder.add L106 Builder.table L110 _digits L114 _amount L118 _date L122 build_table L257 _boundaries L266 _copy L270 first_segment L274 later_segment L281 cut_row L298 make_case

bench/geometry_eval.py 67 lines

How often the image geometry reads a table's shape exactly, on benchmark pages with known truth. Free.

L24 ROOT L27 evaluate L52 main

bench/harness.py 232 lines

Run one benchmark case through the real stage 1 and stage 2 with simulated engines.

L35 LANG_CODE L39 PageReading L48 _jitter L52 SimSarvam L56 SimSarvam.__init__ L61 SimSarvam._blocks L79 SimSarvam.fetch L95 SimBodhan L98 SimBodhan.__init__ L108 SimBodhan.ocr L135 stack L146 readings_for L162 run_case L188 accepted_by_system L195 worst_status L201 score_case

bench/metrics.py 195 lines

Scoring a predicted table against the truth, and join decisions against the page structure.

L20 MIN_ROW_SIM L21 TEDS_MAX_NODES L24 _DASHES L27 norm L32 sim L38 align_rows L59 align_cols L77 _Node L84 _Cfg L101 _tree L113 teds L121 score_table L171 expected_joins L180 score_joins

bench/pilot.py 261 lines

Step 4 pilot: which vision model answers the checker's questions best per rupee?

L30 ROOT L31 OUT L32 DEFAULT_MODELS L36 EXTRA L42 Question L57 Group L67 _where L74 collect L118 ask L150 score L171 summarise L198 main

bench/pilot_report.py 109 lines

Compare every model in bench/results/pilot/ on the same real questions.

L19 PILOT L20 OPEN_WEIGHTS L21 FAMILY L23 SEGMENTS L27 family L31 is_open L35 tamil_numeral L39 N L43 accepted L47 load L59 main

bench/real.py 113 lines

Real-engine run on benchmark pages (PAID: Sarvam Rs 0.50 + Bodhan Rs 0.20 per page).

L29 REAL L30 PER_PAGE_INR L33 SINGLES L36 MULTI L38 SINGLES_V2 L40 MULTI_V2 L43 select L55 plan L63 main

bench/render.py 148 lines

Render a benchmark `Case` to a PDF (headless Chrome) and a scan-like copy.

L23 CHROME L24 FONTS L28 CSS L47 CLEAN L48 TOUGH L50 SCAN L55 Rendered L63 page_html L76 to_pdf L90 table_boxes L109 scan_like L132 render_case

bench/report.py 133 lines

Summarise a saved benchmark run.

L20 pct L24 table_rows L30 group L37 accuracy_block L57 status_block L76 cell_states L99 join_block L111 render

bench/run_bench.py 128 lines

Build the benchmark set, run a checker over it, save the results.

L21 ROOT L22 WORK L23 LANGS L26 stable L31 benchmark_set_v2 L47 benchmark_set L62 _work L72 main

bench/text_dispute_report.py 40 lines

What the free page-text check (src/text_qa.py) finds and settles on assembled Think runs. No API calls.

L16 main

bench/vocab.py 130 lines

Word banks for synthetic tables: curated seeds plus words harvested from cached OCR output.

L21 LANGS L24 SCRIPT_OF_LANG L26 SEEDS L41 TOTAL_WORD L42 HEADERS L50 CODE L51 STATUS L57 _ZERO L60 native_digits L67 _script_ok L80 BENCH_RUN L81 SNAPSHOT L84 harvest L108 harvested L124 words L128 phrase

tests/ : unit tests (no API calls)

tests/fakes.py 141 lines

Fake engines for engine tests: no network, no cost.

L19 FakeSarvam L22 FakeSarvam.__init__ L38 FakeSarvam.first_poll_delay L41 FakeSarvam.submit L51 FakeSarvam.poll L58 FakeSarvam.fetch L92 FakeBodhan L99 FakeBodhan.__init__ L105 FakeBodhan.cache_extra L108 FakeBodhan.ocr L126 FakeBodhan.to_blocks L131 FakeBodhan.cost_inr L135 make_pdf

tests/helpers.py 49 lines

Shared test helpers: a fake clock and hand-built HTTP responses.

L10 FakeClock L13 FakeClock.__init__ L17 FakeClock.__call__ L20 FakeClock.sleep L25 response L35 OK_PAYLOAD L42 THROTTLE_BODY

tests/test_bench.py 146 lines

The benchmark's own parts: generator, error planting, metrics. No Chrome, no network.

L13 grid L18 ROWS L21 GeneratorTest L22 GeneratorTest.test_every_style_kind_and_language_builds_a_consistent_case L32 GeneratorTest.test_same_seed_gives_the_same_case L36 GeneratorTest.test_split_parts_add_up_to_the_truth L46 GeneratorTest.test_cut_row_tail_rejoins_the_description L55 GeneratorTest.test_two_tables_are_different_logical_tables L61 SimulationTest L62 SimulationTest.test_shared_errors_are_identical_in_both_readers_and_independent_ones_are_not L75 SimulationTest.test_corrupted_values_always_differ L81 SimulationTest.test_structure_edits_keep_the_table_consistent L92 SimulationTest.test_flat_dialect_keeps_the_grid_shape_of_merged_headers L99 MetricsTest L100 MetricsTest.test_perfect_table_scores_one L106 MetricsTest.test_one_wrong_digit_costs_one_cell_not_the_row_match L114 MetricsTest.test_dropped_and_extra_rows_are_counted L121 MetricsTest.test_merged_column_is_found_by_content_not_scored_as_a_clean_table L128 MetricsTest.test_whitespace_and_unicode_form_do_not_count_as_errors L133 MetricsTest.test_join_scoring

tests/test_blocks_manifest.py 125 lines

L9 DOCUMENTED_BODHAN_LABELS L14 SEEN_SARVAM_TAGS L18 BlocksTest L19 BlocksTest.test_every_documented_bodhan_label_is_mapped L24 BlocksTest.test_every_seen_sarvam_tag_is_mapped L28 BlocksTest.test_bodhan_normalises_and_merges_furniture L42 BlocksTest.test_sarvam_uses_bbox_norm_and_reading_order L53 ManifestTest L54 ManifestTest.setUp L58 ManifestTest.tearDown L62 ManifestTest.add L65 ManifestTest.test_threads_never_claim_the_same_page L86 ManifestTest.test_priority_then_steal_from_tail L98 ManifestTest.test_sarvam_claims_one_language_in_page_order L105 ManifestTest.test_retry_then_fail_and_requeue_inflight L112 ManifestTest.test_job_lifecycle

tests/test_clients.py 119 lines

L13 tiny_png L20 FakeSession L21 FakeSession.__init__ L25 FakeSession.post L30 free_limiter L35 ClassifyBodhanTest
L36 ClassifyBodhanTest.check L43 ClassifyBodhanTest.test_cases L56 BodhanRequestTest L57 BodhanRequestTest.run_with
L63 BodhanRequestTest.test_null_body_is_resent_as_jpeg L71 BodhanRequestTest.test_max_tokens_falls_back_to_markdown
L79 BodhanRequestTest.test_default_table_format_is_html L83 BodhanRequestTest.test_auth_error_is_fatal L88 SarvamClassifyTest
L89 SarvamClassifyTest.test_submit_never_retries_uncertain_errors
L101 SarvamClassifyTest.test_submit_retries_429_with_retry_after L107 SarvamClassifyTest.test_poll_retries_5xx
L113 SarvamClassifyTest.test_first_poll_delay

tests/test_engine.py 150 lines

End-to-end engine runs with fake engines and an isolated cache (no cost).

L16 ROOT L17 MARATHI L18 PATRICK L21 EngineTest L22 EngineTest.setUp L30 EngineTest.make_run L35 EngineTest.states
L38 EngineTest.test_full_run_then_free_replay_from_cache L60 EngineTest.test_missing_page_is_retried_in_a_new_job
L68 EngineTest.test_page_missing_twice_falls_back_to_bodhan L75 EngineTest.test_hard_stop_then_resume_polls_the_same_job
L98 EngineTest.test_crash_while_submitting_resubmits_and_logs L116 EngineTest.test_budget_cap_stops_cleanly
L123 EngineTest.test_bodhan_steals_from_sarvam_backlog_without_double_work
L136 EngineTest.test_fatal_bodhan_error_stops_only_that_lane

tests/test_languages.py 126 lines

Language handling: identification, OCR mode at triage, the English-mode safety net.

L19 ROOT L20 SAMPLES L22 HINDI L23 MARATHI L24 NEPALI L27 DetectTest L28 DetectTest.test_same_script_languages
L33 DetectTest.test_scripts_with_one_language_and_letter_cues L40 DetectTest.test_markup_is_not_english
L44 DetectTest.test_mixed_page_block_by_block L56 TriageModeTest L57 TriageModeTest.mode L61 TriageModeTest.test_modes
L71 SubmitValidationTest L72 SubmitValidationTest.test_invalid_codes_never_reach_the_api
L79 SubmitValidationTest.test_none_omits_the_field L86 ManifestMigrationTest
L87 ManifestMigrationTest.test_old_run_database_gets_language_columns L104 ModeRetryTest
L105 ModeRetryTest.test_english_mode_page_that_comes_back_hindi_is_reread_once

tests/test_layer_repair.py 53 lines

Learning a scrambled PDF text layer's glyph substitutions (src/layer_repair.py). No API calls.

L10 scramble L19 AGREED L23 LayerRepairTest L24 LayerRepairTest.test_rules_are_learned_from_agreed_cells
L32 LayerRepairTest.test_a_glyph_used_two_ways_is_not_learned L39 LayerRepairTest.test_too_few_examples_give_no_rule
L43 LayerRepairTest.test_reph_stored_after_its_cluster

tests/test_modes.py 73 lines

Fast / Normal / Think (src/modes.py) and the estimate shown before a run (src/estimate.py). No API calls.

L15 _pdf L26 ModesTest L27 ModesTest.test_default_is_normal_and_unknown_modes_fail
L33 ModesTest.test_think_uses_format_a_and_two_of_three_on_scans L38 ModesTest.test_fast_makes_no_model_calls
L45 ModesTest.test_document_type_auto_keeps_older_runs_resumable L50 EstimateTest
L51 EstimateTest.test_estimate_from_page_classes_and_table_pages L63 EstimateTest.test_long_documents_are_sampled_with_a_range

tests/test_page_class.py 92 lines

Page classes (src/page_class.py) on small synthetic PDFs. No API calls.

L12 TEXT L15 _image L21 _page L34 PageClassTest L35 PageClassTest.test_born_digital_text_page
L39 PageClassTest.test_scan_without_text L44 PageClassTest.test_scan_with_hidden_ocr_text_is_a_scan_and_its_text_is_not_evidence
L48 PageClassTest.test_text_over_a_full_page_picture_is_born_digital
L52 PageClassTest.test_legacy_font_page_is_vector_but_its_layer_is_not_used
L56 PageClassTest.test_user_choice_overrides_every_page L65 _grid L77 TableHintTest
L78 TableHintTest.test_a_page_number_box_is_not_a_table_page_but_a_body_table_is

tests/test_pdf_pages.py 101 lines

L15 ROOT L16 MARATHI L17 PATRICK L20 sideways_doc L29 ink L41 RenderTransformTest
L42 RenderTransformTest.test_word_box_lands_on_ink_for_every_rotation
L57 RenderTransformTest.test_upright_rotation_makes_text_run_left_to_right
L67 RenderTransformTest.test_marathi_sideways_pages_detected L72 RenderTransformTest.test_dpi_and_size_cap L80 ChunkTest
L81 ChunkTest.test_pages_order_boxes_and_text L92 ChunkTest.test_rotations_make_chunk_pages_upright

tests/test_rate_limit.py 139 lines

L16 max_in_window L23 LimiterTest L24 LimiterTest.test_bodhan_settings_never_exceed_8_per_62s
L32 LimiterTest.test_window_without_spacing_allows_burst_then_waits L39 LimiterTest.test_pause_delays_next_start
L46 LimiterTest.test_inflight_cap_with_threads L69 LimiterTest.test_pause_blocks_every_thread L90 ClassifyStatusTest
L91 ClassifyStatusTest.test_codes L104 CallWithRetryTest L105 CallWithRetryTest.test_429_pauses_lane_for_retry_after
L114 CallWithRetryTest.test_5xx_retries_then_succeeds L121 CallWithRetryTest.test_fatal_stops_at_once
L133 CallWithRetryTest.test_exhausted

tests/test_table_join.py 84 lines

Joining table segments across pages (src/table_join.py). No API calls.

L11 grid L16 HEAD L20 sig L24 DecideTest L25 DecideTest.test_continuation_without_header_joins
L31 DecideTest.test_repeated_header_is_found_and_dropped L40 DecideTest.test_a_new_table_with_its_own_title_stays_separate
L48 DecideTest.test_unmappable_column_counts_stay_separate L52 DecideTest.test_columns_map_by_position_when_ruled L59 MergeTest
L60 MergeTest.test_a_text_row_cut_by_the_page_break_is_merged L69 MergeTest.test_a_seam_row_printed_twice_is_dropped
L76 MergeTest.test_seam_check_flags_a_column_that_changes_kind

tests/test_table_qa.py 361 lines

The free table checker (src/table_qa.py) and image geometry (src/table_geometry.py). No API calls.

L21 grid L26 ROWS L30 edited L36 CanonTest L37 CanonTest.test_cosmetic_differences_vanish
L44 CanonTest.test_layer_matches_numbers_as_whole_tokens L52 CheckTest L53 CheckTest.test_agreement_is_verified
L58 CheckTest.test_dash_difference_is_cosmetic L63 CheckTest.test_disagreement_without_evidence_becomes_a_question
L70 CheckTest.test_proven_text_layer_decides_a_text_cell L78 CheckTest.test_unproven_text_layer_is_ignored
L85 CheckTest.test_printed_formula_decides_only_when_it_holds L94 CheckTest.test_shared_lookalike_numeral_is_not_verified
L100 CheckTest.test_dropped_row_is_aligned_and_flagged_single
L108 CheckTest.test_phantom_columns_are_squeezed_and_numbering_decides L122 CheckTest.test_image_rules_decide_structure
L131 drawn_table L150 GeometryTest L151 GeometryTest.test_ruled_grid_is_counted_and_headlines_are_not_rules
L156 GeometryTest.test_skewed_scan_is_straightened L161 GeometryTest.test_unruled_text_is_not_mistaken_for_a_grid
L168 GeometryTest.test_column_line_broken_by_merged_cells_still_counts
L176 GeometryTest.test_note_row_inside_the_frame_is_not_a_table_row
L184 GeometryTest.test_a_box_that_cuts_off_the_table_side_is_detected L191 AssembleStandardTest
L192 AssembleStandardTest.test_standard_mode_records_status_and_questions L216 SecondReadingTest
L217 SecondReadingTest.test_a_table_the_other_engine_wrote_inside_a_text_block_is_found_and_keeps_the_primary_box
L236 SecondReadingTest.test_a_table_with_one_reader_is_never_verified L245 JudgeRuleTest L248 JudgeRuleTest._settle
L268 JudgeRuleTest.test_born_digital_accepts_one_model_matching_an_engine
L272 JudgeRuleTest.test_a_reading_no_engine_produced_is_not_accepted
L277 JudgeRuleTest.test_scanned_ruled_needs_two_models_to_agree
L283 JudgeRuleTest.test_printed_text_is_never_deleted_on_a_model_word L288 LeanStage2Test
L291 LeanStage2Test.test_punctuation_only_difference_is_minor_but_numbers_are_not
L303 LeanStage2Test.test_cascade_skips_the_second_model_when_the_first_cannot_decide
L330 LeanStage2Test.test_think_two_of_three_on_scans_asks_the_third_model_only_where_needed

tests/test_tables.py 193 lines

L24 MARATHI_TABLE L36 table L40 ParseTest L41 ParseTest.test_digits_and_numbers L47 ParseTest.test_grid_expands_rowspan
L58 ParseTest.test_markdown_table L63 CheckTest L64 CheckTest.test_formulas_pass_on_a_correct_table
L70 CheckTest.test_misread_total_is_flagged_at_the_right_cell L78 CheckTest.test_consensus_total_column
L88 CheckTest.test_product_rule_skips_serial_and_copied_columns
L104 CheckTest.test_merged_cell_counts_once_and_total_words_are_whole_words
L115 CheckTest.test_fuse_repairs_from_the_other_engine L124 CheckTest.test_format_like_keeps_script_and_decimals L128 SPLIT_A
L130 SPLIT_B L135 JoinTest L136 JoinTest.test_continuation_decision_and_join
L147 JoinTest.test_table_ending_above_footnotes_still_counts_as_page_end
L156 JoinTest.test_one_column_difference_goes_to_the_agent L161 JoinTest.test_different_tables_stay_separate
L167 AssembleEndToEndTest L168 AssembleEndToEndTest.test_table_split_over_two_pages_is_joined_in_doc_md

tests/test_text_qa.py 92 lines

Page-text check (src/text_qa.py): the free part, on synthetic blocks. No API calls.

L10 BASE L14 block L18 TextQaTest L19 TextQaTest.test_identical_text_has_no_disputes
L24 TextQaTest.test_spacing_and_punctuation_differences_are_settled_free
L34 TextQaTest.test_a_misread_word_stays_open_without_a_proven_layer
L38 TextQaTest.test_a_proven_layer_chooses_and_the_choice_is_applied L48 TextQaTest.test_an_unproven_layer_decides_nothing
L54 TextQaTest.test_the_layer_never_deletes_printed_words L61 TextQaTest.test_a_different_reading_order_is_not_compared
L67 TextQaTest.test_latex_and_unicode_superscripts_are_the_same_text
L74 TextQaTest.test_words_one_engine_has_as_prose_and_the_other_as_a_header_or_figure_text_are_settled
L85 TextQaTest.test_furniture_is_ignored

Appendix B: Configuration

Defined in `src/config.py` unless another file is named. Every value can be overridden by an environment variable of the same name (or in `.env`). API keys are read once, never printed and never written to a log.

NAME	DEFAULT	MEANING
`SARVAM_API_KEY · BODHAN_API_KEY · OPENROUTER_API_KEY`	(none)	Keys; a missing key for an engine the policy needs stops the run before it starts
`SARVAM_RATE_PER_MIN`	10	Sarvam starts per window (status polls count)
`BODHAN_RATE_PER_MIN · BODHAN_MAX_INFLIGHT`	8 · 4	Bodhan starts per window; concurrent requests
`BODHAN_MODEL · BODHAN_TABLE_FORMAT · BODHAN_BACKEND`	indic-ocr · html · api	Model name; `html` keeps merged cells, `markdown` flattens them; `local` is the RTX 5070 placeholder
`BODHAN_WORKERS · QWEN_WORKERS`	4 · 3	Week 1 scripts (the pipeline uses `--bodhan-workers`)
`RENDER_DPI · RENDER_MAX_LONG_PX`	200 · 3000	Page images sent to Bodhan and used for crops
`SARVAM_INR_PER_PAGE · BODHAN_INR_PER_PAGE · INR_PER_USD`	0.5 · 0.2 · 85	Accounting in rupees (the exchange rate is approximate)
`QWEN_MODEL · QWEN_USD_PER_M_IN / OUT`	qwen/qwen3.8-flash · 0.15 / 0.47	The optional English-page reader and its price
`AGENT_BASE_URL`	https://openrouter.ai/api/v1	Any OpenAI-compatible server (`http://localhost:8080/v1` for a local llama-server)
`AGENT_MODEL · VERIFIER_VLM_MODEL`	qwen/qwen3.5-9b · google/gemini-3.1-flash-lite	Legacy path only: the join judge and the cell tie-breaker
`OPENROUTER_BUDGET_USD`	10	The ledger cap across all runs
`AGENT_MAX_INFLIGHT · AGENT_STARTS_PER_MIN`	8 · 120	Model-call concurrency and rate
`TABLE_JUDGE_SINGLE`	openai/gpt-5.6-luna	Born-digital judge; audits; seam checks
`TABLE_JUDGE_MODELS`	google/gemini-3.1-flash-lite, openai/gpt-5.6-luna	Scanned ruled judges, cheapest first (comma separated)
`TABLE_JUDGE_THIRD (modes.py)`	google/gemini-3.5-flash-lite	Think's third judge on scanned ruled tables

The settings that are not environment variables are command-line options (chapter 4), the three modes (`modes.py MODES L57`) and the coefficients file `src/estimate_coefficients.json`.

Appendix C: Glossary

Engine
An OCR service that reads a page image: Sarvam Vision or Bodhan indic-ocr.

Primary engine
The engine whose text and layout doc.md shows for a page (Sarvam, else Bodhan, else Qwen).

Manifest
The SQLite file holding every page, task and job of one run.

Block
A piece of a page (heading, paragraph, table, figure) in the shared format.

Text layer
The text stored inside a born-digital PDF; evidence only after it proves itself.

Ruled table
A table with printed lines both ways, so the image shows its exact rows and columns.

Decider
A free rule that picks one of the two readings of a cell (cosmetic, formula, text layer, minor).

Image check
Asking a vision model about open cells with small cut-outs of the page.

Harm
A correct reading changed to a wrong one.

Seam
The place where a table continues on the next page.

Format A / C
Cut-out sheets of whole row strips (12 per sheet) or of cell snippets (24 per sheet).

Fingerprint
The settings stored with a run; a resume must match them.

Preflight
The quick look at up to 40 pages that produces the estimate.

Reading
One engine's version of a table or a text block.

Lane
The code that feeds one engine: one thread for Sarvam, a few workers for Bodhan.

Task
One (page, engine) pair, with a state.

Born digital / scan
A PDF drawn from text and lines, or a picture of paper. Decided per page.

Legacy font
A non-Unicode Indic font that stores letters under unrelated code points.

Free check
Everything in Stage 2 that needs no API call.

Open cell
A cell whose readings differ and that no free decider settled.

Audit
A blind read of a risky cell the engines agree on; it can add doubt, never a value.

Validator
A checker that chooses between existing readings and never writes a new value.

Signature
The description of a table fragment used to decide whether it joins another.

Mode
Fast, Normal or Think: how much is read and checked.

Ledger
The file of every OpenRouter call and its cost.